

# RTL Design Sherpa

# Converters Micro-Architecture Specification 1.11

**October 20, 2026**

# Table of Contents

1 Converter Mas Index....24

2 Document Information....25

2.1 Document Control....25

2.2 Purpose....26

2.3 Audience....27

2.4 Related Documents....28

2.5 Revision History....29

2.6 Conventions....32

2.6.1 Notation....32

2.6.2 Diagrams....32

2.6.3 Code Examples....32

3 Chapter 1: Introduction - Overview....33

3.1 1.1 Purpose....33

3.2 1.2 Problem Statement....34

3.2.1 1.2.1 Data Width Mismatch....34

3.2.2 1.2.2 Protocol Incompatibility....34

3.3 1.3 Solution Architecture....35

3.3.1 Figure 1.1: Converter Module Hierarchy....35

3.3.2 1.3.1 Data Width Converters....35

3.3.3 1.3.2 Protocol Converters....35

3.4 1.4 Key Design Decisions....37

3.4.1 1.4.1 Generic vs. Full Modules....37

3.4.2 1.4.2 Throughput vs. Area Trade-offs....37
3.4.3 1.4.3 Sideband Signal Handling....37
3.5 1.5 Performance Characteristics....39
3.5.1 1.5.1 Latency....39
3.5.2 1.5.2 Throughput....39
3.6 1.6 Scope....40
3.6.1 In Scope....40
3.6.2 Out of Scope....40
3.7 1.7 Target Applications....41
4 2.1 Generic Building Blocks....42
4.1 2.1.1 Module Hierarchy....43
4.2 2.1.2 Design Philosophy....44
4.2.1 Separation of Concerns....44
4.2.2 Interface Pattern....44
4.3 2.1.3 Width Ratio Calculation....45
4.4 2.1.4 Sideband Modes....46
4.4.1 Upsize Sideband Modes....46
4.4.2 Downsize Sideband Modes....46
4.5 2.1.5 Performance Comparison....48
4.5.1 Figure 2.1: Performance Comparison....48
4.6 2.1.6 Resource Utilization....49
4.6.1 Upsize Resources (NARROW=64, WIDE=512)....49
4.6.2 Downsize Resources (WIDE=512, NARROW=64)....49
4.7 2.1.7 Integration Guidelines....50

4.7.1 When to Use Generic Blocks....50
4.7.2 Composition Example....50
5 2.2 axi_data_upsize....51
5.1 2.2.1 Overview....52
5.2 2.2.2 Block Diagram....53
5.2.1 Figure 2.2: axi_data_upsize Architecture....53
5.3 2.2.3 Interface Specification....54
5.3.1 Parameters....54
5.3.2 Ports....54
5.4 2.2.4 Operation....56
5.4.1 Accumulation Cycle....56
5.4.2 Early LAST Handling....56
5.4.3 State Machine....56
5.5 2.2.5 Sideband Handling....57
5.5.1 Concatenate Mode (SB_OR_MODE=0)....57
5.5.2 Severity-Fold Mode (SB_OR_MODE=1)....57
5.6 2.2.6 Implementation....59
5.6.1 Core Logic....59
5.7 2.2.7 Timing....61
5.7.1 Latency....61
5.7.2 Throughput....61
5.7.3 Critical Paths....61
5.8 2.2.8 Resource Utilization....62
5.8.1 Typical Resources (64-bit to 512-bit)....62

5.8.2 Scaling..... 62
5.9 2.2.9 Usage Example..... 63
6 2.3 axi_data_dnsize..... 64
6.1 2.3.1 Overview..... 65
6.2 2.3.2 Block Diagram..... 66
6.2.1 Figure 2.3: axi_data_dnsize Single-Buffer Architecture..... 66
6.3 2.3.3 Interface Specification..... 67
6.3.1 Parameters..... 67
6.3.2 Ports..... 67
6.4 2.3.4 Single-Buffer Mode Operation..... 69
6.4.1 Split Cycle..... 69
6.4.2 State Machine..... 69
6.4.3 Throughput Analysis..... 69
6.5 2.3.5 Sideband Handling..... 71
6.5.1 Slice Mode (SB_BROADCAST=0)..... 71
6.5.2 Broadcast Mode (SB_BROADCAST=1)..... 71
6.6 2.3.6 Burst Tracking..... 72
6.6.1 Purpose..... 72
6.6.2 Operation..... 72
6.7 2.3.7 Implementation..... 73
6.7.1 Single-Buffer Core Logic..... 73
6.8 2.3.8 Resource Utilization..... 74
6.8.1 Single-Buffer (512-bit to 64-bit)..... 74
6.8.2 Measured Throughput..... 74

6.9 2.3.9 Usage Example....75
7 2.5 axi4_dwidth_converter_wr....76
7.1 2.5.1 Overview....77
7.2 2.5.2 Block Diagram....78
7.2.1 Figure 2.6: Write Converter Architecture....78
7.3 2.5.3 Interface Specification....79
7.3.1 Parameters....79
7.3.2 Ports....79
7.4 2.5.4 Burst Length Conversion....83
7.4.1 Ratio Calculation....83
7.4.2 Examples....83
7.4.3 Non-Aligned Bursts....83
7.5 2.5.5 Address Channel Handling....84
7.5.1 AW Passthrough with Adjustment....84
7.5.2 Skid Buffer for AW....85
7.6 2.5.6 Write Data Channel....86
7.6.1 Upsize Instance....86
7.7 2.5.7 Response Channel....87
7.7.1 B Channel....87
7.7.2 Response Ordering....87
7.8 2.5.8 AW/W Synchronization....89
7.8.1 Challenge....89
7.8.2 Solution....89
7.9 2.5.9 Resource Utilization....90

7.9.1 Typical Resources (64→512, ID=4, ADDR=64)....90
7.10 2.5.10 Timing....91
7.10.1 Latency....91
7.10.2 Throughput....91
7.11 2.5.11 Usage Example....92
8 2.6 axi4_dwidth_converter_rd....93
8.1 2.6.1 Overview....94
8.2 2.6.2 Block Diagram....95
8.2.1 Figure 2.7: Read Converter Architecture....95
8.3 2.6.3 Interface Specification....96
8.3.1 Parameters....96
8.3.2 Ports....97
8.4 2.6.4 Burst Length Conversion....100
8.4.1 Ratio Calculation....100
8.4.2 Examples....100
8.5 2.6.5 Address Channel Handling....101
8.5.1 AR Passthrough with Adjustment....101
8.5.2 Burst-Length Tracking....102
8.6 2.6.6 Read Data Channel....103
8.6.1 Downsize Instance....103
8.7 2.6.7 RLAST Generation....104
8.8 2.6.8 RID Handling....105
8.8.1 ID Tracking....105
8.8.2 Ordering Guarantee (read side; write side fixed by BUG-008)....105

8.9 2.6.9 R Interleave (R_INTERLEAVE = 1, 2026-10-18)....106
8.10 2.6.9a No-Interleave Mode (R_INTERLEAVE = 0, 2026-10-16)....107
8.11 2.6.10 Resource Utilization....108
8.11.1 Typical Resources (64→512 UPSIZE, ratio 8, ID=4)....108
8.12 2.6.10 Timing....109
8.12.1 Latency....109
8.12.2 Throughput....109
8.13 2.6.11 Usage Example....110
9 2.7 AXIL Wide-Alignment Converters....111
9.1 2.7.1 Why They Exist....112
9.2 2.7.2 Parameters....113
10 3.1 Protocol Conversion Overview....114
10.1 3.1.1 Available Converters....115
10.1.1 AXI4 to AXI4-Lite (Protocol Downgrade)....115
10.1.2 AXI4-Lite to AXI4 (Protocol Upgrade)....115
10.1.3 Other Protocol Converters....115
10.2 3.1.2 Protocol Comparison....117
10.2.1 Figure 3.1: Protocol Feature Comparison....117
10.2.2 Feature Matrix....117
10.3 3.1.3 Conversion Strategies....118
10.3.1 AXI4 to AXI4-Lite....118
10.3.2 AXI4-Lite to AXI4....118
10.3.3 AXI4 to APB....118
10.4 3.1.4 Performance Characteristics....119

10.4.1 Key Observations....119
10.5 3.1.5 Use Case Guidelines....120
10.5.1 When to Use AXI4 to AXI4-Lite....120
10.5.2 When to Use AXI4-Lite to AXI4....120
10.5.3 When to Use AXI4 to APB....120
10.6 3.1.6 Integration Patterns....121
10.6.1 Pattern 1: CPU to Peripherals....121
10.6.2 Pattern 2: Simple IP in AXI4 Fabric....121
10.6.3 Pattern 3: Mixed System....121
11 3.2 AXI4 to AXI4-Lite Converter....122
11.1 3.2.1 Module Organization....123
11.1.1 Design Philosophy....123
11.2 3.2.2 Read Path (axi4_to_axil4_rd)....124
11.2.1 Figure 3.2: AXI4 to AXI4-Lite Read Path....124
11.2.2 Operation....124
11.2.3 State Machine....125
11.2.4 Implementation....125
11.3 3.2.3 Write Path (axi4_to_axil4_wr)....127
11.3.1 Figure 3.3: AXI4 to AXI4-Lite Write Path....127
11.3.2 Operation....127
11.3.3 AW/W Synchronization Challenge....128
11.3.4 Solution: W gated on the AW that owns it....128
11.3.5 Response Aggregation....129
11.4 3.2.4 Bidirectional Wrapper (axi4_to_axil4)....130

11.4.1 Composition Pattern....130
11.5 3.2.5 Resource Utilization....131
11.6 3.2.6 Timing....132
11.6.1 Throughput....132
11.6.2 Latency....132
11.7 3.2.7 Testing....133
12 3.3 AXI4-Lite to AXI4 Converter....134
12.1 3.3.1 Module Organization....135
12.2 3.3.2 Design Philosophy....136
12.3 3.3.3 Signal Mapping....137
12.3.1 Address Channel Signals....137
12.3.2 Data Channel Signals....137
12.4 3.3.4 Read Path (axil4_to_axi4_rd)....138
12.4.1 Figure 3.4: AXI4-Lite to AXI4 Read Path....138
12.4.2 Implementation....138
12.5 3.3.5 Write Path (axil4_to_axi4_wr)....141
12.5.1 Figure 3.5: AXI4-Lite to AXI4 Write Path....141
12.5.2 Implementation....141
12.6 3.3.6 Bidirectional Wrapper....144
12.7 3.3.7 Resource Utilization....145
12.8 3.3.8 Timing....146
12.9 3.3.9 Testing....147
12.10 3.3.10 Usage Example....148
13 3.4 AXI4 to APB Converter....149

13.1 3.4.1 Overview..... 150
13.2 3.4.2 Block Diagram..... 151
13.2.1 Figure 3.6: AXI4 to APB Converter..... 151
13.3 3.4.3 Interface Specification..... 152
13.3.1 Parameters..... 152
13.3.2 Ports..... 153
13.4 3.4.4 Clock Domains..... 156
13.5 3.4.5 State Machine..... 157
13.5.1 Figure 3.7: AXI4 to APB FSM..... 157
13.5.2 States..... 157
13.6 3.4.6 Burst Handling..... 159
13.6.1 Burst Decomposition..... 159
13.6.2 Address Calculation..... 159
13.7 3.4.7 Address Width Adaptation..... 160
13.7.1 64-bit to 32-bit Conversion..... 160
13.8 3.4.8 Error Response Mapping..... 161
13.8.1 Error Aggregation..... 161
13.9 3.4.9 Implementation..... 162
13.10 3.4.10 Resource Utilization..... 163
13.11 3.4.11 Timing..... 164
13.11.1 Timing Analysis..... 164
13.11.2 Throughput..... 164
13.12 3.4.12 Usage Example..... 165
13.13 3.4.12 Design Decision: Inline Width Conversion..... 166

14 AXI4 to APB5 Shim....167
14.1 Overview....168
14.2 Parameters....169
14.3 Design Notes....170
14.4 Related Modules....171
14.5 Navigation....172
15 AXI4 to AXI5-Lite....173
15.1 Overview....174
15.2 Where each signal gets its value....175
15.3 IDs and response ordering....177
15.4 The AW/AR sideband is held, not passed through....178
15.5 Reset style....179
15.6 Verification....180
15.7 Related Modules....181
15.8 Navigation....182
16 3.5 PeakRDL Adapter....183
16.1 Overview....184
16.2 Parameters....185
16.3 Ports....186
16.4 Functional Description....188
16.4.1 Write....188
16.4.2 Read....188
16.4.3 3.5.5 Implementation....188
16.4.4 3.5.6 Resource Utilization....189

16.5 Block Diagram....190
16.5.1 Figure 3.8: PeakRDL Adapter....190
16.6 Usage Example....191
16.6.1 3.5.7 Use Cases....191
16.6.2 3.5.8 Integration Example....191
16.6.3 Figure 3.9: CSRs Behind the APB Shim....192
16.7 Related Modules....194
16.7.1 3.5.9 APB4 Front End (apb4_to_peakrdl)....194
17 3.7 UART to AXI4-Lite Bridge....197
17.1 3.7.1 Module Organization....198
17.2 3.7.2 Command Protocol....199
17.3 3.7.3 Error Responses....200
17.4 3.7.4 Parameters....201
17.5 3.7.5 Ports....202
17.5.1 uart_axil_bridge....202
17.5.2 uart_rx....203
17.5.3 uart_tx....203
18 3.8 Width-plus-Protocol Chains....205
18.1 3.8.1 axi4_dwidth_to_axil4_wr_chain....206
18.2 3.8.2 axi4_dwidth_to_apb4_chain....207
19 3.10 AXI4-Lite to Wishbone B4 Converter....209
19.1 3.10.1 Module Organization....210
19.2 3.10.2 Design Philosophy....211
19.3 3.10.3 Block Diagram....212

19.3.1 Figure 3.12: AXI4-Lite to Wishbone B4 Converter....212
19.4 3.10.4 Interface Specification....213
19.4.1 Parameters....213
19.4.2 Ports....213
19.5 3.10.5 Command Formation....215
19.6 3.10.6 Response Mapping....216
19.7 3.10.7 Timing....217
19.8 3.10.8 Formal....218
19.9 3.10.9 Testing....219
19.10 3.10.10 Usage Example....220
20 3.11 Wishbone B4 to AXI4-Lite Converter....221
20.1 3.11.1 Module Organization....222
20.2 3.11.2 Why This Direction Is Harder....223
20.3 3.11.3 Interface Specification....224
20.3.1 Parameters....224
20.3.2 Ports....224
20.4 3.11.4 Command Formation....226
20.5 3.11.5 Response Mapping....227
20.6 3.11.6 Formal....228
20.7 3.11.7 Testing....229
20.8 3.11.8 Usage Example....230
20.9 Navigation....231
21 APB to AXI4 (Requester)....232
21.1 Overview....233

21.2 Fields the requester invents....234
21.3 APB5 sideband....235
21.4 Parameters....236
21.5 Verification....237
21.6 Related Modules....238
21.7 Navigation....239
22 AXI4 to Wishbone B4....240
22.1 Overview....241
22.2 Parameters....242
22.3 Verification....243
22.4 Related Modules....244
22.5 Navigation....245
23 Wishbone B4 to AXI4....246
23.1 Overview....247
23.2 Parameters....248
23.3 Verification....249
23.4 Related Modules....250
23.5 Navigation....251
24 3.15 AXI4 to AXI3 Converter....252
24.1 3.15.1 Module Organization....253
24.2 3.15.2 Protocol Deltas....254
24.2.1 Exclusive-access caveat....254
24.3 3.15.3 Read Path (axi4_to_axi3_rd)....255
24.3.1 Operation....255

24.3.2 Per-ID Merge Isolation....255
24.3.3 Address Advance....255
24.4 3.15.4 Write Path (axi4_to_axi3_wr)....256
24.4.1 Operation....256
24.4.2 Early-W Buffering....256
24.4.3 Storage....256
24.4.4 Request Flow and Per-ID Ordering....256
24.5 3.15.5 Bidirectional Wrapper (axi4_to_axi3)....258
24.6 3.15.6 Parameters....259
24.7 3.15.7 Non-goals....260
24.8 3.15.8 Resource Utilization....261
24.9 3.15.9 Testing....262
24.10 References....264
25 3.16 AXI3 to AXI4 Converter....265
25.1 3.16.1 Module Organization....266
25.2 3.16.2 Protocol Deltas....267
25.2.1 Exclusive-access caveat....267
25.3 3.16.3 Read Path (axi3_to_axi4_rd)....268
25.4 3.16.4 Write Path (axi3_to_axi4_wr)....269
25.4.1 Operation....269
25.4.2 Write Ordering Contract....269
25.4.3 WID Order Check....269
25.5 3.16.5 Bidirectional Wrapper (axi3_to_axi4)....270
25.6 3.16.6 Parameters....271

25.7 3.16.7 Non-goals....272
25.8 3.16.8 Resource Utilization....273
25.9 3.16.9 Testing....274
25.10 References....275
26 3.17 AXI-Stream to CXS Adapters....276
26.1 3.17.1 Module Organization....277
26.2 3.17.2 Field Mapping....278
26.3 3.17.3 Interleaved Packets....279
26.4 3.17.4 Parameters....280
26.5 3.17.5 Limits....281
26.6 3.17.6 Used By and Verification....282
27 4.1 Width Converter FSMs....283
27.1 Overview....283
27.2 Functional Description....284
27.2.1 4.1.1 Upsize Structure (no FSM)....284
27.2.2 4.1.2 Downsize Structure (single buffer, no FSM)....284
27.2.3 4.1.3 Full Converter FSMs....285
27.3 Timing....286
27.3.1 Upsize (8:1 ratio, m_ready held high)....286
27.3.2 Figure 4.1: Upsize 8:1 Timing....286
27.3.3 Downsize (8:1 ratio, single buffer, atomic replace)....286
27.3.4 Figure 4.2: Downsize 8:1 Timing....287
28 4.2 Protocol Converter FSMs....288
28.1 Overview....288

28.2 Functional Description....289
28.2.1 4.2.1 AXI4 to AXI4-Lite Read FSM....289
28.2.2 4.2.2 AXI4 to AXI4-Lite Write FSM....289
28.2.3 4.2.3 AXI4 to APB FSMs....289
28.3 Timing....290
28.3.1 4.2.4 Timing Analysis....290
29 4.3 Burst Decomposition....291
29.1 Overview....291
29.2 Functional Description....292
29.2.1 4.3.1 Width Converter Burst Handling....292
29.2.2 4.3.2 Protocol Converter Burst Handling....292
29.2.3 4.3.3 Response Aggregation....293
29.2.4 4.3.4 Burst Tracking Registers....294
29.3 Timing....296
29.3.1 4.3.5 Timing Impact....296
29.4 Waveforms....297
29.4.1 Figure 4.3: Width Burst Conversion....297
29.5 Navigation....298
30 5.1 Test Strategy....299
30.1 Overview....299
30.2 Testing....300
30.2.1 5.1.1 Test Organization....300
30.2.2 5.1.2 Test Levels....300
30.2.3 5.1.3 Test Categories....301

30.2.4 5.1.4 Testbench Architecture....302
30.2.5 5.1.5 Coverage Model....303
30.2.6 5.1.6 Test Execution....304
30.3 Navigation....305
31 5.2 Debug Guide....306
31.1 Overview....306
31.2 Testing....307
31.2.1 5.2.1 Common Issues....307
31.2.2 5.2.2 Debug Signals....308
31.2.3 5.2.3 Simulation Debug....309
31.2.4 Figure 5.1: Upsize Reference Timing....310
31.2.5 5.2.4 Common Mistakes....310
31.2.6 5.2.5 Verification Checklist....311
31.2.7 5.2.6 Performance Validation....311
31.3 Navigation....313

# List of Figures

Figure 1.1: Converter Module Hierarchy....35
Figure 2.1: Performance Comparison....48
Figure 2.2: axi_data_upsize Architecture....53
Figure 2.3: axi_data_dnsize Single-Buffer Architecture....66
Figure 2.6: Write Converter Architecture....78
Figure 2.7: Read Converter Architecture....95
Figure 3.1: Protocol Feature Comparison....117
Figure 3.2: AXI4 to AXI4-Lite Read Path....124
Figure 3.3: AXI4 to AXI4-Lite Write Path....127
Figure 3.4: AXI4-Lite to AXI4 Read Path....138
Figure 3.5: AXI4-Lite to AXI4 Write Path....141
Figure 3.6: AXI4 to APB Converter....151
Figure 3.7: AXI4 to APB FSM....157
Figure 3.8: PeakRDL Adapter....189
Figure 3.9: CSRs Behind the APB Shim....191
Figure 3.12: AXI4-Lite to Wishbone B4 Converter....211
Figure 4.1: Upsize 8:1 Timing....285
Figure 4.2: Downsize 8:1 Timing....286
Figure 4.3: Width Burst Conversion....296
Figure 5.1: Upsize Reference Timing....309

# List of Tables

Table 1: Table 0.1: Document Control Information....25
Table 2: Table 0.2: Related Documents....28
Table 3: Table 0.3: Revision History....29
Table 4: Table 1.1: Common Data Width Configurations....34
Table 5: Table 1.2: Protocol Mismatch Examples....34
Table 6: Table 1.3: Throughput vs. Area Trade-offs....37
Table 7: Table 1.4: Sideband Handling Modes....37
Table 8: Table 1.5: Latency Characteristics....39
Table 9: Table 1.6: Throughput Characteristics....39
Table 10: Table 2.1: Upsize Sideband Modes....46
Table 11: Table 2.2: Downsize Sideband Modes....46
Table 12: Table 2.3: Performance Comparison....48
Table 13: Table 2.4: axi_data_upsize Parameters....54
Table 14: Table 2.5: Upsize Latency....61
Table 15: Table 2.6: Upsize Resource Scaling....62
Table 16: Table 2.7: axi_data_dnsize Parameters....67
Table 17: Table 2.8: Measured Throughput by Mode....70
Table 18: Table 2.9: Resources and measured rate....74
Table 19: Table 2.14: Write Converter Parameters....79
Table 20: Table 2.15: Burst Length Conversion Examples....83
Table 21: Table 2.16: Write Converter Latency....91
Table 22: Table 2.17: Read Converter Parameters....96
Table 23: Table 2.18: Read Burst Length Conversion....100
Table 24: Table 2.20: Read Converter Latency....109
Table 25: Wide-Alignment Converter Parameters....113
Table 26: Table 3.1: AXI4 to AXI4-Lite Converters....115
Table 27: Table 3.2: AXI4-Lite to AXI4 Converters....115
Table 28: Table 3.3: Other Protocol Converters....115
Table 29: Table 3.4: Protocol Feature Comparison....117
Table 30: Table 3.5: Protocol Converter Performance....119
Table 31: Table 3.6: AXI4 to AXIL4 Resources....131
Table 32: Table 3.7: AXI4 to AXIL4 Throughput....132
Table 33: Table 3.8: AXI4 to AXIL4 Latency....132
Table 34: Table 3.9: Test Coverage Summary....133
Table 35: Table 3.10: AR Channel Mapping....137
Table 36: Table 3.11: R Channel Mapping....137

Table 37: Table 3.12: AXIL4 to AXI4 Resources....145
Table 38: Table 3.13: AXIL4 to AXI4 Performance....146
Table 39: Table 3.14: Test Coverage Summary....147
Table 40: Table 3.15: AXI4 vs APB Comparison....150
Table 41: Table 3.16: AXI4 to APB Parameters....152
Table 42: Table 3.17: AXI4 to APB Shim Parameters....152
Table 43: Table 3.18: Error Mapping....161
Table 44: Table 3.19: APB Converter Timing....164
Table 45: AXI4 to APB5 Shim – Inert Parameter....169
Table 46: AXI4 to AXI5-Lite – Signal Disposition....175
Table 47: Table 3.20: PeakRDL Adapter Parameters....184
Table 48: apb4_to_peakrdl Parameters....193
Table 49: apb4_to_peakrdl Ports....194
Table 50: uart_axil_bridge Parameters....200
Table 51: uart_axil_bridge Ports....201
Table 52: uart_rx Ports....202
Table 53: uart_tx Ports....202
Table 54: axi4_dwidth_to_axil4_wr_chain Parameters....205
Table 55: axi4_dwidth_to_apb4_chain Parameters....206
Table 56: Table 3.26: AXI4-Lite to Wishbone Parameters....212
Table 57: Table 3.27: AXI4-Lite to Wishbone Ports....212
Table 58: Table 3.28: Command Formation....214
Table 59: Table 3.29: Response Mapping....215
Table 60: Table 3.30: AXI4-Lite to Wishbone Timing....216
Table 61: Table 3.31: Wishbone to AXI4-Lite Parameters....223
Table 62: Table 3.32: Wishbone to AXI4-Lite Ports....223
Table 63: Table 3.33: Command Formation....225
Table 64: Table 3.34: Response Mapping....226
Table 65: Table 3.35: APB-to-AXI4 requester pieces....232
Table 66: Table 3.36: AXI4 request fields from an APB transfer....233
Table 67: Table 3.37: APB5 USER mapping....234
Table 68: Table 3.38: APB-to-AXI4 parameters....235
Table 69: Table 3.39: axi4_to_wb4 stages....240
Table 70: Table 3.40: axi4_to_wb4 parameters....241
Table 71: Table 3.41: AXI4 fields the wrapper invents....246
Table 72: Table 3.45: AXI4 to AXI3 protocol differences....253
Table 73: Table 3.46: AXI4 to AXI3 parameters....258
Table 74: Table 3.47: AXI4 to AXI3 resources....260

Table 75: Table 3.48: AXI4 to AXI3 test coverage....261
Table 76: Table 3.49: AXI3 to AXI4 protocol differences....266
Table 77: Table 3.50: AXI3 to AXI4 parameters....270
Table 78: Table 3.51: AXI3 to AXI4 resources....272
Table 79: Table 3.52: AXI3 to AXI4 test coverage....273
Table 80: Table 3.53: AXI-Stream to CXS field mapping....277
Table 81: Table 3.54: AXI-Stream to CXS adapter parameters....279
Table 82: Table 3.55: AXI-Stream to CXS adapter users....281
Table 83: Table 4.6: Decomposition Overhead....295
Table 84: Table 5.1: Functional Test Categories....300
Table 85: Table 5.2: Stress Test Categories....300
Table 86: Table 5.3: Error Test Categories....300
Table 87: Table 5.4: Coverage Targets....302
Table 88: Table 5.5: Expected Throughput....311

# 1 Converter Mas Index

**Generated:** 2026-10-20

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 2 Document Information

## 2.1 Document Control

*Table 0.1: Document Control Information*

| Field | Value |
|---|---|
| Document Title | Converters Micro-Architecture Specification |
| Document Version | 1.11 |
| Component | Converters |
| Status | Active |
| Classification | Internal Technical |
| Last Updated | 2026-10-20 |

## 2.2 Purpose

This Micro-Architecture Specification (MAS) describes how the Converters component is actually built — the internal detail behind the feature set. It covers:

- Internal block architectures
- State machine designs
- Signal timing and handshaking
- Resource utilization estimates
- Debug and verification strategies

## 2.3 Audience

This document is for:

- RTL designers implementing or modifying converter modules
- Verification engineers creating testbenches
- Integration engineers connecting converters to systems
- Performance engineers optimizing throughput and latency

## 2.4 Related Documents

*Table 0.2: Related Documents*

| Document | Purpose |
|---|---|
| Bridge MAS | Related crossbar micro-architecture |
| Stream MAS | Related datapath micro-architecture |

(The old converter spec tree was migrated into this MAS, and converters has no standalone PRD — this book is the authoritative document.)

## 2.5 Revision History

*Table 0.3: Revision History*

| Version | Date | Author | Description |
|---|---|---|---|
| 1.0 | 2026-01-03 | RTL Design Sherpa | Initial MAS release |
| 1.2 | 2026-09-09 | RTL Design Sherpa | Added chapter 3.10, AXI4-Lite to Wishbone B4 |
| 1.8 | 2026-10-18 | RTL Design Sherpa | 3.11: wb4_to_axil4 keeps memory order, not only termination order – a command waits for the other direction to drain (a read behind a slow write to the same word returned old data), with a directed hazard probe. 3.15: axi4_to_axi3 stages a chopped request and checks its ID from registers, and runs the B merge from the registered BID, so neither the incoming request nor BID is in series with per-ID state (bridge W13); two |

| Version | Date | Author | Description |
|---|---|---|---|
| | | | directed same-ID tests. |
| 1.9 | 2026-10-18 | RTL Design Sherpa | 2.6 axi4_dwidth_converter_rd: R interleave without the beat pool. With `R_INTERLEAVE = 1` (the default, and what the bridge builds) each R beat is converted against its own ID's state as it arrives: downsize keeps a partial wide beat per ID in LUTRAM, upsize holds one wide beat and a narrow-beat count per ID. RASM_DEPTH / RASM_MAX_OUTSTANDING are unused. |
| 1.10 | 2026-10-20 | RTL Design Sherpa | Block diagrams redrawn to the repo element key (v3): class fills, double-bordered registered blocks, green return paths and a generated key |

| Version | Date | Author | Description |
|---|---|---|---|
| | | | under each figure. No technical content changed. |
| 1.11 | 2026-10-20 | RTL Design Sherpa | Section (h2) headings now open a new page, the pipeline default. No technical content changed. |

## 2.6 Conventions

### 2.6.1 Notation

- `signal_name` - RTL signals and parameters
- **ModuleName** - Module names and key concepts
- *Figure X.X* - Figure references

### 2.6.2 Diagrams

Block and microarchitecture figures are graphviz, rendered to PNG: - Source: `assets/graphviz/*.dot` - Rendered: `assets/graphviz/*.png`

State charts, sequence charts and comparison cards are Mermaid: - Source: `assets/mermaid/*.mmd` - Rendered: `assets/mermaid/*.png`

### 2.6.3 Code Examples

Treat the SystemVerilog snippets as implementation guidance: they show the intended design pattern but may differ slightly from the actual RTL.

---

**Next:** Chapter 1: Introduction

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 3 Chapter 1: Introduction - Overview

## 3.1 1.1 Purpose

The Converters component provides data width and protocol conversion — the modules that let blocks with different data widths or communication protocols talk to each other.

## 3.2 1.2 Problem Statement

Sooner or later, every SoC design runs into the same two integration problems:

### 3.2.1 1.2.1 Data Width Mismatch

*Table 1.1: Common Data Width Configurations*

| Component | Typical Data Width |
|---|---|
| CPU | 64-bit |
| DDR Controller | 512-bit |
| PCIe Endpoint | 128-bit |
| GPU | 256-bit |

**Challenge:** You can't wire mismatched widths together directly — a width converter has to sit between them.

### 3.2.2 1.2.2 Protocol Incompatibility

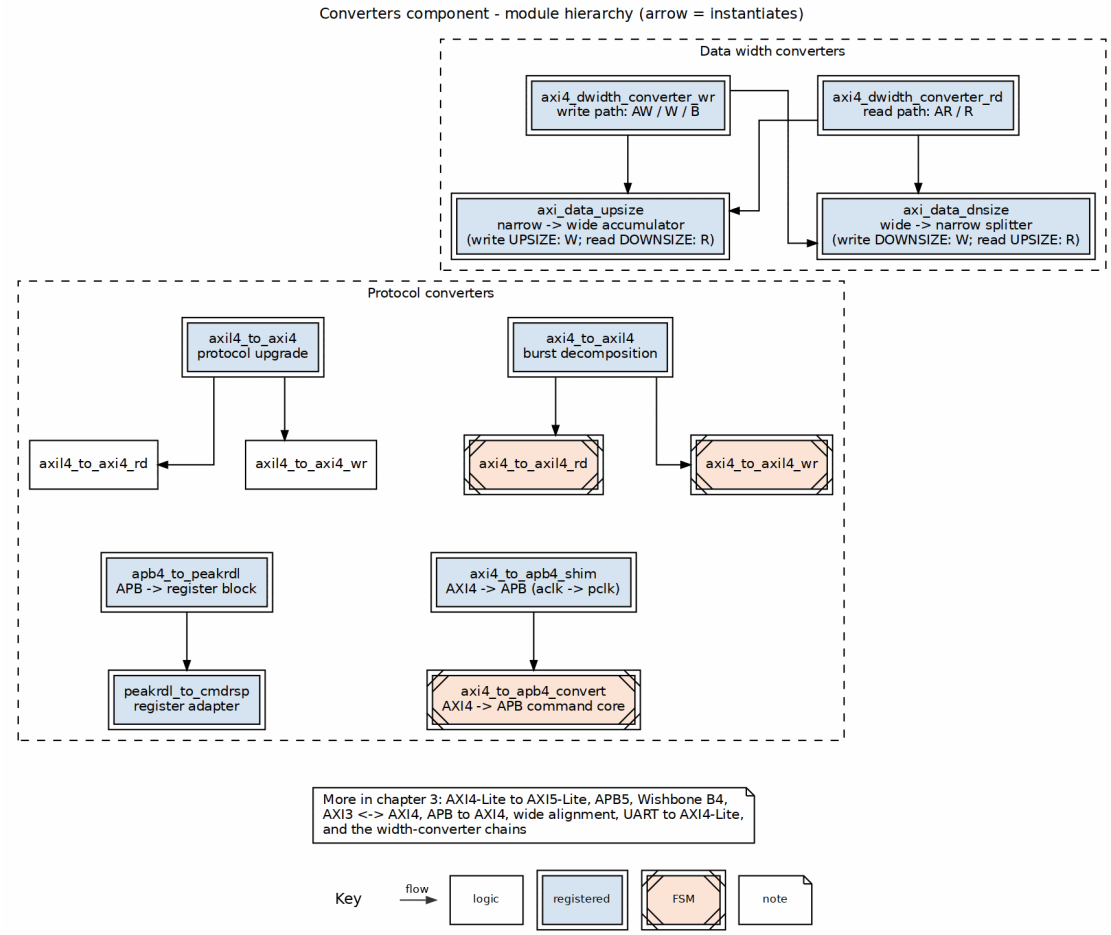
*Table 1.2: Protocol Mismatch Examples*

| Master Type | Protocol | Slave Type | Protocol |
|---|---|---|---|
| CPU | AXI4 | DDR | AXI4 |
| DMA | AXI4 | UART | APB |
| CPU | AXI4 | GPIO | APB |
| Custom IP | AXI4-Lite | Fabric | AXI4 |

**Challenge:** Different protocols can't exchange a single transfer without a bridge between them.

## 3.3 1.3 Solution Architecture

### 3.3.1 Figure 1.1: Converter Module Hierarchy




*Converter Module Hierarchy*

### 3.3.2 1.3.1 Data Width Converters

**Generic Building Blocks:** - **axi_data_upsize** - Accumulates N narrow beats into 1 wide beat - **axi_data_dnsize** - Splits 1 wide beat into N narrow beats

**Full AXI4 Converters:** - **axi4_dwidth_converter_wr** - Complete write path (AW + W + B channels) - **axi4_dwidth_converter_rd** - Complete read path (AR + R channels)

### 3.3.3 1.3.2 Protocol Converters

**AXI4 to AXI4-Lite:** - **axi4_to_axil4_rd** - Read path burst decomposition - **axi4_to_axil4_wr** - Write path burst decomposition - **axi4_to_axil4** - Full bidirectional wrapper

**AXI4-Lite to AXI4:** - **axil4_to_axi4_rd** - Read path protocol upgrade - **axil4_to_axi4_wr** - Write path protocol upgrade - **axil4_to_axi4** - Full bidirectional wrapper

**Other Converters:** - **axi4_to_apb4_convert** - Full AXI4-to-APB bridge - **peakrdl_to_cmdrsp** - Register interface adapter

## 3.4 1.4 Key Design Decisions

### 3.4.1 1.4.1 Generic vs. Full Modules

The converters are built in three layers:

```
Layer 1: Generic Building Blocks
         axi_data_upsize, axi_data_dnsize
         - Protocol-agnostic data manipulation
         - Reusable across different contexts

Layer 2: Full AXI4 Converters
         axi4_dwidth_converter_wr, axi4_dwidth_converter_rd
         - Compose generic blocks with AXI4 channel management
         - Handle burst length adjustment, ID tracking

Layer 3: Protocol Converters
         axi4_to_axil4, axi4_to_apb4, etc.
         - Full protocol translation
         - State machine control
```

### 3.4.2 1.4.2 Throughput vs. Area Trade-offs

*Table 1.3: Throughput vs. Area Trade-offs*

| Configuration | Throughput | Area | Use Case |
|---|---|---|---|
| Upsize (single buffer) | 100% | 1x | All narrow-to-wide |
| Downsize (single buffer) | 0.992 beats/cycle | 1x | Area-constrained |

**Design Decision:** One buffer per direction. The downsize accepts its next wide beat during the last narrow beat of the current one — measured at 0.992 beats/cycle — so a second buffer has nothing to recover. A ping-pong mode existed for this and was removed once the single buffer was fixed.

### 3.4.3 1.4.3 Sideband Signal Handling

Sideband signals (WSTRB, RRESP, etc.) support three handling modes:

*Table 1.4: Sideband Handling Modes*

| Mode | Upsize Behavior | Downsize Behavior |
|---|---|---|
| Concatenate | Pack narrow strobes | Slice wide strobes |
| Broadcast | N/A | Repeat value |

| Mode | Upsize Behavior | Downsize Behavior |
|---|---|---|
| Severity fold | Keep numeric max | N/A |

## 3.5 1.5 Performance Characteristics

### 3.5.1 1.5.1 Latency

*Table 1.5: Latency Characteristics*

| Module | Single-Beat | Burst (N beats) |
|---|---|---|
| axi_data_upsize | 1 cycle (registered output) | RATIO narrow beats to fill a group |
| axi_data_dnsize (single) | 1 cycle | N cycles (replacement wide beat accepted during the last narrow beat — no gap) |
| axi4_to_axil4 | 0 cycles | slave-limited (see 3.2.6) |
| axil4_to_axi4 | 0 cycles | N/A (single only) |

### 3.5.2 1.5.2 Throughput

*Table 1.6: Throughput Characteristics*

| Module | Configuration | Peak Throughput |
|---|---|---|
| axi_data_upsize | Single buffer | 1 beat/cycle |
| axi_data_dnsize | Single buffer | 0.992 beats/cycle (measured, ratio 4) |
| axi4_to_axil4 | Burst | slave-limited; requests stream independently of responses |
| axil4_to_axi4 | Any | 1 beat/cycle |

## 3.6 1.6 Scope

### 3.6.1 In Scope

- Integer width ratios (2:1, 4:1, 8:1, 16:1, etc.)
- AXI4, AXI4-Lite, and APB protocol support
- Configurable throughput vs. area trade-offs
- Generic building blocks for custom converters
- Burst-aware width conversion

### 3.6.2 Out of Scope

- Non-integer width ratios (e.g., 3:2 conversion)
- AXI4-Stream protocol (see Stream component)
- Buffering deeper than one beat per direction
- Clock domain crossing (use separate CDC modules)
- Address translation (handled by crossbar)

## 3.7 1.7 Target Applications

1. **CPU-to-DDR Integration** - 64-bit CPU to 512-bit memory controller
2. **DMA Engines** - Variable width data movers
3. **Mixed Protocol Systems** - AXI4 fabric with APB peripheral bus
4. **FPGA Fabric Interfaces** - Width matching for IP integration
5. **Register Access** - PeakRDL to custom control protocols

---

**Next:** Chapter 2: Data Width Converter Blocks

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 4 2.1 Generic Building Blocks

Two generic building blocks handle the data movement: **axi_data_upsize** (narrow-to-wide) and **axi_data_dnsize** (wide-to-narrow). Both are protocol-agnostic — compose them into full AXI4 converters or use them directly in custom designs.

## 4.1 2.1.1 Module Hierarchy

```
Generic Building Blocks
|
+-- axi_data_upsize.sv      # Narrow-to-Wide accumulator
|   +-- Accumulator buffer
|   +-- Beat counter
|   +-- Sideband packing
|
+-- axi_data_dnsize.sv      # Wide-to-Narrow splitter
    +-- Data buffer
    +-- Beat counter
    +-- Sideband extraction
    +-- Optional burst tracking
```

## 4.2 2.1.2 Design Philosophy

### 4.2.1 Separation of Concerns

The generic blocks handle **only data manipulation**: - Data packing/unpacking - Sideband signal handling - Valid/ready handshaking

They do **not** handle: - Address manipulation - Burst length adjustment - ID tracking - Protocol-specific signals (ARLEN, AWLEN, etc.)

That separation buys you three things: - Reuse in multiple contexts - Simpler verification - Cleaner composition into full converters

### 4.2.2 Interface Pattern

Both modules use a consistent valid/ready interface:

```
// Input (narrow for upsize, wide for dnsize)
input  logic                     i_valid,
output logic                     o_ready,
input  logic [DATA_WIDTH-1:0]    i_data,
input  logic [SB_WIDTH-1:0]      i_sideband,
input  logic                     i_last,      // Optional

// Output (wide for upsize, narrow for dnsize)
output logic                     o_valid,
input  logic                     i_ready,
output logic [DATA_WIDTH-1:0]    o_data,
output logic [SB_WIDTH-1:0]      o_sideband,
output logic                     o_last       // Optional
```

## 4.3 2.1.3 Width Ratio Calculation

Both modules calculate the width ratio at elaboration time:

```
localparam int RATIO = WIDE_WIDTH / NARROW_WIDTH;
localparam int RATIO_LOG2 = $clog2(RATIO);

// Example: 64-bit to 512-bit
// RATIO = 512 / 64 = 8
// RATIO_LOG2 = 3
```

**Constraints:** - `WIDE_WIDTH` must be an integer multiple of `NARROW_WIDTH` - Minimum ratio: 2 - Maximum ratio: Typically 16 (limited by timing)

## 4.4 2.1.4 Sideband Modes

### 4.4.1 Upsize Sideband Modes

*Table 2.1: Upsize Sideband Modes*

| Mode | Parameter | Behavior | Use Case |
|---|---|---|---|
| Concatenate | SB_OR_MODE=0 | Pack N narrow sidebands | WSTRB packing |
| Severity fold | SB_OR_MODE=1 | Keep the numeric max of the narrow sidebands | RRESP worst-case fold |

**Concatenate Example (WSTRB):**

```
8 beats of 8-bit WSTRB → 1 beat of 64-bit WSTRB
[0]: 0xFF → output[7:0]   = 0xFF
[1]: 0xF0 → output[15:8]  = 0xF0
...
[7]: 0x0F → output[63:56] = 0x0F
```

**OR Example (Error flags):**

```
8 beats of error flags → 1 beat with any error
[0]: 0 → output = 0
[1]: 1 → output = 1 (error detected)
...
[7]: 0 → output remains 1
```

### 4.4.2 Downsize Sideband Modes

*Table 2.2: Downsize Sideband Modes*

| Mode | Parameter | Behavior | Use Case |
|---|---|---|---|
| Slice | SB_BROADCAST=0 | Extract slice per beat | WSTRB extraction |
| Broadcast | SB_BROADCAST=1 | Repeat full value | RRESP broadcast |

**Slice Example (WSTRB):**

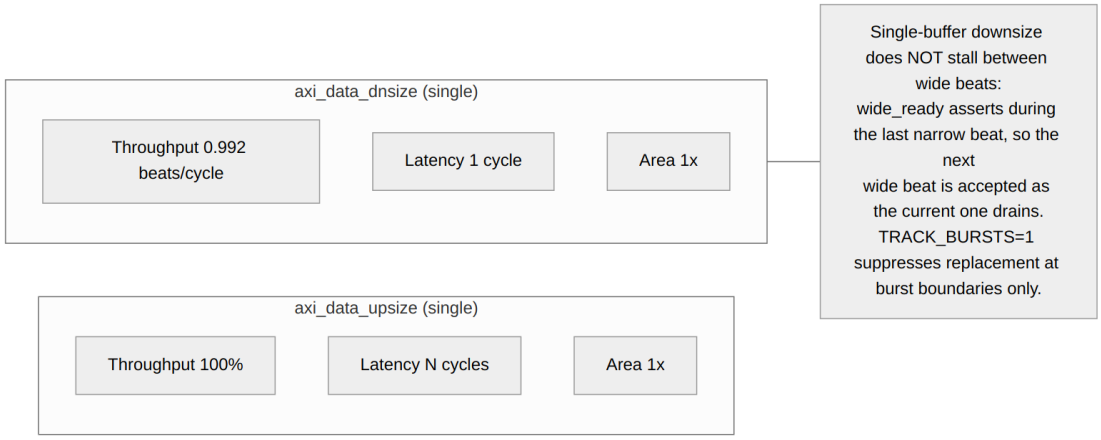
```
1 beat of 64-bit WSTRB → 8 beats of 8-bit WSTRB
input = 0xFF_F0_..._0F
[0]: output = 0xFF (bits [7:0])
[1]: output = 0xF0 (bits [15:8])
...
[7]: output = 0x0F (bits [63:56])
```

**Broadcast Example (RRESP):**

```
1 beat of 2-bit RRESP → 8 beats of 2-bit RRESP
input = OKAY (2'b00)
[0-7]: output = OKAY (all beats get same response)
```

## 4.5 2.1.5 Performance Comparison

### 4.5.1 Figure 2.1: Performance Comparison




*Performance Comparison*

*Table 2.3: Performance Comparison*

| Module | Mode | Throughput | Latency | Area |
|---|---|---|---|---|
| axi_data_upsize | Single | 100% | N cycles | 1x |
| axi_data_dnsize | Single | 0.992 beats/cycle | 1 cycle | 1x |

**Single-buffer downsize does not stall between wide beats.**

`wide_ready` is asserted during the last narrow beat, so the next wide beat is accepted as the current one finishes draining. With `TRACK_BURSTS=1` the replacement is suppressed at burst boundaries only.

(The former dual-buffer mode is deleted; the atomic replace above is the whole story.)

## 4.6 2.1.6 Resource Utilization

### 4.6.1 Upsize Resources (NARROW=64, WIDE=512)

```
Accumulator buffer:   512 bits (output register)
Beat counter:         3 bits (clog2(8))
Sideband accumulator: 64 bits (WSTRB)
Control logic:        ~50 LEs

Total: ~70-100 LEs, ~580 registers
```

### 4.6.2 Downsize Resources (WIDE=512, NARROW=64)

**Single Buffer:**

```
Data buffer:          512 bits
Beat counter:         3 bits
Sideband logic:       ~20 LEs
Control logic:        ~50 LEs

Total: ~70-100 LEs, ~520 registers
```

## 4.7 2.1.7 Integration Guidelines

### 4.7.1 When to Use Generic Blocks

**Use directly when:** - Building custom data pipelines - Data width conversion without AXI4 protocol - Simple valid/ready streaming interfaces

**Use full converters when:** - Need AXI4 channel management (AW, W, B, AR, R) - Burst length adjustment required - ID tracking needed

### 4.7.2 Composition Example

```
// Full AXI4 write path composition
axi4_dwidth_converter_wr
#(.S_AXI_DATA_WIDTH(32), .M_AXI_DATA_WIDTH(128)) u_wr (
    // This module internally instantiates:
    // 1. Address phase skid buffer
    // 2. axi_data_upsize for write data
    // 3. Response path passthrough
    // 4. Burst length adjustment logic
);
```

---

**Next:** axi_data_upsize Module



---

# 5 2.2 axi_data_upsize

The **axi_data_upsize** module accumulates N narrow beats into 1 wide beat. It's the core building block for narrow-to-wide data width conversion.

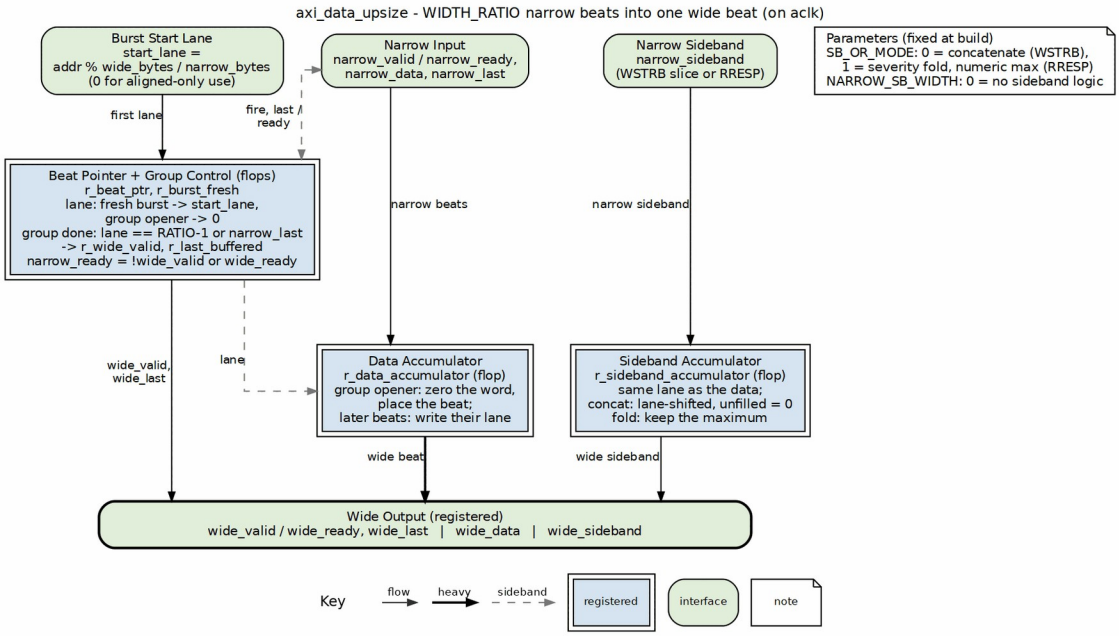
## 5.1 2.2.1 Overview

The upsize module does four things:

1. **Data Accumulation**: Collects N narrow data beats into accumulator buffer
2. **Sideband Packing**: Concatenates or ORs sideband signals (WSTRB, etc.)
3. **Flow Control**: Manages valid/ready handshaking with single-cycle latency
4. **LAST Tracking**: Detects input LAST to generate output LAST

## 5.2 2.2.2 Block Diagram

### 5.2.1 Figure 2.2: axi_data_upsize Architecture




*axi_data_upsize Architecture*

## 5.3 2.2.3 Interface Specification

### 5.3.1 Parameters

*Table 2.4: axi_data_upsize Parameters*

| Parameter | Type | Default | Description |
|---|---|---|---|
| NARROW_WIDTH | int | 32 | Input data width (bits) |
| WIDE_WIDTH | int | 128 | Output data width (bits) |
| NARROW_SB_WIDTH | int | 0 | Input sideband width (bits), 0 if unused |
| WIDE_SB_WIDTH | int | 0 | Output sideband width, 0 if unused |
| SB_OR_MODE | int | 0 | 0=concatenate, 1=severity fold (numeric max) |

### 5.3.2 Ports

```
module axi_data_upsize #(
    // Width Configuration
    parameter int NARROW_WIDTH    = 32,
    parameter int WIDE_WIDTH      = 128,
    parameter int NARROW_SB_WIDTH = 0,        // Sideband width (0 if
unused)
    parameter int WIDE_SB_WIDTH   = 0,        // Wide sideband width
    parameter int SB_OR_MODE      = 0,        // 0=concatenate,
1=severity fold (numeric max)

    // Calculated Parameters
    localparam int WIDTH_RATIO = WIDE_WIDTH / NARROW_WIDTH,
    localparam int PTR_WIDTH   = $clog2(WIDTH_RATIO),
    // Ensure sideband widths are at least 1 for port declarations
(unused if actual width is 0)
    localparam int NARROW_SB_PORT_WIDTH = (NARROW_SB_WIDTH > 0) ?
NARROW_SB_WIDTH : 1,
    localparam int WIDE_SB_PORT_WIDTH = (WIDE_SB_WIDTH > 0) ?
WIDE_SB_WIDTH : 1
```

```
) (
    input  logic                                aclk,
    input  logic                                aresetn,

    // Narrow Input (from slave or master)
    input  logic                                narrow_valid,
    output logic                                narrow_ready,
    input  logic [NARROW_WIDTH-1:0]             narrow_data,
    input  logic [NARROW_SB_PORT_WIDTH-1:0] narrow_sideband,  // Min
width 1 to avoid [-1:0]
    input  logic                                narrow_last,

    // Lane the FIRST narrow beat of a burst occupies inside the wide
    // word (addr % wide_bytes / narrow_bytes); later groups start at
    // lane 0. Tie '0 for aligned-only behavior.
    input  logic [PTR_WIDTH-1:0]                start_lane,

    // Wide Output (to master or slave)
    output logic                                wide_valid,
    input  logic                                wide_ready,
    output logic [WIDE_WIDTH-1:0]               wide_data,
    output logic [WIDE_SB_PORT_WIDTH-1:0]   wide_sideband,  // Min
width 1 to avoid [-1:0]
    output logic                                wide_last
);
```

## 5.4 2.2.4 Operation

### 5.4.1 Accumulation Cycle

```
Cycle 0: s_data[0] → buffer[63:0],   count = 0 → 1
Cycle 1: s_data[1] → buffer[127:64], count = 1 → 2
...
Cycle 7: s_data[7] → buffer[511:448], count = 7 → 0, m_valid = 1
Cycle 8: m_ready handshake, output complete
```

### 5.4.2 Early LAST Handling

If `s_last` arrives before buffer is full:

```
Cycle 0: s_data[0] → buffer[63:0],   count = 0 → 1
Cycle 1: s_data[1] + s_last → buffer[127:64], count = 1 → 0
         m_valid = 1, m_last = 1
         Remaining bytes = don't care (masked by WSTRB)
```

### 5.4.3 State Machine

```
IDLE (count=0):
  - s_valid=1 → load beat, increment count
  - count < RATIO-1 → stay in IDLE
  - count = RATIO-1 OR s_last → OUTPUT

OUTPUT (m_valid=1):
  - m_ready=1 → clear buffer, → IDLE
  - m_ready=0 → hold output
```

## 5.5 2.2.5 Sideband Handling

### 5.5.1 Concatenate Mode (SB_OR_MODE=0)

Used for WSTRB packing:

```
// Pack narrow sidebands into wide sideband
always_ff @(posedge clk) begin
    if (s_valid && s_ready) begin
        r_sideband[r_count * NARROW_SB_WIDTH +: NARROW_SB_WIDTH] <=
s_sideband;
    end
end
```

**Example**: 8 beats of 8-bit WSTRB to 64-bit WSTRB

```
Beat 0: WSTRB = 0xFF → output[7:0]   = 0xFF
Beat 1: WSTRB = 0xF0 → output[15:8]  = 0xF0
Beat 2: WSTRB = 0x0F → output[23:16] = 0x0F
...
Beat 7: WSTRB = 0xAA → output[63:56] = 0xAA
Final:  output = 0xAA_..._0F_F0_FF
```

### 5.5.2 Severity-Fold Mode (SB_OR_MODE=1)

Folds per-sub-beat responses into one wide-beat response by keeping the **numeric maximum**, not a bitwise OR. The distinction matters for the mode's primary use case, 2-bit RRESP: with the AXI encoding (OKAY=00, EXOKAY=01, SLVERR=10, DECERR=11), `SLVERR | EXOKAY = DECERR` — an OR inflates the error class the moment an exclusive-read beat mixes with a slave error (projects/components/utility-ip/converters BUG-005, was CONV-005). Numeric max is exactly severity order for RRESP, so the fold keeps the worst response instead:

```
// gen_or_mode: keep the numeric max across the group's sub-beats
if (narrow_valid && narrow_ready) begin
    if (r_beat_ptr == '0)
        r_sideband_accumulator <=
WIDE_SB_PORT_WIDTH'(narrow_sideband);
    else if (WIDE_SB_PORT_WIDTH'(narrow_sideband) >
r_sideband_accumulator)
        r_sideband_accumulator <=
WIDE_SB_PORT_WIDTH'(narrow_sideband);
end
```

For 1-bit sidebands (a bare error flag) max and OR coincide, so "any error in the group propagates" still holds:

```
Beat 0: RRESP = OKAY   (00) → accumulator = 00
Beat 1: RRESP = EXOKAY (01) → accumulator = 01
Beat 2: RRESP = SLVERR (10) → accumulator = 10 (worst so far)
```

```
Beat 3: RRESP = OKAY   (00) → accumulator = 10 (max retained)
Final:  wide RRESP = SLVERR — not DECERR, which a bitwise OR would
fabricate
```

## 5.6 2.2.6 Implementation

### 5.6.1 Core Logic

Wide valid is **registered**, not combinational. It is set by the narrow beat that *completes* a group, so the wide word is only offered after its final lane has actually been written:

```
// Lane the incoming narrow beat lands in (start_lane on a fresh
burst)
assign w_lane = (r_beat_ptr == '0) ? start_lane : r_beat_ptr;

// The accepted narrow beat that closes a group: last lane, or an
early LAST
logic narrow_completes_group;
assign narrow_completes_group = narrow_valid && narrow_ready &&
                                (w_lane == PTR_WIDTH'(WIDTH_RATIO-1)
||
                                 narrow_last);

logic wide_accept;
assign wide_accept = r_wide_valid && wide_ready;

always_ff @(posedge aclk or negedge aresetn) begin
    if (!aresetn) begin
        r_data_accumulator <= '0;
        r_beat_ptr         <= '0;
        r_wide_valid       <= 1'b0;
        r_last_buffered    <= 1'b0;
        r_burst_fresh      <= 1'b1;
    end else begin
        if (narrow_valid && narrow_ready) begin
            // A group-opening beat ZEROES the whole accumulator and
places
            // its data in one whole-register assignment; later beats
write
            // only their own lane. Without the zeroing, an early
narrow_last
            // (or a burst starting mid-word) leaves residual data and
WSTRB
            // in the unfilled lanes and they leak into the emitted
beat.
            if (r_beat_ptr == '0)
                r_data_accumulator <= {{(WIDE_WIDTH-NARROW_WIDTH)
{1'b0}}, narrow_data}
                                      << (w_lane * NARROW_WIDTH);
            else
                r_data_accumulator[w_lane*NARROW_WIDTH +:
```

```
NARROW_WIDTH] <= narrow_data;

            r_beat_ptr    <= narrow_completes_group ? '0 : (w_lane +
1'b1);
            r_burst_fresh <= narrow_last;
        end

        // ONE point of control for r_wide_valid. A newly completed
group
        // dominates an in-flight wide accept -- when both land on the
same
        // cycle, two separate NBAs to this register would race.
        if (narrow_completes_group) begin
            r_wide_valid    <= 1'b1;
            r_last_buffered <= narrow_last;
        end else if (wide_accept) begin
            r_wide_valid    <= 1'b0;
            r_last_buffered <= 1'b0;
        end
    end
end

assign narrow_ready = !r_wide_valid || wide_ready;
assign wide_valid   = r_wide_valid;
assign wide_last    = r_last_buffered;
```

The registered form is what makes the throughput claim in 2.2.7 true. An earlier version of this page showed a combinational `w_output_valid = (r_count == RATIO-1) || r_last` driving `m_valid`, with `s_ready` gated on `m_ready`. That machine is wrong in both directions: it asserts valid during the cycle the completing narrow beat is still being accepted, so with `m_ready` high the wide beat handshakes before the final lane's non-blocking write lands (a wide word missing its last narrow beat), and with `m_ready` low the completing narrow beat stalls (a bubble every group). It also contradicted 2.2.4, which correctly places the transition *after* the completing beat.

## 5.7 2.2.7 Timing

### 5.7.1 Latency

*Table 2.5: Upsize Latency*

| Scenario | Latency |
|---|---|
| Full buffer (N beats) | N cycles |
| Early LAST (M beats) | M cycles |
| Output handshake | 0-1 cycles |

### 5.7.2 Throughput

**100% throughput** — no gaps required between input beats.

The accumulator accepts one beat per cycle, every cycle. When the output buffer completes its handshake, accumulation of the next wide beat starts immediately.

### 5.7.3 Critical Paths

Typical critical paths: - `s_data` → accumulator buffer → `m_data` - `r_count` → comparison → `s_ready`

**Timing closure**: The module is designed for single-cycle operation, with combinatorial paths only within registered stages.

## 5.8 2.2.8 Resource Utilization

### 5.8.1 Typical Resources (64-bit to 512-bit)

```
Accumulator buffer:   512 flip-flops
Sideband buffer:      64 flip-flops (WSTRB)
Beat counter:         3 flip-flops
Control logic:        ~20 flip-flops
                      ~50-70 LUTs

Total: ~600 flip-flops, ~50-70 LUTs
```

### 5.8.2 Scaling

*Table 2.6: Upsize Resource Scaling*

| Configuration | Registers | LUTs |
|---|---|---|
| 32 → 128 (4:1) | ~170 | ~30 |
| 64 → 256 (4:1) | ~330 | ~40 |
| 64 → 512 (8:1) | ~600 | ~60 |
| 128 → 1024 (8:1) | ~1150 | ~80 |

## 5.9 2.2.9 Usage Example

W-channel upsize (32 → 128) with WSTRB as the concatenated sideband:

```
axi_data_upsize #(
    .NARROW_WIDTH    (32),
    .WIDE_WIDTH      (128),
    .NARROW_SB_WIDTH (4),    // WSTRB, narrow side
    .WIDE_SB_WIDTH   (16),   // WSTRB, wide side
    .SB_OR_MODE      (0)     // concatenate strobes
) u_w_upsize (
    .aclk            (aclk),
    .aresetn         (aresetn),
    .narrow_valid    (s_wvalid),
    .narrow_ready    (s_wready),
    .narrow_data     (s_wdata),
    .narrow_sideband (s_wstrb),
    .narrow_last     (s_wlast),
    // '0 = aligned-only; wire addr%wide_bytes/narrow_bytes for
    // mid-word burst starts (see the wr converter's AW-lane queue)
    .start_lane      ('0),
    .wide_valid      (m_wvalid),
    .wide_ready      (m_wready),
    .wide_data       (m_wdata),
    .wide_sideband   (m_wstrb),
    .wide_last       (m_wlast)
);
```

---

**Next:** axi_data_dnsize Module



---

# 6 2.3 axi_data_dnsize

The **axi_data_dnsize** module splits 1 wide beat into N narrow beats. It accepts the next wide beat during the last narrow beat of the current one, so a steady stream costs no per-beat stall.

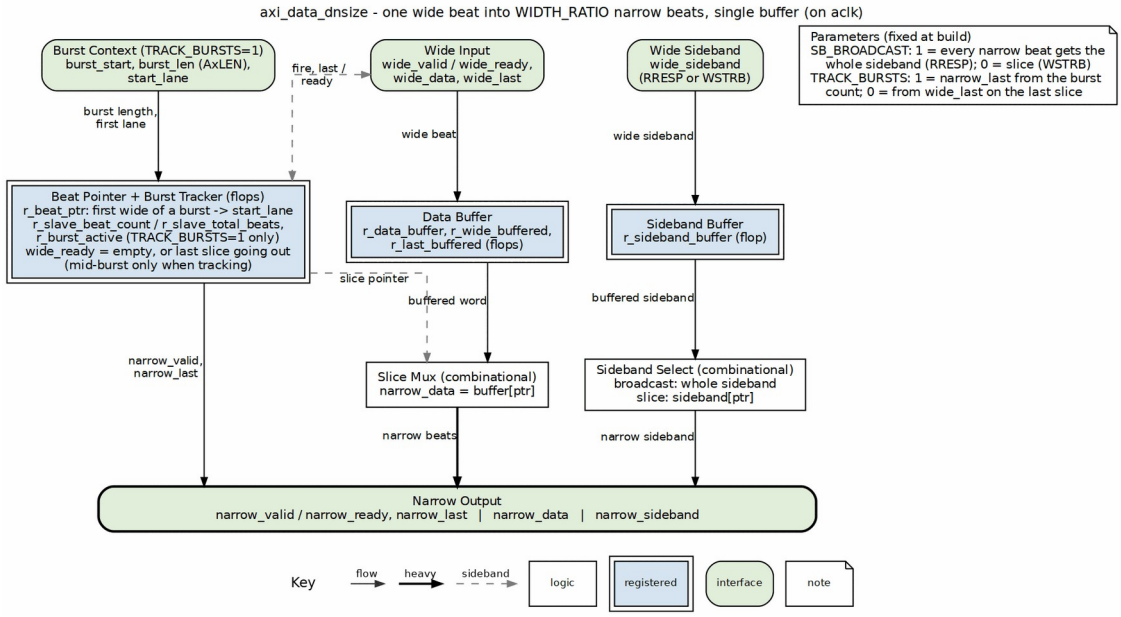
## 6.1 2.3.1 Overview

The downsize module does three things:

1. **Data Splitting**: Extracts N narrow beats from one wide beat
2. **Sideband Extraction**: Slices or broadcasts sideband signals
3. **Burst Tracking**: Optional LAST signal generation based on burst length

## 6.2 2.3.2 Block Diagram

### 6.2.1 Figure 2.3: axi_data_dnsize Single-Buffer Architecture




*axi_data_dnsize Single Buffer*

## 6.3 2.3.3 Interface Specification

### 6.3.1 Parameters

*Table 2.7: axi_data_dnsize Parameters*

| Parameter | Type | Default | Description |
|---|---|---|---|
| WIDE_WIDTH | int | 128 | Input data width (bits) |
| NARROW_WIDTH | int | 32 | Output data width (bits) |
| WIDE_SB_WIDTH | int | 0 | Input sideband width, 0 if unused |
| NARROW_SB_WIDTH | int | 0 | Output sideband width, 0 if unused |
| SB_BROADCAST | int | 1 | 0=slice, 1=broadcast sidebands |
| TRACK_BURSTS | int | 0 | Enable burst-aware LAST generation |
| BURST_LEN_WIDTH | int | 8 | Width of burst length input |

### 6.3.2 Ports

```
module axi_data_dnsize #(
    // Width Configuration
    parameter int WIDE_WIDTH        = 128,
    parameter int NARROW_WIDTH      = 32,
    parameter int WIDE_SB_WIDTH     = 0,        // Sideband width (0
if unused)
    parameter int NARROW_SB_WIDTH   = 0,
    parameter int SB_BROADCAST      = 1,        // 1=broadcast,
0=slice
    parameter int TRACK_BURSTS      = 0,        // 1=track bursts for
LAST
    parameter int BURST_LEN_WIDTH   = 8,        // Burst length
counter width
```

```
    // Calculated Parameters
    localparam int WIDTH_RATIO = WIDE_WIDTH / NARROW_WIDTH,
    localparam int PTR_WIDTH   = $clog2(WIDTH_RATIO),
    // Ensure sideband widths are at least 1 for port declarations
    localparam int WIDE_SB_PORT_WIDTH = (WIDE_SB_WIDTH > 0) ?
WIDE_SB_WIDTH : 1,
    localparam int NARROW_SB_PORT_WIDTH = (NARROW_SB_WIDTH > 0) ?
NARROW_SB_WIDTH : 1
) (
    input  logic                            aclk,
    input  logic                            aresetn,

    // Burst Control (only if TRACK_BURSTS=1)
    input  logic [BURST_LEN_WIDTH-1:0]      burst_len,       // From
address channel (ARLEN/AWLEN)
    input  logic                            burst_start,     // Pulse
to start new burst
    // TRACK_BURSTS only: lane the burst's first narrow beat occupies
in
    // the FIRST wide word; later wide words slice from lane 0.
    input  logic [PTR_WIDTH-1:0]            start_lane,

    // Wide Input (from slave or master)
    input  logic                            wide_valid,
    output logic                            wide_ready,
    input  logic [WIDE_WIDTH-1:0]           wide_data,
    input  logic [WIDE_SB_PORT_WIDTH-1:0]   wide_sideband,  // Min
width 1 to avoid [-1:0]
    input  logic                            wide_last,

    // Narrow Output (to master or slave)
    output logic                            narrow_valid,
    input  logic                            narrow_ready,
    output logic [NARROW_WIDTH-1:0]         narrow_data,
    output logic [NARROW_SB_PORT_WIDTH-1:0] narrow_sideband,  // Min
width 1 to avoid [-1:0]
    output logic                            narrow_last
);
```

## 6.4 2.3.4 Single-Buffer Mode Operation

### 6.4.1 Split Cycle

```
Cycle 0: s_data loaded → buffer, s_ready = 0
Cycle 1: m_data = buffer[63:0],   count = 0, m_valid = 1
Cycle 2: m_data = buffer[127:64], count = 1
...
Cycle 8: m_data = buffer[511:448], count = 7, m_last possible
Cycle 9: wide_ready already asserted during cycle 8's last narrow beat
-- no gap
```

### 6.4.2 State Machine

```
IDLE:
  - s_valid=1 → load buffer → SPLITTING

SPLITTING:
  - Output beats 0 to RATIO-1
  - m_ready=1 → increment count
  - count=RATIO-1 AND m_ready → IDLE
```

### 6.4.3 Throughput Analysis

**No per-beat load cycle**

For ratio N, a wide beat costs N cycles outputting narrow beats. The load of the next wide beat is not an extra cycle: `wide_ready` is asserted during the Nth narrow beat, so the replacement is accepted as the current beat finishes.

`TRACK_BURSTS=1` is the exception. Its ready condition is `mid_burst_replace`, which excludes the final beat of a burst, so a cycle is given up at each burst boundary — not at each wide beat.

Measured on `axi_data_dnsize` with both sides driven by the shared `backtoback` randomizer profile, 64 wide beats per run, timing the drain only (see `measure_throughput` in the dnsize TB):

| Configuration | Narrow beats | Cycles | Beats/cycle |
|---|---|---|---|
| ratio 4, single buffer | 256 | 258 | **0.992** |
| ratio 2, single buffer | 128 | 130 | **0.985** |

Both buffering modes sustain a narrow beat every cycle, which is the most the narrow side can carry. The shortfall is a constant 2-cycle pipeline fill, not a per-beat cost — it does not grow with the run, which is how the earlier “one gap cycle per wide beat” model was ruled out.

`TRACK_BURSTS=1` gives up one cycle per BURST boundary (its replace condition excludes each burst’s final beat) and measures 0.914-0.941 beats/cycle over 8 framed bursts of 32 narrow beats

— the shortfall being the same fixed fill, paid once per burst (`measure_burst_throughput` in the dnsize TB).

*Table 2.8: Measured Throughput by Mode*

| Mode | Measured | Cost model |
|---|---|---|
| Simple, ratio 4 | 0.992 beats/cycle | fixed 2-cycle fill only |
| Simple, ratio 2 | 0.985 beats/cycle | fixed 2-cycle fill only |
| TRACK_BURSTS, ratio 4 | 0.914-0.941 | one bubble per burst boundary |

## 6.5 2.3.5 Sideband Handling

### 6.5.1 Slice Mode (SB_BROADCAST=0)

Used for WSTRB extraction:

```
// Extract sideband slice per beat
assign m_sideband = r_sideband[r_count * NARROW_SB_WIDTH +:
NARROW_SB_WIDTH];
```

**Example**: 64-bit WSTRB to 8-bit WSTRB

```
Input: 0xAA_BB_CC_DD_EE_FF_00_11

Beat 0: output WSTRB = 0x11 (bits [7:0])
Beat 1: output WSTRB = 0x00 (bits [15:8])
Beat 2: output WSTRB = 0xFF (bits [23:16])
...
Beat 7: output WSTRB = 0xAA (bits [63:56])
```

### 6.5.2 Broadcast Mode (SB_BROADCAST=1)

Used for RRESP:

```
// Broadcast same sideband to all beats
assign m_sideband = r_sideband[NARROW_SB_WIDTH-1:0];
```

**Example**: RRESP = OKAY for all beats

```
Input: RRESP = 2'b00 (OKAY)

Beat 0: output RRESP = 2'b00
Beat 1: output RRESP = 2'b00
...
Beat 7: output RRESP = 2'b00
```

## 6.6 2.3.6 Burst Tracking

### 6.6.1 Purpose

When TRACK_BURSTS=1, the module generates narrow_last from the AXI4 burst length instead of relying on the incoming wide_last.

### 6.6.2 Operation

```
// from the RTL: burst_len is already in NARROW beats (the caller
frames
// the burst in output units -- ARLEN/AWLEN scaled by the
instantiator),
// so there is no xRATIO multiply here
if (TRACK_BURSTS != 0 && burst_start && !r_burst_active) begin
    r_slave_total_beats <= burst_len + 1'b1;
    r_slave_beat_count  <= '0;
    r_burst_active      <= 1'b1;
end

// count every narrow beat sent; LAST on the final one
assign narrow_last = r_wide_buffered && r_burst_active &&
                     (r_slave_beat_count + 1'b1 >=
r_slave_total_beats);
```

An earlier revision showed the module multiplying burst_len by RATIO internally. It never did — the units contract is narrow beats in, and mis-framing in wide beats makes LAST fire WIDTH_RATIO times early (a real bug this exact confusion caused in the TB).

**Example**: ARLEN=3 (4 beats), ratio 8:1

```
Total narrow beats = 4 * 8 = 32
Beat 0-30: m_last = 0
Beat 31: m_last = 1
```

## 6.7 2.3.7 Implementation

### 6.7.1 Single-Buffer Core Logic

```
// from the RTL -- the whole point is that wide_ready re-asserts
DURING
// the last narrow beat (atomic replace), so there is NO
// `s_ready = !r_active` style stall between wide beats:
assign wide_ready = !gen_single_buffer.r_wide_buffered ||
                    (narrow_ready && w_last_narrow_beat);

// output slice selection
assign narrow_data = gen_single_buffer.r_data_buffer
                     [r_beat_ptr * NARROW_WIDTH +: NARROW_WIDTH];
assign narrow_valid = gen_single_buffer.r_wide_buffered;

// simple mode: LAST = buffered wide_last AND final slice
assign narrow_last = gen_single_buffer.r_wide_buffered &&
                     gen_single_buffer.r_last_buffered &&
                     w_last_narrow_beat;
```

An earlier pseudocode block here used `s_ready = !r_active` — the pre-fix behaviour that stalls one cycle per wide beat, which the measured 0.992 beats/cycle disproves.

## 6.8 2.3.8 Resource Utilization

### 6.8.1 Single-Buffer (512-bit to 64-bit)

```
Data buffer:          512 flip-flops
Sideband buffer:      64 flip-flops
Beat counter:         3 flip-flops
Control logic:        ~10 flip-flops
                      ~30-50 LUTs

Total: ~590 flip-flops, ~30-50 LUTs
```

### 6.8.2 Measured Throughput

*Table 2.9: Resources and measured rate*

| Registers | LUTs | Throughput |
|---|---|---|
| 590 | 40 | 0.992 beats/cycle (ratio 4) |

A narrow beat every cycle, so there is nothing left for a second buffer to recover. The ping-pong `DUAL_BUFFER` mode this table used to compare against was removed once the single buffer was fixed to accept its replacement during the last narrow beat — nothing instantiated it and it measured no faster.

## 6.9 2.3.9 Usage Example

R-channel downsize (128 → 32) with RRESP broadcast and burst tracking. `burst_start`/`burst_len` are NOT optional with `TRACK_BURSTS(1)`: `r_burst_active` only sets on a `burst_start` pulse, so leaving them unconnected silently produces framing with no LAST at all. `burst_len` is in NARROW beats minus one.

```
axi_data_dnsize #(
    .WIDE_WIDTH      (128),
    .NARROW_WIDTH    (32),
    .WIDE_SB_WIDTH   (2),    // RRESP
    .NARROW_SB_WIDTH (2),
    .SB_BROADCAST    (1),
    .TRACK_BURSTS    (1),
    .BURST_LEN_WIDTH (8)
) u_r_dnsize (
    .aclk            (aclk),
    .aresetn         (aresetn),
    .burst_len       (8'(narrow_beats - 1)),  // narrow-beat units
    .burst_start     (ar_accepted),
    // '0 = aligned-only; wire addr%wide_bytes/narrow_bytes for
    // mid-word burst starts (see the rd converter's length+lane FIFO)
    .start_lane      ('0),
    .wide_valid      (s_rvalid),
    .wide_ready      (s_rready),
    .wide_data       (s_rdata),
    .wide_sideband   (s_rresp),
    .wide_last       (1'b0),                  // counter drives LAST
here
    .narrow_valid    (m_rvalid),
    .narrow_ready    (m_rready),
    .narrow_data     (m_rdata),
    .narrow_sideband (m_rresp),
    .narrow_last     (m_rlast)
);
```

---

**Next:** axi4_dwidth_converter_wr



---

# 7 2.5 axi4_dwidth_converter_wr

The **axi4_dwidth_converter_wr** module is the complete AXI4 write path — AW, W, and B channels with burst length adjustment for the new width.

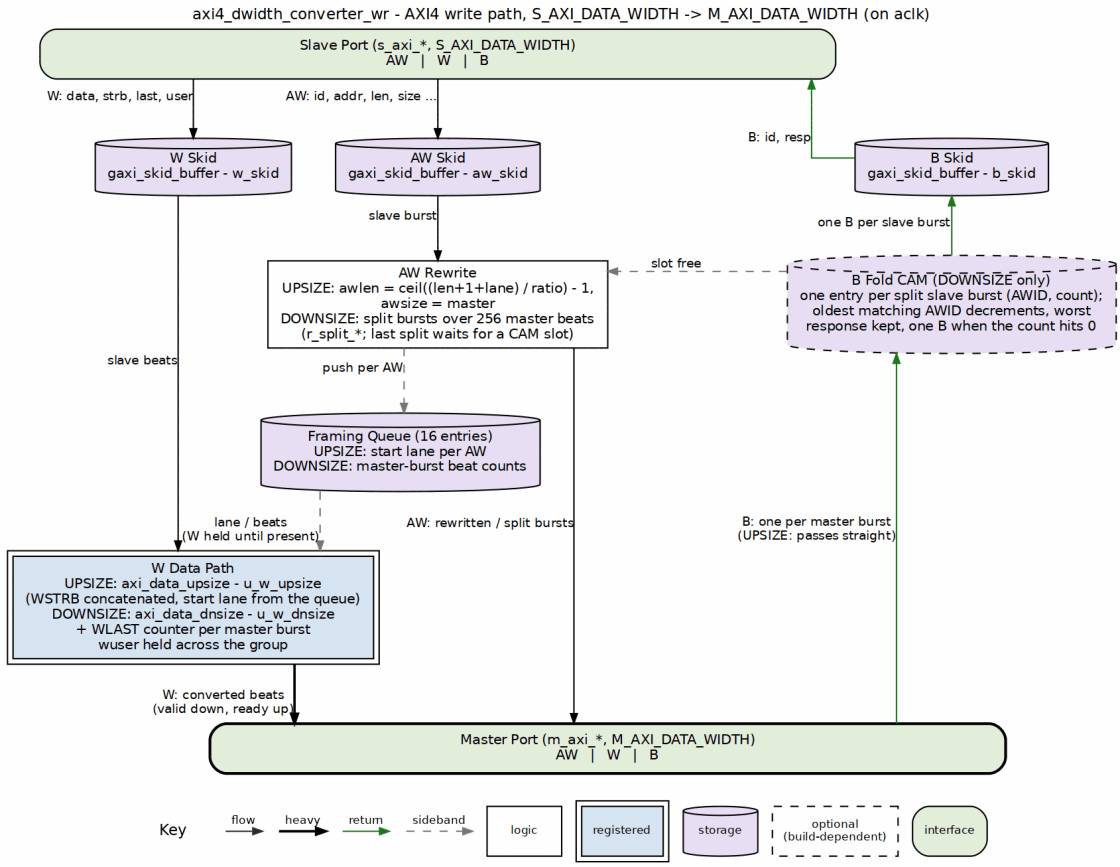
## 7.1 2.5.1 Overview

The write converter combines the generic `axi_data_upsize` with AXI4 protocol handling:

1. **Address Channel (AW)**: Passes through with burst length adjustment
2. **Write Data Channel (W)**: Uses `axi_data_upsize` for data packing
3. **Response Channel (B)**: Passes through unchanged
4. **Burst Length Adjustment**: Converts AWLEN based on width ratio

## 7.2 2.5.2 Block Diagram

### 7.2.1 Figure 2.6: Write Converter Architecture




*Write Converter Architecture*

## 7.3 2.5.3 Interface Specification

### 7.3.1 Parameters

*Table 2.14: Write Converter Parameters*

| Parameter | Type | Default | Description |
|---|---|---|---|
| S_AXI_DATA_WIDTH | int | 32 | Slave-side data width |
| M_AXI_DATA_WIDTH | int | 128 | Master-side data width |
| AXI_ID_WIDTH | int | 8 | Transaction ID width |
| AXI_ADDR_WIDTH | int | 32 | Address width |
| AXI_USER_WIDTH | int | 1 | User-signal width |
| SKID_DEPTH_AW | int | 2 | AW skid buffer depth |
| SKID_DEPTH_W | int | 4 | W skid buffer depth |
| SKID_DEPTH_B | int | 2 | B skid buffer depth |

### 7.3.2 Ports

```
module axi4_dwidth_converter_wr #(
    // Width Configuration
    parameter int S_AXI_DATA_WIDTH  = 32,
    parameter int M_AXI_DATA_WIDTH  = 128,
    parameter int AXI_ID_WIDTH      = 8,
    parameter int AXI_ADDR_WIDTH    = 32,
    parameter int AXI_USER_WIDTH    = 1,

    // Skid Buffer Depths (for timing closure)
    parameter int SKID_DEPTH_AW     = 2,
    parameter int SKID_DEPTH_W      = 4,
    parameter int SKID_DEPTH_B      = 2,

    // Calculated Parameters
    localparam int S_STRB_WIDTH = S_AXI_DATA_WIDTH / 8,
    localparam int M_STRB_WIDTH = M_AXI_DATA_WIDTH / 8,
```

```
    localparam int WIDTH_RATIO  = (S_AXI_DATA_WIDTH <
M_AXI_DATA_WIDTH) ?
                                  (M_AXI_DATA_WIDTH /
S_AXI_DATA_WIDTH) :
                                  (S_AXI_DATA_WIDTH /
M_AXI_DATA_WIDTH),
    localparam bit UPSIZE       = (S_AXI_DATA_WIDTH <
M_AXI_DATA_WIDTH) ? 1'b1 : 1'b0,
    localparam bit DOWNSIZE     = (S_AXI_DATA_WIDTH >
M_AXI_DATA_WIDTH) ? 1'b1 : 1'b0,

    // Skid buffer packed widths
    localparam int AW_WIDTH = AXI_ID_WIDTH + AXI_ADDR_WIDTH + 8 + 3 +
2 + 1 + 4 + 3 + 4 + 4 + AXI_USER_WIDTH,
    localparam int W_WIDTH  = S_AXI_DATA_WIDTH + S_STRB_WIDTH + 1 +
AXI_USER_WIDTH,
    localparam int B_WIDTH  = AXI_ID_WIDTH + 2 + AXI_USER_WIDTH
) (
    // Clock and Reset
    input  logic                        aclk,
    input  logic                        aresetn,


//=================================================================
======
    // Slave AXI Write Interface

//=================================================================
======

    // Write Address Channel
    input  logic [AXI_ID_WIDTH-1:0]     s_axi_awid,
    input  logic [AXI_ADDR_WIDTH-1:0]   s_axi_awaddr,
    input  logic [7:0]                  s_axi_awlen,
    input  logic [2:0]                  s_axi_awsize,
    input  logic [1:0]                  s_axi_awburst,
    input  logic                        s_axi_awlock,
    input  logic [3:0]                  s_axi_awcache,
    input  logic [2:0]                  s_axi_awprot,
    input  logic [3:0]                  s_axi_awqos,
    input  logic [3:0]                  s_axi_awregion,
    input  logic [AXI_USER_WIDTH-1:0]   s_axi_awuser,
    input  logic                        s_axi_awvalid,
    output logic                        s_axi_awready,

    // Write Data Channel
    input  logic [S_AXI_DATA_WIDTH-1:0] s_axi_wdata,
```

```
    input  logic [S_STRB_WIDTH-1:0]     s_axi_wstrb,
    input  logic                        s_axi_wlast,
    input  logic [AXI_USER_WIDTH-1:0]   s_axi_wuser,
    input  logic                        s_axi_wvalid,
    output logic                        s_axi_wready,

    // Write Response Channel
    output logic [AXI_ID_WIDTH-1:0]     s_axi_bid,
    output logic [1:0]                  s_axi_bresp,
    output logic [AXI_USER_WIDTH-1:0]   s_axi_buser,
    output logic                        s_axi_bvalid,
    input  logic                        s_axi_bready,


//=============================================================================
======
    // Master AXI Write Interface

//=============================================================================
======

    // Write Address Channel
    output logic [AXI_ID_WIDTH-1:0]     m_axi_awid,
    output logic [AXI_ADDR_WIDTH-1:0]   m_axi_awaddr,
    output logic [7:0]                  m_axi_awlen,
    output logic [2:0]                  m_axi_awsize,
    output logic [1:0]                  m_axi_awburst,
    output logic                        m_axi_awlock,
    output logic [3:0]                  m_axi_awcache,
    output logic [2:0]                  m_axi_awprot,
    output logic [3:0]                  m_axi_awqos,
    output logic [3:0]                  m_axi_awregion,
    output logic [AXI_USER_WIDTH-1:0]   m_axi_awuser,
    output logic                        m_axi_awvalid,
    input  logic                        m_axi_awready,

    // Write Data Channel
    output logic [M_AXI_DATA_WIDTH-1:0] m_axi_wdata,
    output logic [M_STRB_WIDTH-1:0]     m_axi_wstrb,
    output logic                        m_axi_wlast,
    output logic [AXI_USER_WIDTH-1:0]   m_axi_wuser,
    output logic                        m_axi_wvalid,
    input  logic                        m_axi_wready,

    // Write Response Channel
    input  logic [AXI_ID_WIDTH-1:0]     m_axi_bid,
    input  logic [1:0]                  m_axi_bresp,
```

```
    input  logic [AXI_USER_WIDTH-1:0]   m_axi_buser,
    input  logic                        m_axi_bvalid,
    output logic                        m_axi_bready
);
```

## 7.4 2.5.4 Burst Length Conversion

### 7.4.1 Ratio Calculation

```
// Direction-aware: wider / narrower (a plain M/S divide evaluates to
// 0 in DOWNSIZE mode and $clog2(0) is illegal)
localparam int RATIO = (S_AXI_DATA_WIDTH < M_AXI_DATA_WIDTH)
                     ? (M_AXI_DATA_WIDTH / S_AXI_DATA_WIDTH)
                     : (S_AXI_DATA_WIDTH / M_AXI_DATA_WIDTH);
localparam int RATIO_LOG2 = $clog2(RATIO);

// upsize: New AWLEN = ceil((AWLEN + 1) / RATIO) - 1  (round UP --
// a floor divide underflows on non-multiples, see 2.5.5)
// downsize: split into master bursts of <= 256 beats (see 2.5.5)
```

### 7.4.2 Examples

*Table 2.15: Burst Length Conversion Examples*

| S_DATA | M_DATA | Ratio | S_AWLEN | S_beats | M_AWLEN | M_beats |
|---|---|---|---|---|---|---|
| 64 | 512 | 8 | 7 | 8 | 0 | 1 |
| 64 | 512 | 8 | 15 | 16 | 1 | 2 |
| 64 | 256 | 4 | 3 | 4 | 0 | 1 |
| 64 | 256 | 4 | 7 | 8 | 1 | 2 |

### 7.4.3 Non-Aligned Bursts

When the burst length is not a multiple of the ratio:

```
S_AWLEN = 5 (6 beats), RATIO = 8
M_AWLEN = 0 (1 beat)

The 6 narrow beats pack into 1 wide beat.
Remaining 2 positions have WSTRB = 0 (no write).
```

## 7.5 2.5.5 Address Channel Handling

### 7.5.1 AW Passthrough with Adjustment

The rewrite differs by direction, so the RTL has two generate branches.

**Narrow → wide (upsize).** Beats combine, so the count divides — and it must round UP, because a burst that is not a whole multiple of the ratio still needs a final partial wide beat:

```
assign m_axi_awlen = ((int_awlen + 8'(WIDTH_RATIO)) / 8'(WIDTH_RATIO))
- 8'd1;
```

**Wide → narrow (downsize).** Each wide beat becomes RATIO narrow ones — and the product does not fit a burst. AXI4 allows 256 beats, so a full-length slave burst needs up to `256 * WIDTH_RATIO` narrow beats, which is neither expressible in the 8-bit AWLEN nor a legal burst. One slave burst is therefore **split** into as many master bursts of <= 256 beats as it takes:

```
// remaining-beat counter, 9 + $clog2(WIDTH_RATIO) bits -- 8 is
exactly
// what a multiply-into-AWLEN overflows
r_split_remaining <= (CNTW'(int_awlen) + CNTW'(1)) *
CNTW'(WIDTH_RATIO);
...
assign m_axi_awlen = 8'(w_this_beats - 9'd1);   // w_this_beats =
min(remaining, 256)
```

Each issued master burst is recorded in a split queue — its beat count frames that burst's WLAST on the W path, and a final-burst flag drives the B fold (several master responses collapse into one slave response, worst case wins). The address advances by `256 * M_STRB_WIDTH` per burst for INCR and holds for FIXED; WRAP never reaches a split, since AXI4 caps it at 16 beats and `16 * RATIO <= 256` for every supported ratio. The slave's AW is consumed only when its final master burst has been issued.

Rounding up is the whole point of the `+ WIDTH_RATIO` term. A floor divide underflows: AWLEN=5 (6 beats) at RATIO=8 gives `(6 >> 3) - 1`, which is -1 — 8'hFF, a 256-beat burst — where the correct answer is 0, one wide beat.

**Size** is not derived from the incoming AWSIZE. Both branches drive the master side at its own full width:

```
assign m_axi_awsize = MASTER_SIZE[2:0];
```

Neither converter checks address alignment on the downsize path. The read converter aligns the address it issues on its UPSIZE path only — a wide access cannot start mid-word (see 2.6.5).

**Upsize INCR bursts may start mid-wide-word.** The AXI-correct behavior: the first wide beat carries the narrow data in the byte lanes the ADDRESS selects, with WSTRB covering only those lanes (leading lanes byte-disabled, not clobbered). Three pieces cooperate:

- `m_axi_awlen` counts the lane offset: `ceil((start_lane + narrow_beats) / RATIO)` wide beats. AWADDR passes through unchanged — unaligned is legal AXI; the first beat is partial within its size container.
- an AW-lane queue (same inline pattern as the read converter's burst-length FIFO) records each accepted AW's start lane; W beats are held off until their AW is queued (the packer's lane comes from AWADDR), and the entry pops on the NARROW side's last beat — the head must track the burst the CURRENT narrow beat belongs to, and a wide-side pop raced back-to-back bursts.
- `axi_data_upsize` takes a `start_lane` input: the first narrow beat of a burst lands at that lane (data AND WSTRB shifted, leading slots '0); later wide groups of the burst start at lane 0.

FIXED and WRAP keep the wide-aligned requirement (asserted in simulation) — their lane semantics through the packer are not defined.

### 7.5.2 Skid Buffer for AW

The channel buffers are `gaxi_skid_buffer` (there is no `axi_skid_buffer`), and they carry the UNMODIFIED slave fields — the length rewrite happens after the skid, in the splitter:

```
gaxi_skid_buffer #(
    .DEPTH(SKID_DEPTH_AW),
    .DATA_WIDTH(AW_WIDTH)
) aw_skid (
    .axi_aclk   (aclk),
    .axi_aresetn(aresetn),
    .wr_valid   (s_axi_awvalid),
    .wr_ready   (s_axi_awready),
    .wr_data    ({s_axi_awid, s_axi_awaddr, s_axi_awlen, s_axi_awsize,
                  s_axi_awburst, s_axi_awlock, s_axi_awcache,
s_axi_awprot,
                  s_axi_awqos, s_axi_awregion, s_axi_awuser}),
    .rd_valid   (int_aw_valid),
    .rd_ready   (int_aw_ready),
    .rd_data    (int_aw_data),
    .count      (),
    .rd_count   ()
);
```

## 7.6 2.5.6 Write Data Channel

### 7.6.1 Upsize Instance

```
axi_data_upsize #(
    .NARROW_WIDTH    (S_AXI_DATA_WIDTH),
    .WIDE_WIDTH      (M_AXI_DATA_WIDTH),
    .NARROW_SB_WIDTH (S_STRB_WIDTH),   // WSTRB
    .WIDE_SB_WIDTH   (M_STRB_WIDTH),
    .SB_OR_MODE      (0)               // Concatenate WSTRB
) u_w_upsize (
    .aclk            (aclk),
    .aresetn         (aresetn),
    .narrow_valid    (int_w_valid),
    .narrow_ready    (int_w_ready),
    .narrow_data     (int_wdata),
    .narrow_sideband (int_wstrb),
    .narrow_last     (int_wlast),
    .start_lane      (w_upsize_start_lane[$clog2(WIDTH_RATIO)-1:0]),
    .wide_valid      (m_axi_wvalid),
    .wide_ready      (m_axi_wready),
    .wide_data       (m_axi_wdata),
    .wide_sideband   (m_axi_wstrb),
    .wide_last       (m_axi_wlast)
);
```

## 7.7 2.5.7 Response Channel

### 7.7.1 B Channel

Upsize passes the response through unchanged. Downsize cannot: a split slave burst receives several master responses, and the slave expects ONE. Non-final responses are consumed immediately and folded worst-case- wins; the final one carries the folded result (see 2.5.5). The passthrough below is the UPSIZE branch:

```
// gen_b_pass (upsize)
assign s_bvalid = m_bvalid;
assign m_bready = s_bready;
assign s_bid    = m_bid;
assign s_bresp  = m_bresp;
```

### 7.7.2 Response Ordering

In DOWNSIZE mode the converter must collapse several master B responses into one slave B response. This is now done with a per-burst CAM keyed by AWID:

- Each slave burst is entered into the CAM when its final master AW is issued, recording the burst's AWID, the number of master bursts it was split into, and an initially cleared worst-case BRESP.
- Every master B response searches the CAM for the oldest outstanding entry with the same AWID, decrements that entry's counter, folds the response (worst-case BRESP wins), and forwards a slave B only when the counter reaches zero.
- Same-ID bursts complete in issue order: each entry records, when it is pushed, which live entries of its ID are older, and a B matches the entry whose older entries have all retired. Exactly one entry qualifies, so the index is an OR reduction. Cross-ID bursts can complete in any order because each ID is matched independently. (Until 2026-10-16 this was a 16-way serial age compare; inside a bridge it synthesized to a ~32-LUT chain, ~30 ns at 100 MHz on the Artix-7, and its 5-bit age wrapped at 32 – bridge BUG-016.)
- Master B enters through a 2-deep skid, so the CAM match and fold start from a flop and `m_axi_bready` is the skid's registered ready rather than a function of the match. Without it, a bridge put a slave CAM lookup, the crossbar's B arbiter, this match and the slave CAM's free in one cycle. Cost: one cycle of B latency on a downsize path.
- Every consumer of the match reads the one-hot hit vector directly: the per-entry updates, the folded response (an AND-OR), and the "this B ends the

slave burst" decision, an AND with a registered per-entry `count == 1` flag. No index is encoded and decoded back; that round trip fanned out to every entry's enable.

- The CAM drains on B return while the split queue drains on W framing, so split-queue space does not bound CAM occupancy: a burst's FINAL master AW is gated on a free CAM slot (`w_b_cam_slot_free`). Without the gate a flood of unsplit bursts with slow B return overflows the CAM, drops the push, and that burst's slave B never arrives (regression: `test_axi4_dwidth_converter_wr_b_cam_flood`).

UPSIZE mode passes B responses through unchanged; AXI4 already guarantees same-ID ordering on the master side and the converter does not reorder across IDs.

This replaced the original single-FIFO fold that assumed all B responses arrived in issue order across IDs (fixed 2026-10-04, projects/components/utility-ip/converters BUG-008).

## 7.8 2.5.8 AW/W Synchronization

### 7.8.1 Challenge

AXI4 allows AW to arrive before, with, or after W data. The converter must handle all cases:

1. **AW before W**: Normal pipelining
2. **W before AW**: Data buffered until AW arrives
3. **Interleaved**: Multiple transactions in flight

### 7.8.2 Solution

The **upsize** path carries an AW-lane queue (see 2.5.5): each accepted AW pushes its start lane, and W beats are held off until their AW is queued — the packer's lane placement comes from AWADDR, so W genuinely cannot run ahead of its address. The entry pops on the narrow side's last beat.

The **downsize** path does carry one — the split queue of 2.5.5. Each issued master burst pushes `{final-burst flag, beat count}`; the beat count frames that burst's WLAST on the W path and the flag drives the B fold. AW runs ahead of W, which is precisely why that AW-derived information must be queued.

Compare the read converter, which needs a queue in its UPSIZE mode: its downsize block has no length queue of its own, so overlapping read bursts would lose their framing without an external ARLEN queue.

## 7.9 2.5.9 Resource Utilization

### 7.9.1 Typical Resources (64→512, ID=4, ADDR=64)

```
AW skid buffer:     ~200 flip-flops
W upsize:           ~600 flip-flops, ~60 LUTs
B skid buffer:      ~20 flip-flops
Control logic:      ~100 LUTs

Total: ~820 flip-flops, ~160 LUTs
```

Hand estimates, not synthesis results. AW-derived storage: the downsize split queue, or the upsize AW-lane queue (16 x lane-width entries + two 5-bit pointers) -- see 2.5.5 for both.

## 7.10 2.5.10 Timing

### 7.10.1 Latency

*Table 2.16: Write Converter Latency*

| Path | Latency |
|---|---|
| AW passthrough | 1-2 cycles (skid) |
| W upsize | N cycles (accumulation) |
| B passthrough | 1 cycle (skid) |

### 7.10.2 Throughput

- AW channel: 1 transaction/cycle
- W channel: 100% (upsize is 100%)
- B channel: 1 response/cycle

## 7.11 2.5.11 Usage Example

```
axi4_dwidth_converter_wr #(
    .S_AXI_DATA_WIDTH (128),   // slave wide
    .M_AXI_DATA_WIDTH (32),    // master narrow (downsize)
    .AXI_ID_WIDTH     (8),
    .AXI_ADDR_WIDTH   (32),
    .SKID_DEPTH_AW    (2),
    .SKID_DEPTH_W     (4),
    .SKID_DEPTH_B     (2)
) u_conv (
    .aclk             (aclk),
    .aresetn          (aresetn),
    // slave side: s_axi_aw*/... (full AXI4 channel set)
    .s_axi_awvalid   (cpu_awvalid),
    .s_axi_awready   (cpu_awready),
    // ...
    // master side: m_axi_* toward the narrow fabric
    .m_axi_awvalid   (mem_awvalid),
    .m_axi_awready   (mem_awready)
    // ...
);
```

All channel ports carry the `s_axi_`/`m_axi_` prefix; the full list is the module header. Skid depths are per channel — there is no single `SKID_DEPTH`.

---

**Next:** axi4_dwidth_converter_rd



---

# 8 2.6 axi4_dwidth_converter_rd

The **axi4_dwidth_converter_rd** module is the complete AXI4 read path — AR and R channels with burst length adjustment and burst-aware RLAST generation.

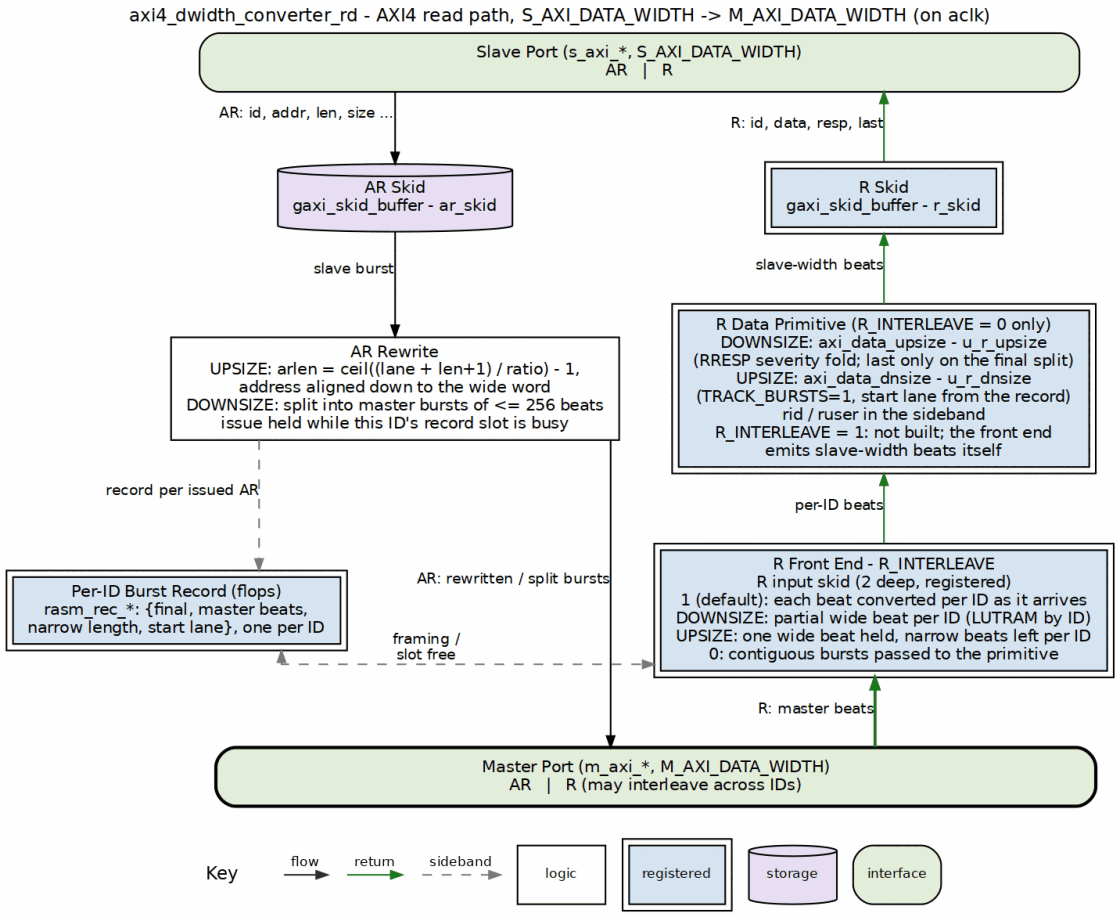
## 8.1 2.6.1 Overview

The read converter combines the generic `axi_data_dnsize` with AXI4 protocol handling:

1. **Address Channel (AR)**: Passes through with burst length adjustment
2. **Read Data Channel (R)**: Uses `axi_data_dnsize` for data splitting
3. **Burst Tracking**: Generates correct RLAST based on original ARLEN
4. **Response Broadcasting**: Propagates RRESP to all narrow beats

## 8.2 2.6.2 Block Diagram

### 8.2.1 Figure 2.7: Read Converter Architecture




*Read Converter Architecture*

## 8.3 2.6.3 Interface Specification

### 8.3.1 Parameters

*Table 2.17: Read Converter Parameters*

| Parameter | Type | Default | Description |
|---|---|---|---|
| S_AXI_DATA_WIDTH | int | 32 | Slave-side data width |
| M_AXI_DATA_WIDTH | int | 128 | Master-side data width |
| AXI_ID_WIDTH | int | 8 | Transaction ID width |
| AXI_ADDR_WIDTH | int | 32 | Address width |
| AXI_USER_WIDTH | int | 1 | User-signal width |
| SKID_DEPTH_AR | int | 4 | AR skid buffer depth |
| SKID_DEPTH_R | int | 4 | R skid buffer depth |
| RASM_DEPTH | int | 272 | Unused. Sized the retired BUG-009 beat pool; kept so existing instances still elaborate |
| RASM_MAX_OUTSTANDING | int | 16 | Unused, as RASM_DEPTH |
| R_INTERLEAVE | bit | 1 | 1: the `m_axi` R source may interleave beats across IDs; each beat is converted per ID as it arrives (2.6.9). 0: R bursts arrive contiguously (in any order across |

| Parameter | Type | Default | Description |
|---|---|---|---|
| | | | IDs); beats pass straight to the data primitive (2.6.9a) |

### 8.3.2 Ports

```
module axi4_dwidth_converter_rd #(
    // Width Configuration
    parameter int S_AXI_DATA_WIDTH  = 32,
    parameter int M_AXI_DATA_WIDTH  = 128,
    parameter int AXI_ID_WIDTH      = 8,
    parameter int AXI_ADDR_WIDTH    = 32,
    parameter int AXI_USER_WIDTH    = 1,

    // Skid Buffer Depths (for timing closure)
    parameter int SKID_DEPTH_AR     = 2,
    parameter int SKID_DEPTH_R      = 4,

    // Calculated Parameters
    localparam int S_STRB_WIDTH = S_AXI_DATA_WIDTH / 8,
    localparam int M_STRB_WIDTH = M_AXI_DATA_WIDTH / 8,
    localparam int WIDTH_RATIO  = (S_AXI_DATA_WIDTH <
M_AXI_DATA_WIDTH) ?
                                  (M_AXI_DATA_WIDTH /
S_AXI_DATA_WIDTH) :
                                  (S_AXI_DATA_WIDTH /
M_AXI_DATA_WIDTH),
    localparam bit UPSIZE       = (S_AXI_DATA_WIDTH <
M_AXI_DATA_WIDTH) ? 1'b1 : 1'b0,
    localparam bit DOWNSIZE     = (S_AXI_DATA_WIDTH >
M_AXI_DATA_WIDTH) ? 1'b1 : 1'b0,

    // Skid buffer packed widths
    localparam int AR_WIDTH = AXI_ID_WIDTH + AXI_ADDR_WIDTH + 8 + 3 +
2 + 1 + 4 + 3 + 4 + 4 + AXI_USER_WIDTH,
    localparam int R_WIDTH  = S_AXI_DATA_WIDTH + 2 + AXI_USER_WIDTH +
1 + AXI_ID_WIDTH
) (
    // Clock and Reset
    input  logic                        aclk,
    input  logic                        aresetn,


//=============================================================
```

```
======
    // Slave AXI Read Interface

//================================================================
======

    // Read Address Channel
    input  logic [AXI_ID_WIDTH-1:0]     s_axi_arid,
    input  logic [AXI_ADDR_WIDTH-1:0]   s_axi_araddr,
    input  logic [7:0]                  s_axi_arlen,
    input  logic [2:0]                  s_axi_arsize,
    input  logic [1:0]                  s_axi_arburst,
    input  logic                        s_axi_arlock,
    input  logic [3:0]                  s_axi_arcache,
    input  logic [2:0]                  s_axi_arprot,
    input  logic [3:0]                  s_axi_arqos,
    input  logic [3:0]                  s_axi_arregion,
    input  logic [AXI_USER_WIDTH-1:0]   s_axi_aruser,
    input  logic                        s_axi_arvalid,
    output logic                        s_axi_arready,

    // Read Data Channel
    output logic [AXI_ID_WIDTH-1:0]     s_axi_rid,
    output logic [S_AXI_DATA_WIDTH-1:0] s_axi_rdata,
    output logic [1:0]                  s_axi_rresp,
    output logic                        s_axi_rlast,
    output logic [AXI_USER_WIDTH-1:0]   s_axi_ruser,
    output logic                        s_axi_rvalid,
    input  logic                        s_axi_rready,


//================================================================
======
    // Master AXI Read Interface

//================================================================
======

    // Read Address Channel
    output logic [AXI_ID_WIDTH-1:0]     m_axi_arid,
    output logic [AXI_ADDR_WIDTH-1:0]   m_axi_araddr,
    output logic [7:0]                  m_axi_arlen,
    output logic [2:0]                  m_axi_arsize,
    output logic [1:0]                  m_axi_arburst,
    output logic                        m_axi_arlock,
    output logic [3:0]                  m_axi_arcache,
    output logic [2:0]                  m_axi_arprot,
```

```
    output logic [3:0]                  m_axi_arqos,
    output logic [3:0]                  m_axi_arregion,
    output logic [AXI_USER_WIDTH-1:0]   m_axi_aruser,
    output logic                        m_axi_arvalid,
    input  logic                        m_axi_arready,

    // Read Data Channel
    input  logic [AXI_ID_WIDTH-1:0]     m_axi_rid,
    input  logic [M_AXI_DATA_WIDTH-1:0] m_axi_rdata,
    input  logic [1:0]                  m_axi_rresp,
    input  logic                        m_axi_rlast,
    input  logic [AXI_USER_WIDTH-1:0]   m_axi_ruser,
    input  logic                        m_axi_rvalid,
    output logic                        m_axi_rready
);
```

## 8.4 2.6.4 Burst Length Conversion

### 8.4.1 Ratio Calculation

Same as write converter:

```
// Direction-aware, same as the write converter (2.5.4)
localparam int RATIO = (S_AXI_DATA_WIDTH < M_AXI_DATA_WIDTH)
                     ? (M_AXI_DATA_WIDTH / S_AXI_DATA_WIDTH)
                     : (S_AXI_DATA_WIDTH / M_AXI_DATA_WIDTH);
localparam int RATIO_LOG2 = $clog2(RATIO);

// upsize: New ARLEN = ceil((start_lane + ARLEN + 1) / RATIO) - 1
// (round UP; the start lane counts for mid-word bursts, see 2.6.5)
// downsize: split into master bursts of <= 256 beats (see 2.6.5)
```

### 8.4.2 Examples

*Table 2.18: Read Burst Length Conversion*

| S_DATA | M_DATA | Ratio | S_ARLEN | S_beats | M_ARLEN | M_beats |
|---|---|---|---|---|---|---|
| 64 | 512 | 8 | 7 | 8 | 0 | 1 |
| 64 | 512 | 8 | 15 | 16 | 1 | 2 |
| 64 | 512 | 8 | 31 | 32 | 3 | 4 |

## 8.5 2.6.5 Address Channel Handling

### 8.5.1 AR Passthrough with Adjustment

Same two directions as the write converter (see 2.5.4 for why the upsize divide rounds up):

```
// narrow -> wide (upsize): beats combine, round up; the start lane
// counts toward the wide total for mid-word bursts
assign m_axi_arlen  = 8'(((w_ar_lane + 10'(int_arlen) +
10'(WIDTH_RATIO))
                          / 10'(WIDTH_RATIO)) - 10'd1);

// wide -> narrow (downsize): each wide beat becomes RATIO narrow
beats,
// and the product can exceed both AWLEN's 8 bits and the 256-beat
legal
// maximum -- so one slave burst is SPLIT into master bursts of <= 256
// beats (same mechanism as the write converter, see 2.5.5)
assign m_axi_arlen  = 8'(w_this_beats - 9'd1);  // min(remaining, 256)

// size is the master's own full width, not a shift of ARSIZE
assign m_axi_arsize = MASTER_SIZE[2:0];
```

For a split read the slave must still see ONE burst: each master burst returns its own RLAST, and every one except the final master burst's is masked out of the upsize, whose accumulation simply continues across the boundary. The masking is safe by construction — 256 narrow beats is a whole number of wide beats at every ratio, so a masked boundary can never land mid-accumulation. A one-bit flag queue, pushed per issued AR and popped per master RLAST, says which burst is final.

On the UPSIZE path the issued address is aligned down to the master data width — the slave returns whole wide words — and mid-word INCR starts are handled by the R slicer: the burst-length FIFO carries the start lane alongside the narrow length, `m_axi_arlen` counts it (`ceil((start_lane + narrow_beats) / RATIO)` wide beats), and `axi_data_dnsize` slices the burst's FIRST wide word from that lane, so the narrow master receives the bytes it actually addressed. Later wide words slice from lane 0. FIXED/WRAP keep the wide-aligned requirement (asserted in simulation). The downsize path issues narrow accesses and passes the address through the splitter unmodified apart from the per-burst advance:

```
localparam int ALIGN_BITS = $clog2(M_STRB_WIDTH);
assign aligned_araddr = {int_araddr[AXI_ADDR_WIDTH-1:ALIGN_BITS],
                         {ALIGN_BITS{1'b0}}};
assign m_axi_araddr   = aligned_araddr;
```

### 8.5.2 Burst-Length Tracking

Only the wide→narrow R data path needs explicit framing — that is the converter's **UPSIZE** mode (S narrower than M: wide master read data sliced down to narrow slave beats through `axi_data_dnsize`). The downsize block ignores a `burst_start` pulse while a burst is active and keeps no length queue of its own, so framing only the first burst would collapse N read bursts into one — bursts 2..N would drain with `narrow_last` never asserting.

The framing record is the per-ID AR record (2.6.8), pushed at AR accept and holding the burst's narrow length and start lane; it replaces the 16-deep burst-length FIFO this section originally described. The record is per-ID (one outstanding master burst per ID by reservation), so no shared queue is needed and overlapping bursts are framed independently. With `R_INTERLEAVE = 1` the same record frames the per-ID slicer (2.6.9) instead of the primitive.

The narrow→wide R data path (the converter's DOWNSIZE mode) needs no such framing; its generate branch ties the shared handshake wires off inert.

## 8.6 2.6.6 Read Data Channel

### 8.6.1 Downsize Instance

```
axi_data_dnsize #(
    .WIDE_WIDTH      (M_AXI_DATA_WIDTH),
    .NARROW_WIDTH    (S_AXI_DATA_WIDTH),
    .WIDE_SB_WIDTH   (2),          // RRESP
    .NARROW_SB_WIDTH (2),
    .SB_BROADCAST    (1),          // Broadcast RRESP
    .TRACK_BURSTS    (1),
    .BURST_LEN_WIDTH (8)
) u_r_dnsize (
    .aclk            (aclk),
    .aresetn         (aresetn),
    // burst framing is MANDATORY with TRACK_BURSTS(1): burst_len is
in
    // NARROW beats - 1, and burst_start is LEVEL-HELD while the
length
    // FIFO is non-empty (the dnsize samples it only when idle, which
is
    // what frames every overlapping burst); leaving them off silently
    // produces framing with no LAST (see 2.3.9)
    .burst_len       (w_blen_rd_data),
    .burst_start     (w_blen_rd_valid),
    .start_lane      (w_blen_rd_lane[$clog2(WIDTH_RATIO)-1:0]),
    .wide_valid      (m_axi_rvalid),
    .wide_ready      (m_axi_rready),
    .wide_data       (m_axi_rdata),
    .wide_sideband   (m_axi_rresp),
    .wide_last       (m_axi_rlast),
    .narrow_valid    (int_r_valid),
    .narrow_ready    (int_r_ready),
    .narrow_data     (int_rdata),
    .narrow_sideband (int_rresp),
    .narrow_last     (int_rlast)
);
```

## 8.7 2.6.7 RLAST Generation

With `R_INTERLEAVE = 1`, RLAST is the per-ID slicer's last narrow beat (upsize) or the final split's RLAST (downsize), 2.6.9. With `R_INTERLEAVE = 0` there is no local RLAST tracker: the dnsize primitive generates `narrow_last` itself in TRACK_BURSTS mode, framed by the per-ID AR record (2.6.8): the record supplies the burst's original narrow length and start lane, `burst_start` pulses at the first accepted wide beat, and the dnsize counts narrow beats against it — `int_rlast` comes out of the dnsize and passes to `s_axi_rlast` through the R skid.

(An earlier revision showed a standalone counter loading `(arlen + 1) * RATIO - 1`, the same xRATIO framing 2.3.4 calls out as the classic mis-framing bug; a later one used a 16-deep burst-length FIFO of narrow arlen/lane per outstanding read. No such multiply exists anywhere in the converter, and the FIFO was replaced by the per-ID record when the BUG-009 reassembly layer, since retired, landed.)

## 8.8 2.6.8 RID Handling

### 8.8.1 ID Tracking

Every converted beat carries its own burst's ID. With `R_INTERLEAVE = 1` the output beat is built from the arriving beat (downsize) or from the wide beat being sliced (upsize), so `int_rid` / `int_ruser` are that beat's RID / RUSER. With `R_INTERLEAVE = 0` they ride the data primitive's sideband, `{RRESP, RID, RUSER}`, RRESP in the top field so the upsize primitive's numeric-max fold keeps the worst response.

### 8.8.2 Ordering Guarantee (read side; write side fixed by BUG-008)

**Write side: no constraint beyond AXI4 itself.** Since 2026-10-04 the B fold is a per-burst CAM keyed by AWID (see `05_dwidth_converter_wr.md`): any cross-ID B completion order is exact (fixed as projects/components/utility-ip/converters BUG-008).

**Read side: full AXI4 R interleaving across IDs is supported.** Beats of different IDs may arrive in any interleaving and any burst order; beats of one ID arrive in order, as AXI4 requires. Each ID has at most one master burst outstanding: master AR issue for an ID waits while that ID's record slot is occupied, and the slot frees when the burst has arrived (downsize) or its converted slave burst has gone out (upsize). Same-ID read chains serialize on that; cross-ID traffic stays concurrent.

An R beat whose ID has no outstanding AR record is flagged by a `SIMULATION`-guarded `$error`.

## 8.9 2.6.9 R Interleave (R_INTERLEAVE = 1, 2026-10-18)

AXI4 permits a slave to interleave R beats across ARIDs. The converter converts each beat against its own ID's state as it arrives; nothing waits for a whole burst, and the data primitives are not built.

1. **Registered input.** `m_axi` R enters through a 2-deep skid, so every per-ID lookup starts from a flop.
2. **Downsize (narrow master beats folded into wide slave beats).** Each ID keeps the lanes of its partial wide beat – `RATIO - 1` narrow lanes, one ID-indexed distributed-RAM array per lane – plus a lane count and the worst RRESP so far. The beat that fills the last lane, or the final split's RLAST, emits the wide beat with the lanes already held. Because the arriving beat supplies the last lane itself, emitting costs no extra cycle.
3. **Upsize (wide master beats sliced into narrow slave beats).** One wide beat is held and sliced lane by lane. Each ID keeps the narrow beats left in its burst and whether the burst has started, so the ID's next wide beat resumes at lane 0 with the right count; the burst's first wide beat starts at the record's start lane. A next beat of the same ID continuing the same burst takes its count from the beat being sliced, not from the table (bypass). RLAST is the burst's last narrow beat.
4. **Records and reservation** are the same as 2.6.9a: the per-ID AR record supplies the final-split flag (downsize) and the narrow length and start lane (upsize).

Cost per instance: `(RATIO - 1) x 2^ID_WIDTH` narrow beats of LUTRAM on the downsize side, and one wide beat of flops plus a 9-bit count per ID on the upsize side.

This replaced the BUG-009 reassembly layer (2026-10-04), a shared beat pool of `RASM_MAX_OUTSTANDING x RASM_DEPTH` entries (16 x 272 by default) that stored each master burst whole before feeding it to the primitive. In a generated bridge that was about 128k LUTs per read converter on the Artix-7; a bridge with ten read converters did not finish synthesis. It was dropped on 2026-10-16 in favour of the no-interleave mode below, and interleaving came back without the pool two days later.

Verified by the converter suite at both modes (the R-interleave tests drive an interleaving slave BFM; the upsize bursts span several wide beats so the per-ID count is exercised), by formal (`prove` and `prove_rbypass`), and by the bridge regression, whose slave BFMs interleave R by default.

## 8.10 2.6.9a No-Interleave Mode (R_INTERLEAVE = 0, 2026-10-16)

When the m_axi R source never interleaves beats of different bursts, beats go straight to the data primitive as they arrive:

1. **Pass-through.** Each R beat goes to the data primitive through one 2-deep skid that registers it (so the record lookup and the primitive's burst framing start from a flop rather than from the source's R mux); no burst is buffered.
2. **Records and reservation.** Bursts of different IDs may still return in any order, so the per-ID AR record is looked up by the arriving beat's RID, and the one-burst-per-ID reservation applies.
3. **Attribution in the sideband.** See 2.6.8.
4. **Downsize accepts other IDs between splits.** Every master burst is a whole number of wide beats from lane 0 (splits are 256 narrow beats), so no wide beat spans two bursts. Another ID's burst arriving between the splits of a slave burst is taken.
5. **Upsize takes one burst at a time.** The dnsize primitive frames from burst_len, so a new burst enters once the previous one is leaving it.

A SIMULATION check flags an interleaved beat. Covered by the converter suite's _ri0 rows. The bridge used this mode from 2026-10-16 to 2026-10-18; it now builds every read converter with R_INTERLEAVE = 1.

## 8.11 2.6.10 Resource Utilization

### 8.11.1 Typical Resources (64→512 UPSIZE, ratio 8, ID=4)

The burst-length FIFO exists only in the converter's UPSIZE mode (see 2.6.5), so the configuration here is upsize — an earlier revision labeled this table 512→64 (DOWNSIZE), a mode in which that FIFO is tied off and does not exist. Hand estimates, not synthesis results, except the FIFO, which is counted from its declarations.

```
AR skid buffer:      ~150 flip-flops
R dnsize data path:  ~600 flip-flops, ~50 LUTs  (single-buffer slicer)
Burst-length FIFO:   186 flip-flops
                     (16 x 8b length + 16 x 3b start lane + two 5b
pointers)
Control logic:       ~80 LUTs

Total: ~940 flip-flops, ~130 LUTs (sum of the lines above)
```

There is no separate "burst tracker" block — 2.6.7 explains RLAST comes from the dnsize's own counter, framed by the FIFO. The R data path is the single-buffer `axi_data_dnsize` — the only implementation; its ping-pong `DUAL_BUFFER` variant was removed, see 2.3.8.

## 8.12 2.6.10 Timing

### 8.12.1 Latency

*Table 2.20: Read Converter Latency*

| Path | Latency |
|---|---|
| AR passthrough | 1-2 cycles (skid) |
| First R beat | 1 cycle (load buffer) |
| Subsequent R beats | 1 beat/cycle |

### 8.12.2 Throughput

- AR channel: 1 transaction/cycle
- R channel: the downsize accepts its next wide beat during the last narrow beat — 0.992 beats/cycle measured in simple mode (TRACK_BURSTS=1 pays one bubble per burst boundary and measures ~0.93; see 2.3)

## 8.13 2.6.11 Usage Example

```
axi4_dwidth_converter_rd #(
    .S_AXI_DATA_WIDTH (128),
    .M_AXI_DATA_WIDTH (32),
    .AXI_ID_WIDTH     (8),
    .AXI_ADDR_WIDTH   (32),
    .SKID_DEPTH_AR    (2),
    .SKID_DEPTH_R     (4)
) u_conv (
    .aclk             (aclk),
    .aresetn          (aresetn),
    // slave side: s_axi_ar*/... (full AXI4 channel set)
    .s_axi_arvalid    (cpu_arvalid),
    .s_axi_arready    (cpu_arready),
    // ...
    // master side: m_axi_* toward the narrow fabric
    .m_axi_arvalid    (mem_arvalid),
    .m_axi_arready    (mem_arready)
    // ...
);
```

All channel ports carry the `s_axi_`/`m_axi_` prefix; the full list is the module header. Skid depths are per channel — there is no single `SKID_DEPTH`.

---

**Next:** Protocol Conversion Overview

# 9 2.7 AXIL Wide-Alignment Converters

`axil_to_axi4_wide_align_wr` and `axil_to_axi4_wide_align_rd` are drop-in replacements for `axi4_dwidth_converter_wr` / `_rd` for the one case those converters get wrong: a slave-side master that is effectively AXI4-Lite – every transfer single-beat, narrow, at a sub-row address.

## 9.1 2.7.1 Why They Exist

The generic converters place data by **beat counter**. `axi_data_upsize` aggregates N narrow beats into one wide beat in arrival order, and `axi_data_dnsize` emits narrow beats the same way. That is correct for a true multi-beat narrow burst.

An AXI4-Lite master never issues one. Every transfer is a single beat with `awlen=0`, and AXI4 requires the data for a narrow single-beat transfer to sit in the byte lanes selected by the address, not in lane 0. With the generic upsize, every such write lands at lane 0 whatever `awaddr` says, and every such read is taken from lane 0 rather than the requested slot.

These two modules place the beat at byte offset `(addr & wide_mask) * 8` instead, which is what the specification requires.

The path this matters on is the bridge generator's: it wraps an AXI4-Lite master with `axi4_slave_wr`, which presents `fub_axi_*` with `awlen=0`, a narrow `awsize` and `awburst=01`. That is exactly the shape the generic upsize mishandles.

## 9.2 2.7.2 Parameters

Both modules take the same set. Defaults differ from the generic converters because the intended direction is narrow master to wide slave.

*Wide-Alignment Converter Parameters*

| Parameter | Default | Description |
|---|---|---|
| `S_AXI_DATA_WIDTH` | 32 | Slave-side (narrow, AXIL-like) data width |
| `M_AXI_DATA_WIDTH` | 256 | Master-side (wide) data width |
| `AXI_ID_WIDTH` | 8 | ID width |
| `AXI_ADDR_WIDTH` | 32 | Address width |
| `AXI_USER_WIDTH` | 1 | User-signal width |
| `SKID_DEPTH_AW` | 2 | **Accepted for API parity; unused.** Tied into the module's `_unused` sink |
| `SKID_DEPTH_W` | 2 | Likewise unused (write module) |
| `SKID_DEPTH_B` | 2 | Likewise unused (write module) |
| `SKID_DEPTH_AR` | 2 | Likewise unused (read module) |
| `SKID_DEPTH_R` | 4 | Likewise unused (read module) |

The `SKID_DEPTH_*` parameters exist so these modules can be swapped in for the generic converters without editing the instantiation. They buffer nothing – the RTL lists them explicitly in a `verilator lint_off UNUSED` sink rather than leaving the non-use implicit.

Several AW/AR sideband inputs are declared and deliberately unused for the same reason – `awlen`, `awsize`, `awburst`, `awlock`, `awqos`, `awregion` and `wlast` on the write side. A single-beat AXIL transfer carries no useful burst description, so the module ignores it rather than trusting it.

# 10 3.1 Protocol Conversion Overview

Protocol converters bridge components that speak different bus protocols — the glue in any SoC that mixes AXI4, AXI4-Lite, and APB.

## 10.1 3.1.1 Available Converters

### 10.1.1 AXI4 to AXI4-Lite (Protocol Downgrade)

*Table 3.1: AXI4 to AXI4-Lite Converters*

| Module | Function | Test Status |
|---|---|---|
| axi4_to_axil4_rd | Read burst decomposition | 5/5 configs passing |
| axi4_to_axil4_wr | Write burst decomposition | 5/5 configs passing |
| axi4_to_axil4 | Full bidirectional wrapper | Composed |

### 10.1.2 AXI4-Lite to AXI4 (Protocol Upgrade)

*Table 3.2: AXI4-Lite to AXI4 Converters*

| Module | Function | Test Status |
|---|---|---|
| axil4_to_axi4_rd | Read protocol upgrade | 7/7 passing |
| axil4_to_axi4_wr | Write protocol upgrade | 7/7 passing |
| axil4_to_axi4 | Full bidirectional wrapper | Composed |

### 10.1.3 Other Protocol Converters

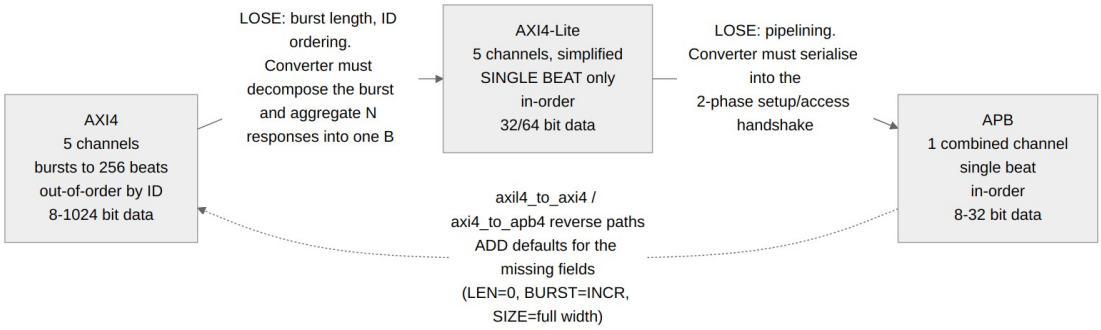
*Table 3.3: Other Protocol Converters*

| Module | Function | Status |
|---|---|---|
| axi4_to_apb4_convert | Full AXI4-to-APB bridge | Production |
| peakrdl_to_cmdrsp | Register interface adapter | Production |
| uart_axil_bridge | UART to AXI4-Lite | Implemented (`rtl/uart_to_axil4/`, `dv/tests/test_uart_axil_bridge.py`) |
| axil4_to_wb4 | AXI4-Lite to Wishbone B4 (pipelined or classic) | Implemented (`dv/tests/test_axil4_to_wb4.py`, formal on the |

| Module | Function | Status |
|---|---|---|
| | | core) |
| wb4_to_axil4 | Wishbone B4 to AXI4-Lite; merges AXI's two response channels back into B4's in-order termination | Implemented (`dv/tests/test_wb4_to_axil4.py`, formal on the core) |
| apb4_to_axi4 / apb5_to_axi4 | APB4 / APB5 completer in, single-beat AXI4 requester out (the mirror of axi4_to_apb4); APB5 USER mapped onto AXI USER | Implemented (`dv/tests/test_apb{4,5}_to_axi4.py`; the bridge's APB master ports, bridge TASK-004 (was BRIDGE-014)) |
| apb_cmdrsp_to_axi4 | The requester half of the two above: APB cmd/rsp stream to one AXI4 transaction per transfer | Implemented (covered by the wrappers' tests) |
| axi4_to_wb4 | Full AXI4 completer in, Wishbone B4 requester out: the AXI4-Lite decomposers + axil4_to_wb4 | Implemented (`dv/tests/test_axi4_to_wb4.py`; the bridge's WB4 slave ports, bridge TASK-008 (was BRIDGE-019)) |
| wb4_to_axi4 | Wishbone B4 completer in, single-beat AXI4 requester out: wb4_to_axil4 + the AXI4 promotion | Implemented (`dv/tests/test_wb4_to_axi4.py`; the bridge's WB4 master ports, bridge TASK-008 (was BRIDGE-019)) |
| axis_to_cxs / cxs_to_axis | AXI-Stream beats to CXS flits and back: one packet per TLAST run, TSTRB/TUSER in CXSUSER, per-stream framing (section 3.17) | Implemented (covered by `val/amba/test_cxs_pattern_pair.py` and the RAPIDS CXS tops) |

## 10.2 3.1.2 Protocol Comparison

### 10.2.1 Figure 3.1: Protocol Feature Comparison




*Protocol Comparison*

### 10.2.2 Feature Matrix

*Table 3.4: Protocol Feature Comparison*

| Feature | AXI4 | AXI4-Lite | APB |
|---|---|---|---|
| Channels | 5 (AW, W, B, AR, R) | 5 (simplified) | 1 (combined) |
| Bursts | Up to 256 beats | Single beat only | Single beat |
| Out-of-order | Yes (ID-based) | No | No |
| Pipelining | Yes | Optional | No (2-phase) |
| Data widths | 8-1024 bits | 32/64 bits | 8-32 bits |

## 10.3 3.1.3 Conversion Strategies

### 10.3.1 AXI4 to AXI4-Lite

**Challenge:** Decompose multi-beat bursts into sequential single beats.

**Strategy:** 1. Accept AXI4 burst transaction 2. Issue N single-beat AXI4-Lite transactions 3. Aggregate responses 4. Return combined response to AXI4 master

**Complexity:** Medium (FSM-based decomposition)

### 10.3.2 AXI4-Lite to AXI4

**Challenge:** Add AXI4 burst signals with appropriate defaults.

**Strategy:** 1. Pass through all AXI4-Lite signals 2. Add default values for burst signals (LEN=0, SIZE=$clog2(DATA_WIDTH/8), BURST=INCR) 3. Add default IDs (configurable)

**Complexity:** Very low (combinational only)

### 10.3.3 AXI4 to APB

**Challenge:** Bridge 5-channel AXI4 to 2-phase APB protocol.

**Strategy:** 1. Accept AXI4 transaction 2. Execute APB setup phase 3. Execute APB access phase 4. Return AXI4 response

**Complexity:** High (full protocol FSM)

## 10.4 3.1.4 Performance Characteristics

*Table 3.5: Protocol Converter Performance*

| Converter | Single-Beat | Burst (N) | Area |
|---|---|---|---|
| axi4_to_axil4 | 0 cycles | slave-limited, ~N cycles best case (see 3.2.6) | ~450 LUTs |
| axil4_to_axi4 | 0 cycles | N/A | ~110 LUTs |
| axi4_to_apb4 | 3-5 cycles | ~2N+1 APB cycles sustained (see 3.4.11) | ~150 LUTs (hand estimate; matches 3.4.10) |

### 10.4.1 Key Observations

1. **Zero-overhead upgrade:** AXI4-Lite to AXI4 is purely combinational
2. **Burst rate:** AXI4 to AXI4-Lite adds no wait state between beats — decomposed requests stream at whatever rate the AXIL4 slave sustains (a zero-wait-state slave is served one beat per cycle)
3. **APB overhead:** 3-5 cycle minimum per APB transaction

## 10.5 3.1.5 Use Case Guidelines

### 10.5.1 When to Use AXI4 to AXI4-Lite

**Use when:** - CPU/DMA with burst support needs simple peripheral access - Want to simplify peripheral design (no burst handling) - Data widths match (no width conversion needed) - Burst performance is not critical

**Avoid when:** - High-bandwidth streaming data - Latency-critical paths - Many back-to-back bursts

### 10.5.2 When to Use AXI4-Lite to AXI4

**Use when:** - Legacy AXI4-Lite IP connects to AXI4 fabric - Designing simple peripheral for AXI4 system - Want zero-overhead protocol upgrade - Don’t need burst capability

**Avoid when:** - Need actual burst support (use native AXI4) - Width conversion needed (use width converters)

### 10.5.3 When to Use AXI4 to APB

**Use when:** - AXI4 masters need APB peripheral access - Building CPU-to-peripheral bridges - Integrating legacy APB devices

**Avoid when:** - High-performance paths (APB is slow) - Streaming data (APB is sequential)

## 10.6 3.1.6 Integration Patterns

### 10.6.1 Pattern 1: CPU to Peripherals

```
CPU (AXI4) → AXI4-to-AXIL4 → AXIL4 Peripherals
                           → AXI4-to-APB → APB Peripherals
```

### 10.6.2 Pattern 2: Simple IP in AXI4 Fabric

```
Simple IP (AXIL4) → AXIL4-to-AXI4 → AXI4 Crossbar
```

### 10.6.3 Pattern 3: Mixed System

```
CPU (AXI4) → AXI4 Crossbar → DDR (AXI4)
                           → AXI4-to-AXIL4 → Config Regs (AXIL4)
                           → AXI4-to-APB → UART, GPIO (APB)
```

---

**Next:** AXI4 to AXI4-Lite

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 11 3.2 AXI4 to AXI4-Lite Converter

The **axi4_to_axil4** converter family decomposes AXI4 burst transactions into sequential AXI4-Lite single-beat transactions.

## 11.1 3.2.1 Module Organization

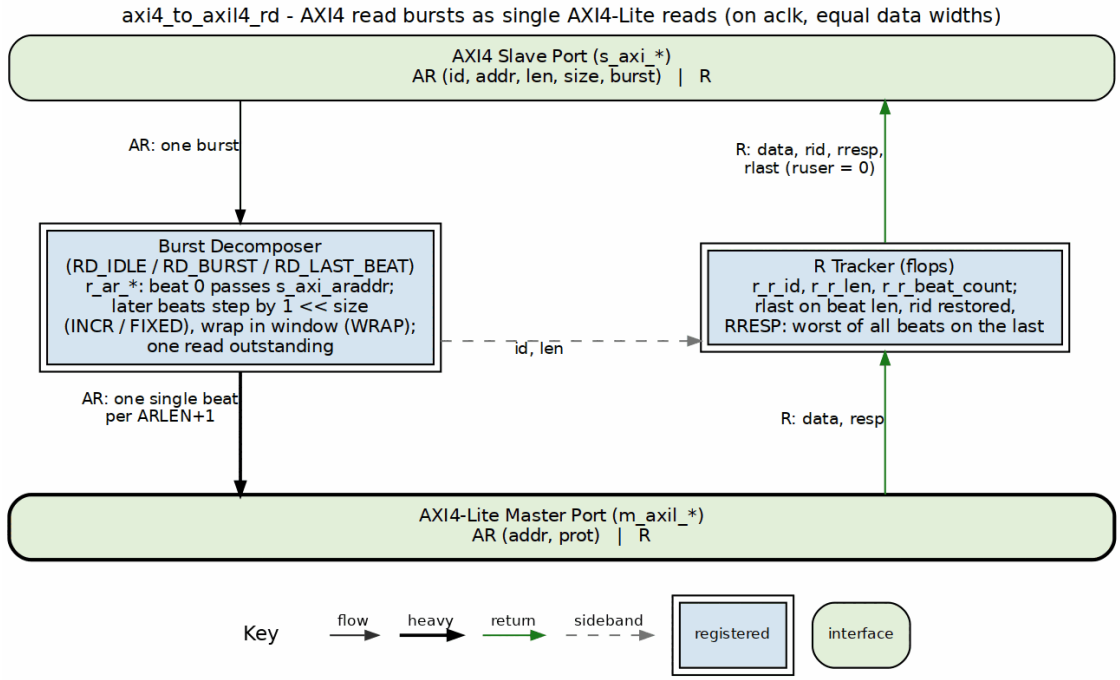
```
axi4_to_axil4.sv          # Full bidirectional wrapper
├── axi4_to_axil4_rd.sv   # Read path converter
└── axi4_to_axil4_wr.sv   # Write path converter
```

### 11.1.1 Design Philosophy

Read and write are separate modules, which buys: - Independent optimization - Selective instantiation (read-only, write-only, or both) - Simpler verification (test paths independently)

## 11.2 3.2.2 Read Path (axi4_to_axil4_rd)

### 11.2.1 Figure 3.2: AXI4 to AXI4-Lite Read Path




*AXI4 to AXIL4 Read*

### 11.2.2 Operation

**Single-Beat (ARLEN=0):**

```
Cycle 0: AR accepted (passthrough)
Cycle 1: R returned (passthrough)
Total: 0 extra cycles (pure passthrough)
```

**Multi-Beat (ARLEN=N-1):**

```
Cycle 0:   AR[0] issued to AXIL4
Cycle 1:   R[0] received, AR[1] issued
Cycle 2:   R[1] received, AR[2] issued
...
           R[N-1] received (RLAST)
```

AR issue and R return are fully independent: the address FSM streams one AXIL4 AR per beat as fast as `m_axil_arready` accepts, with no reference to R at all — nothing in the AR path looks at `m_axil_rvalid`. Against a slave with multi-cycle response latency the requests pipeline ahead of the data, and end-to-end duration is set by the slave, not the converter.

Bursts do not overlap each other: AR is held off until the burst in flight has delivered its last beat, because the burst-tracking registers hold one burst at a time.

### 11.2.3 State Machine

```
typedef enum logic [1:0] {
    RD_IDLE       = 2'b00,   // No burst in flight
    RD_BURST      = 2'b01,   // Issuing the AXIL4 reads of a burst
    RD_LAST_BEAT  = 2'b10    // Issuing the burst's final read
} rd_state_t;
```

A single-beat read (`ARLEN == 0`) never leaves `RD_IDLE`: it takes the passthrough leg rather than the decomposition FSM. That is also why the one-outstanding-burst guard cannot rely on the FSM state alone.

### 11.2.4 Implementation

```
    // Read state machine
    typedef enum logic [1:0] {
        RD_IDLE       = 2'b00,
        RD_BURST      = 2'b01,
        RD_LAST_BEAT  = 2'b10
    } rd_state_t;

    rd_state_t r_rd_state, w_rd_next_state;

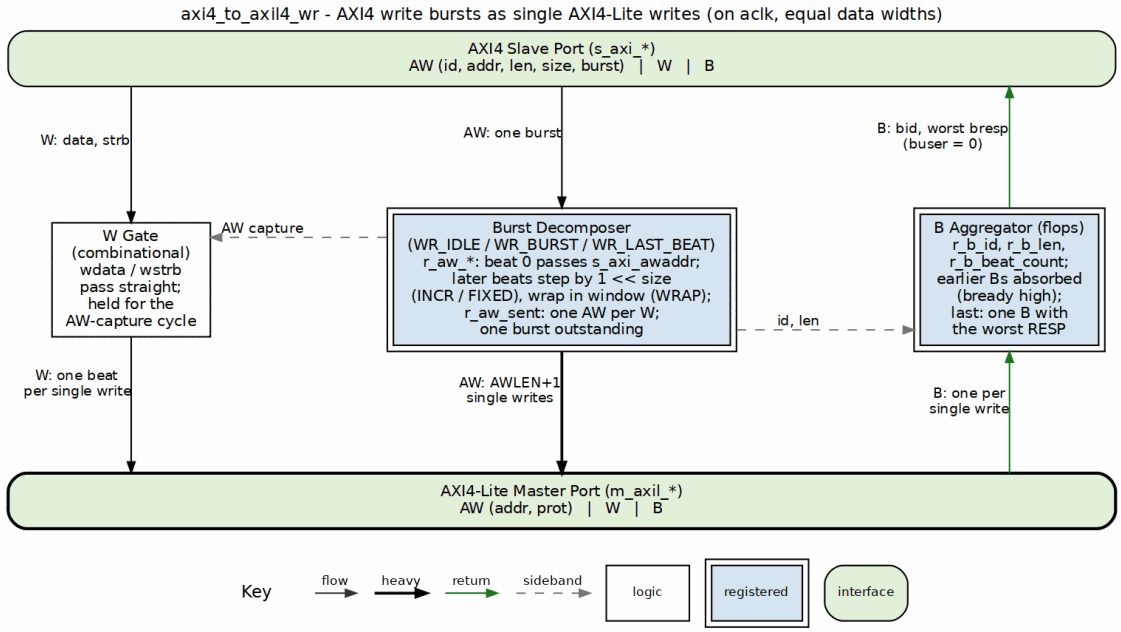
    always_ff @(posedge aclk or negedge aresetn) begin
        if (!aresetn) begin
            r_rd_state <= RD_IDLE;
        end else begin
            r_rd_state <= w_rd_next_state;
        end
    end

    // Read next state logic
    always_comb begin
        w_rd_next_state = r_rd_state;
        case (r_rd_state)
            RD_IDLE: begin
                if (s_axi_arvalid && s_axi_arready) begin
                    if (s_axi_arlen == 0)
                        w_rd_next_state = RD_IDLE;  // Single beat,
stay idle
                    else
                        w_rd_next_state = RD_BURST;  // Start burst
decomposition
                end
            end
            RD_BURST: begin
```

```
            if (m_axil_arvalid && m_axil_arready) begin
                if (r_ar_beat_count == r_ar_len - 1)
                    w_rd_next_state = RD_LAST_BEAT;
            end
        end
        RD_LAST_BEAT: begin
            if (m_axil_arvalid && m_axil_arready)
                w_rd_next_state = RD_IDLE;
        end
        default: w_rd_next_state = RD_IDLE;
    endcase
end
```

## 11.3 3.2.3 Write Path (axi4_to_axil4_wr)

### 11.3.1 Figure 3.3: AXI4 to AXI4-Lite Write Path




*AXI4 to AXIL4 Write*

### 11.3.2 Operation

**Single-Beat (AWLEN=0):**

```
Cycle 0: AW+W accepted (passthrough)
Cycle 1: B returned (passthrough)
Total: 0 extra cycles
```

**Multi-Beat (AWLEN=N-1):**

```
Cycle 0:   AW[0] + W[0] issued
Cycle 1:   B[0] received, AW[1] + W[1] issued
...
           B[N-1] received (timing set by the AXIL4 slave)
           B[N-1] received
```

As on the read path, AW/W issue is independent of B return — the write FSM streams beats as fast as the AXIL4 slave accepts them, and mid-burst B responses are consumed immediately (`m_axil_bready = s_axi_bready || !w_b_all_beats_done`). The single B the master receives is emitted once every decomposed write has responded. Bursts do not overlap each other: the next AW is held off until the burst in flight has its B accepted.

### 11.3.3 AW/W Synchronization Challenge

AXI4 allows AW and W to arrive in any order: - AW before W - W before AW - Interleaved

### 11.3.4 Solution: W gated on the AW that owns it

W is gated directly against the state of the AW it belongs to, rather than through pending-arrival flags, so a data beat cannot reach the AXIL4 slave ahead of its own address:

- **Burst capture cycle.** When a burst's AW is being taken into the registers, W is held off for that cycle (`w_burst_capture`). Without it, the first W beat of the next burst slips out while the previous burst is still finishing, and the AXIL4 slave pairs an address from burst N with data from burst N+1.
- **During a burst.** W passes only once this burst's AW has actually been sent (`r_aw_sent`), or is being sent in this very cycle.
- **Single beats.** Straight passthrough; AW and W are presented together.

```
    // s_axi_awready restricts the block to the ACCEPT cycle. Without
it
    // a burst AW merely PARKED by the one-outstanding guard
(awvalid=1,
    // awready=0) deadlocked the write already in flight
(projects/components/utility-ip/converters BUG-006, was CONV-007).
    wire w_burst_capture = !r_aw_active && s_axi_awvalid &&
s_axi_awready &&
                           (s_axi_awlen > 0);

    assign m_axil_wvalid = w_burst_capture ? 1'b0 :
                           r_aw_active     ? (s_axi_wvalid &&
                                              (r_aw_sent ||
                                               (m_axil_awvalid &&
m_axil_awready))) :
                                             s_axi_wvalid;
    assign s_axi_wready  = w_burst_capture ? 1'b0 :
                           r_aw_active     ? (m_axil_wready &&
                                              (r_aw_sent ||
                                               (m_axil_awvalid &&
m_axil_awready))) :
                                             m_axil_wready;
```

The failure this guards against only appears back to back — the next AW arriving the cycle after `WR_LAST_BEAT` completes. A sequential test with a cooldown between bursts never opens that window, which is why it survived the FUB tests and was caught by a bridge-level probe. Classic bring-up bug, in other words: it hides from every directed test and fires the moment real traffic shows up.

### 11.3.5 Response Aggregation

```
// from the RTL: accumulate per response, reset on AW acceptance
always_ff @(posedge aclk or negedge aresetn) begin
    ...
    if (s_axi_awvalid && s_axi_awready)
        r_b_resp_accum <= 2'b00;
    if (m_axil_bvalid && m_axil_bready)
        if (m_axil_bresp > r_b_resp_accum)
            r_b_resp_accum <= m_axil_bresp;
end

// live-beat fold at emission (see 4.3.3 for why the registered-only
// form was a shipped bug)
assign w_b_resp_worst = (m_axil_bresp > r_b_resp_accum) ? m_axil_bresp
                                                        :
r_b_resp_accum;
assign s_axi_bresp = w_b_resp_worst;
```

## 11.4 3.2.4 Bidirectional Wrapper (axi4_to_axil4)

### 11.4.1 Composition Pattern

```
module axi4_to_axil4 #(
    parameter int AXI_ID_WIDTH   = 8,
    parameter int AXI_ADDR_WIDTH = 32,
    parameter int AXI_DATA_WIDTH = 32,
    parameter int AXI_USER_WIDTH = 1
) (
    // aclk/aresetn + full s_axi_*/m_axil_* channel set
);

    axi4_to_axil4_rd #(
        .AXI_ID_WIDTH   (AXI_ID_WIDTH),
        .AXI_ADDR_WIDTH (AXI_ADDR_WIDTH),
        .AXI_DATA_WIDTH (AXI_DATA_WIDTH),
        .AXI_USER_WIDTH (AXI_USER_WIDTH)
    ) u_rd_converter ( /* read channels */ );

    axi4_to_axil4_wr #(
        .AXI_ID_WIDTH   (AXI_ID_WIDTH),
        .AXI_ADDR_WIDTH (AXI_ADDR_WIDTH),
        .AXI_DATA_WIDTH (AXI_DATA_WIDTH),
        .AXI_USER_WIDTH (AXI_USER_WIDTH)
    ) u_wr_converter ( /* write channels */ );

endmodule
```

## 11.5 3.2.5 Resource Utilization

*Table 3.6: AXI4 to AXIL4 Resources*

| Module | Registers | LUTs | BRAM |
|---|---|---|---|
| axi4_to_axil4_rd | ~120 | ~180 | 0 |
| axi4_to_axil4_wr | ~150 | ~220 | 0 |
| axi4_to_axil4 (combined) | ~270 | ~400 | 0 |

## 11.6 3.2.6 Timing

### 11.6.1 Throughput

*Table 3.7: AXI4 to AXIL4 Throughput*

| Transaction Type | Behaviour |
|---|---|
| Single-beat | Passthrough; no converter-inserted wait state |
| N-beat burst | One AXIL4 access per beat; requests stream at the slave's accept rate, independent of responses |
| Back-to-back bursts | Serialized: the next AR/AW waits for the previous burst's last beat |

The previous "50% for any burst" figure was not measured and does not follow from the design: the converter adds no wait state between beats, so the achievable rate is whatever the AXIL4 slave sustains. A measured characterization per slave latency has not been done and is not claimed here.

### 11.6.2 Latency

*Table 3.8: AXI4 to AXIL4 Latency*

| Transaction Type | Latency |
|---|---|
| Single-beat | 0 extra cycles |
| N-beat burst | slave-limited; requests stream independently of responses (see 3.2.6) |

## 11.7 3.2.7 Testing

**Test Suite:** both directions green — `test_axi4_to_axil4_{wr,rd}.py`, 5 parametrized configurations each, every scenario suite asserted (the counts below are scenario groups within those runs)

*Table 3.9: Test Coverage Summary*

| Test Category | Tests | Status |
|---|---|---|
| Single-beat read | 4 | Pass |
| Multi-beat read | 6 | Pass |
| Single-beat write | 4 | Pass |
| Multi-beat write | 6 | Pass |
| Mixed traffic | 8 | Pass |
| Error injection | 6 | Pass |
| Edge cases | 8 | Pass |

**Next:** AXI4-Lite to AXI4

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

# 12 3.3 AXI4-Lite to AXI4 Converter

The **axil4_to_axi4** converter family upgrades AXI4-Lite single-beat transactions to full AXI4 protocol by adding default burst signals.

## 12.1 3.3.1 Module Organization

```
axil4_to_axi4.sv          # Full bidirectional wrapper
├── axil4_to_axi4_rd.sv   # Read path converter
└── axil4_to_axi4_wr.sv   # Write path converter
```

## 12.2 3.3.2 Design Philosophy

**Zero-Overhead Upgrade:** - Purely combinational logic - No state machines or buffers - Adds default values for missing AXI4 signals

**Why This Works:** - AXI4-Lite is a subset of AXI4 - All AXI4-Lite transactions are single-beat - Missing AXI4 signals have well-defined defaults

## 12.3 3.3.3 Signal Mapping

### 12.3.1 Address Channel Signals

*Table 3.10: AR Channel Mapping*

| AXI4-Lite Signal | AXI4 Signal | Default/Mapping |
|---|---|---|
| ARADDR | ARADDR | Passthrough |
| ARPROT | ARPROT | Passthrough |
| ARVALID | ARVALID | Passthrough |
| ARREADY | ARREADY | Passthrough |
| - | ARLEN | 8'h00 (single beat) |
| - | ARSIZE | $clog2(DATA_WIDTH/8) |
| - | ARBURST | 2'b01 (INCR) |
| - | ARLOCK | 1'b0 (normal) |
| - | ARCACHE | 4'b0011 (bufferable, modifiable) |
| - | ARQOS | 4'(DEFAULT_QOS), parameter, default 0 |
| - | ARID | Configurable default |

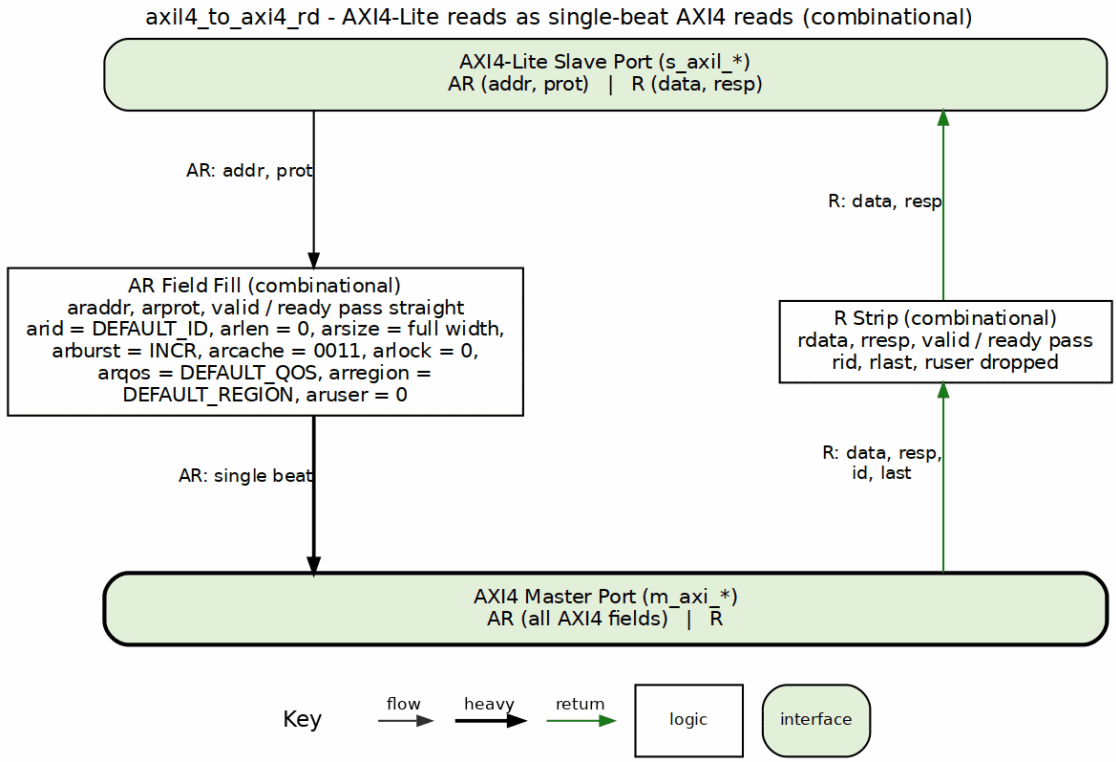
### 12.3.2 Data Channel Signals

*Table 3.11: R Channel Mapping*

| AXI4-Lite Signal | AXI4 Signal | Default/Mapping |
|---|---|---|
| RDATA | RDATA | Passthrough |
| RRESP | RRESP | Passthrough |
| RVALID | RVALID | Passthrough |
| RREADY | RREADY | Passthrough |
| - | RLAST | 1'b1 (always last) |
| - | RID | Matches ARID |

## 12.4 3.3.4 Read Path (axil4_to_axi4_rd)

### 12.4.1 Figure 3.4: AXI4-Lite to AXI4 Read Path




*AXIL4 to AXI4 Read*

### 12.4.2 Implementation

```
module axil4_to_axi4_rd #(
    // Width Configuration
    parameter int AXI_ID_WIDTH      = 8,
    parameter int AXI_ADDR_WIDTH    = 32,
    parameter int AXI_DATA_WIDTH    = 32,
    parameter int AXI_USER_WIDTH    = 1,

    // Default Values for AXI4-only Fields
    parameter int DEFAULT_ID        = 0,
    parameter int DEFAULT_REGION    = 0,
    parameter int DEFAULT_QOS       = 0,

    // Skid Buffer Depths (for timing closure)
```

```
    // Calculated Parameters
    localparam int STRB_WIDTH = AXI_DATA_WIDTH / 8,
    localparam int SIZE_VAL   = $clog2(STRB_WIDTH)  // ARSIZE for full
width
) (
    // Clock and Reset
    input  logic                        aclk,
    input  logic                        aresetn,


//=================================================================
======
    // Slave AXI4-Lite Read Interface (Input - Simplified Protocol)

//=================================================================
======

    // Read Address Channel
    input  logic [AXI_ADDR_WIDTH-1:0]   s_axil_araddr,
    input  logic [2:0]                  s_axil_arprot,
    input  logic                        s_axil_arvalid,
    output logic                        s_axil_arready,

    // Read Data Channel
    output logic [AXI_DATA_WIDTH-1:0]   s_axil_rdata,
    output logic [1:0]                  s_axil_rresp,
    output logic                        s_axil_rvalid,
    input  logic                        s_axil_rready,


//=================================================================
======
    // Master AXI4 Read Interface (Output - Full Protocol)

//=================================================================
======

    // Read Address Channel
    output logic [AXI_ID_WIDTH-1:0]     m_axi_arid,
    output logic [AXI_ADDR_WIDTH-1:0]   m_axi_araddr,
    output logic [7:0]                  m_axi_arlen,
    output logic [2:0]                  m_axi_arsize,
    output logic [1:0]                  m_axi_arburst,
    output logic                        m_axi_arlock,
    output logic [3:0]                  m_axi_arcache,
    output logic [2:0]                  m_axi_arprot,
    output logic [3:0]                  m_axi_arqos,
```

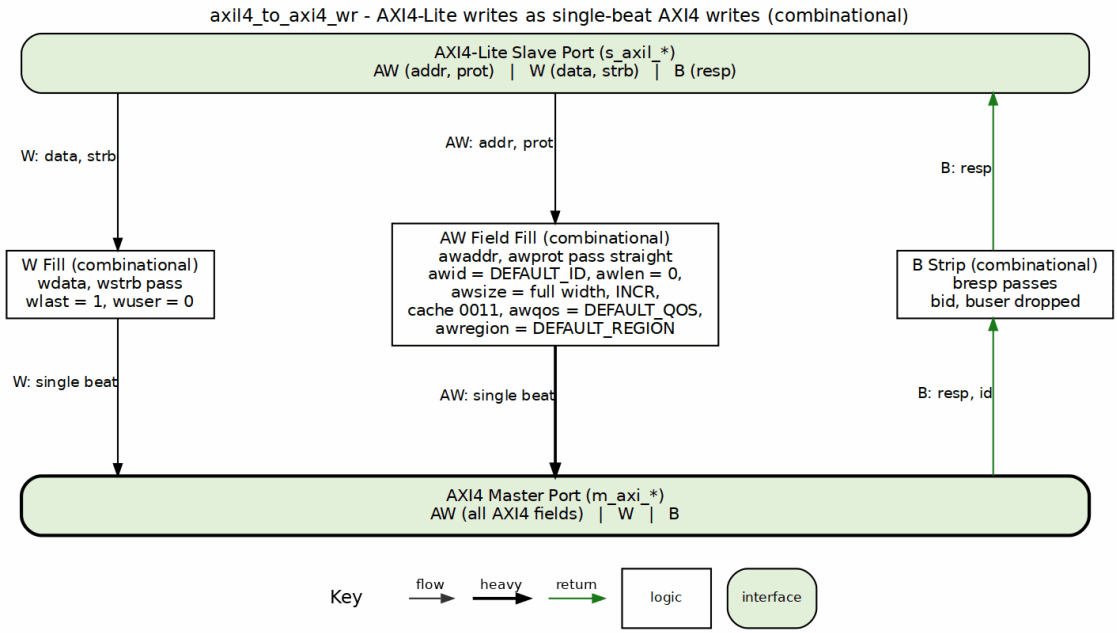
```
    output logic [3:0]                  m_axi_arregion,
    output logic [AXI_USER_WIDTH-1:0]   m_axi_aruser,
    output logic                        m_axi_arvalid,
    input  logic                        m_axi_arready,

    // Read Data Channel
    input  logic [AXI_ID_WIDTH-1:0]     m_axi_rid,
    input  logic [AXI_DATA_WIDTH-1:0]   m_axi_rdata,
    input  logic [1:0]                  m_axi_rresp,
    input  logic                        m_axi_rlast,
    input  logic [AXI_USER_WIDTH-1:0]   m_axi_ruser,
    input  logic                        m_axi_rvalid,
    output logic                        m_axi_rready
);
```

**Key Points:** - No registers - purely combinational - RLAST from AXI4 is ignored (always 1 for AXIL4) - RID from AXI4 is ignored (no ID tracking in AXIL4)

## 12.5 3.3.5 Write Path (axil4_to_axi4_wr)

### 12.5.1 Figure 3.5: AXI4-Lite to AXI4 Write Path




*AXIL4 to AXI4 Write*

### 12.5.2 Implementation

```
module axil4_to_axi4_wr #(
    // Width Configuration
    parameter int AXI_ID_WIDTH      = 8,
    parameter int AXI_ADDR_WIDTH    = 32,
    parameter int AXI_DATA_WIDTH    = 32,
    parameter int AXI_USER_WIDTH    = 1,

    // Default Values for AXI4-only Fields
    parameter int DEFAULT_ID        = 0,
    parameter int DEFAULT_REGION    = 0,
    parameter int DEFAULT_QOS       = 0,

    // Skid Buffer Depths (for timing closure)



    // Calculated Parameters
    localparam int STRB_WIDTH = AXI_DATA_WIDTH / 8,
    localparam int SIZE_VAL   = $clog2(STRB_WIDTH)  // AWSIZE for full
width
```

```
) (
    // Clock and Reset
    input  logic                            aclk,
    input  logic                            aresetn,


//=============================================================================
======
    // Slave AXI4-Lite Write Interface (Input - Simplified Protocol)

//=============================================================================
======

    // Write Address Channel
    input  logic [AXI_ADDR_WIDTH-1:0]   s_axil_awaddr,
    input  logic [2:0]                  s_axil_awprot,
    input  logic                        s_axil_awvalid,
    output logic                        s_axil_awready,

    // Write Data Channel
    input  logic [AXI_DATA_WIDTH-1:0]   s_axil_wdata,
    input  logic [STRB_WIDTH-1:0]       s_axil_wstrb,
    input  logic                        s_axil_wvalid,
    output logic                        s_axil_wready,

    // Write Response Channel
    output logic [1:0]                  s_axil_bresp,
    output logic                        s_axil_bvalid,
    input  logic                        s_axil_bready,


//=============================================================================
======
    // Master AXI4 Write Interface (Output - Full Protocol)

//=============================================================================
======

    // Write Address Channel
    output logic [AXI_ID_WIDTH-1:0]     m_axi_awid,
    output logic [AXI_ADDR_WIDTH-1:0]   m_axi_awaddr,
    output logic [7:0]                  m_axi_awlen,
    output logic [2:0]                  m_axi_awsize,
    output logic [1:0]                  m_axi_awburst,
    output logic                        m_axi_awlock,
    output logic [3:0]                  m_axi_awcache,
    output logic [2:0]                  m_axi_awprot,
```

```
    output logic [3:0]                  m_axi_awqos,
    output logic [3:0]                  m_axi_awregion,
    output logic [AXI_USER_WIDTH-1:0]   m_axi_awuser,
    output logic                        m_axi_awvalid,
    input  logic                        m_axi_awready,

    // Write Data Channel
    output logic [AXI_DATA_WIDTH-1:0]   m_axi_wdata,
    output logic [STRB_WIDTH-1:0]       m_axi_wstrb,
    output logic                        m_axi_wlast,
    output logic [AXI_USER_WIDTH-1:0]   m_axi_wuser,
    output logic                        m_axi_wvalid,
    input  logic                        m_axi_wready,

    // Write Response Channel
    input  logic [AXI_ID_WIDTH-1:0]     m_axi_bid,
    input  logic [1:0]                  m_axi_bresp,
    input  logic [AXI_USER_WIDTH-1:0]   m_axi_buser,
    input  logic                        m_axi_bvalid,
    output logic                        m_axi_bready
);
```

## 12.6 3.3.6 Bidirectional Wrapper

```
module axil4_to_axi4 #(
    parameter int AXI_ID_WIDTH   = 8,
    parameter int AXI_ADDR_WIDTH = 32,
    parameter int AXI_DATA_WIDTH = 32,
    parameter int AXI_USER_WIDTH = 1,
    parameter int DEFAULT_ID     = 0,
    parameter int DEFAULT_REGION = 0,
    parameter int DEFAULT_QOS    = 0
) (
    // ... all port declarations
);

    axil4_to_axi4_rd #(
        .AXI_ID_WIDTH    (AXI_ID_WIDTH),
        .AXI_ADDR_WIDTH  (AXI_ADDR_WIDTH),
        .AXI_DATA_WIDTH  (AXI_DATA_WIDTH),
        .AXI_USER_WIDTH  (AXI_USER_WIDTH),
        .DEFAULT_ID      (DEFAULT_ID),
        .DEFAULT_REGION  (DEFAULT_REGION),
        .DEFAULT_QOS     (DEFAULT_QOS)
    ) u_rd_converter (/* connections */);

    axil4_to_axi4_wr #(
        .AXI_ID_WIDTH    (AXI_ID_WIDTH),
        .AXI_ADDR_WIDTH  (AXI_ADDR_WIDTH),
        .AXI_DATA_WIDTH  (AXI_DATA_WIDTH),
        .AXI_USER_WIDTH  (AXI_USER_WIDTH),
        .DEFAULT_ID      (DEFAULT_ID),
        .DEFAULT_REGION  (DEFAULT_REGION),
        .DEFAULT_QOS     (DEFAULT_QOS)
    ) u_wr_converter (/* connections */);

endmodule
```

## 12.7 3.3.7 Resource Utilization

*Table 3.12: AXIL4 to AXI4 Resources*

| Module | Registers | LUTs |
|---|---|---|
| axil4_to_axi4_rd | 0 | ~50 |
| axil4_to_axi4_wr | 0 | ~60 |
| axil4_to_axi4 (combined) | 0 | ~110 |

**Note:** Zero registers — purely combinational logic.

## 12.8 3.3.8 Timing

*Table 3.13: AXIL4 to AXI4 Performance*

| Metric | Value |
|---|---|
| Latency | 0 cycles |
| Throughput | 100% |
| Max frequency | Wire speed |

## 12.9 3.3.9 Testing

**Test Suite:** 14 tests passing

*Table 3.14: Test Coverage Summary*

| Test Category | Tests | Status |
|---|---|---|
| Single-beat read | 3 | Pass |
| Single-beat write | 3 | Pass |
| Mixed traffic | 4 | Pass |
| Default ID verification | 2 | Pass |
| Edge cases | 2 | Pass |

## 12.10 3.3.10 Usage Example

```
axil4_to_axi4 #(
    .AXI_DATA_WIDTH (32),
    .AXI_ADDR_WIDTH (32),
    .AXI_ID_WIDTH   (8),
    .DEFAULT_ID     (8'h05),
    .DEFAULT_QOS    (0)
) u_axil2axi (
    .aclk           (aclk),
    .aresetn        (aresetn),
    // AXI4-Lite slave side: s_axil_*
    .s_axil_arvalid (lite_arvalid),
    .s_axil_arready (lite_arready),
    .s_axil_araddr  (lite_araddr),
    // ...
    // AXI4 master side: m_axi_* (ID/len/size driven from parameters;
    // the conversion is combinational, no internal pipelining)
    .m_axi_arvalid  (fab_arvalid),
    .m_axi_arready  (fab_arready)
    // ...
);
```

---

**Next:** AXI4 to APB



---

# 13 3.4 AXI4 to APB Converter

The **axi4_to_apb4_convert** module provides full protocol translation from AXI4 to APB, enabling AXI4 masters to access APB peripherals.

## 13.1 3.4.1 Overview

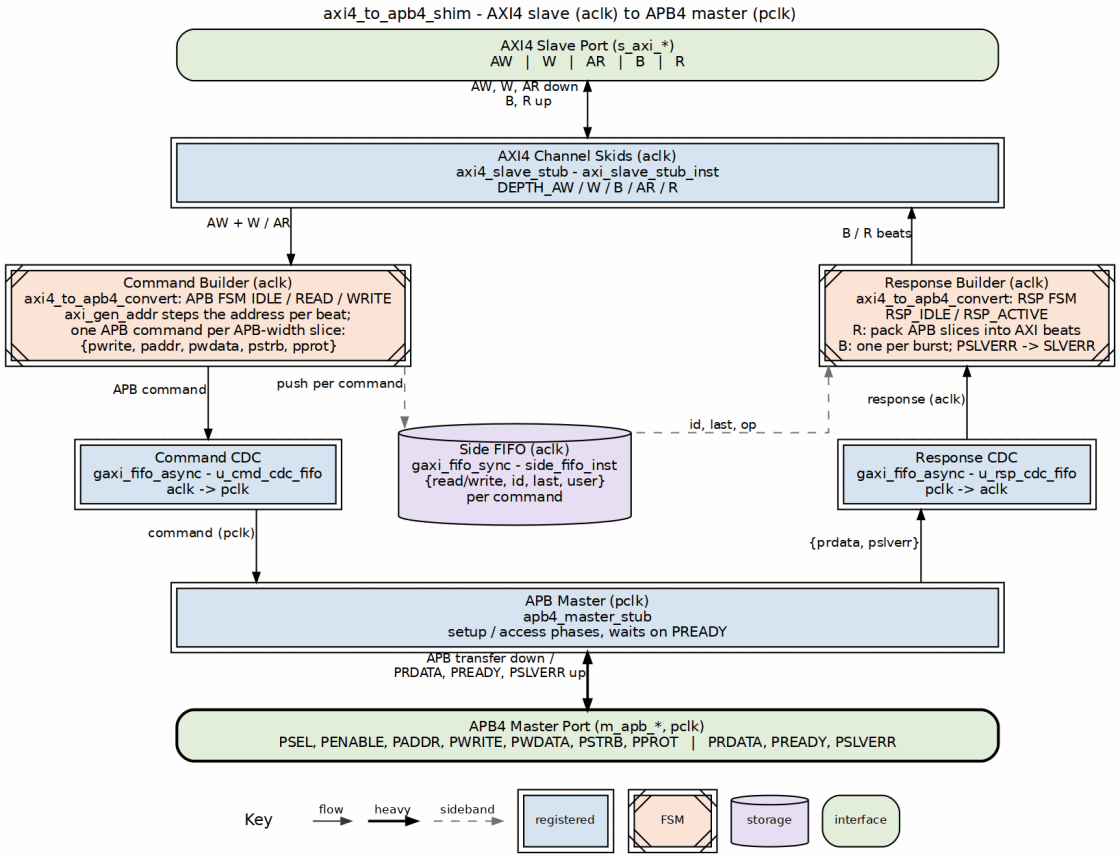
AXI4 and APB differ on just about every axis:

*Table 3.15: AXI4 vs APB Comparison*

| Aspect | AXI4 | APB |
|---|---|---|
| Channels | 5 (AW, W, B, AR, R) | 1 (combined) |
| Phases | Pipelined | 2-phase (setup, access) |
| Bursts | Up to 256 beats | Single transfer |
| Address width | Up to 64 bits | Typically 32 bits |
| Data width | 8-1024 bits | 8-32 bits |

## 13.2 3.4.2 Block Diagram

### 13.2.1 Figure 3.6: AXI4 to APB Converter




*AXI4 to APB*

## 13.3 3.4.3 Interface Specification

### 13.3.1 Parameters

**`axi4_to_apb4_convert`** — the conversion core:

*Table 3.16: AXI4 to APB Parameters*

| Parameter | Type | Default | Description |
|---|---|---|---|
| AXI_ADDR_WIDTH | int | 32 | AXI4 address width |
| AXI_DATA_WIDTH | int | 32 | AXI4 data width |
| AXI_ID_WIDTH | int | 8 | AXI4 ID width |
| AXI_USER_WIDTH | int | 1 | AXI4 user-signal width |
| APB_ADDR_WIDTH | int | 32 | APB address width |
| APB_DATA_WIDTH | int | 32 | APB data width |
| SIDE_DEPTH | int | 6 | Side-channel FIFO depth (ID/user carried past the APB hop) |

**`axi4_to_apb4_shim`** — the wrapper integrators actually instantiate. It adds the channel FIFOs around the core, and is what the bridge generator emits:

*Table 3.17: AXI4 to APB Shim Parameters*

| Parameter | Type | Default | Description |
|---|---|---|---|
| DEPTH_AW / DEPTH_AR | int | 2 | Write/read address channel FIFO depth |
| DEPTH_W / DEPTH_R | int | 4 | Write/read data channel FIFO depth |
| DEPTH_B | int | 2 | Write response channel FIFO |

| Parameter | Type | Default | Description |
|---|---|---|---|
| | | | depth |
| SIDE_DEPTH | int | 4 | Side-channel FIFO depth |
| APB_CMD_DEPTH | int | 4 | APB command FIFO depth |
| APB_RSP_DEPTH | int | 4 | APB response FIFO depth |
| USE_JOHNSON | int | 0 | CDC-FIFO pointer encoding: 0 = Gray (power-of-2 depth only), 1 = Johnson |
| USE_2_PHASE_CDC | bit | 1 | **Deprecated and ignored.** Declared for compatibility and never referenced, so setting it changes nothing |
| AXI_/*APB*_ widths | int | as above | Same width parameters as the core |

The shim's defaults differ from the core's `SIDE_DEPTH` (4 vs 6); it sets its own rather than inheriting.

### 13.3.2 Ports

```
// The conversion CORE. It does not sit on the AXI4 channels directly:
// the shim packs each channel into a packet plus a count and hands
those
// in, which is why the ports below are r_s_axi_*_pkt rather than the
raw
// AXI signals. Integrators instantiate axi4_to_apb4_shim instead; its
// parameters are in the shim table above.

module axi4_to_apb4_convert #(
    parameter int SIDE_DEPTH        = 6,
    parameter int AXI_ID_WIDTH      = 8,
```

```
    parameter int AXI_ADDR_WIDTH    = 32,
    parameter int AXI_DATA_WIDTH    = 32,
    parameter int AXI_USER_WIDTH    = 1,
    parameter int APB_ADDR_WIDTH    = 32,
    parameter int APB_DATA_WIDTH    = 32,
    parameter int AXI_WSTRB_WIDTH   = AXI_DATA_WIDTH / 8,
    parameter int APB_WSTRB_WIDTH   = APB_DATA_WIDTH / 8,
    // short and calculated params
    parameter int AW                = AXI_ADDR_WIDTH,
    parameter int DW                = AXI_DATA_WIDTH,
    parameter int IW                = AXI_ID_WIDTH,
    parameter int UW                = AXI_USER_WIDTH,
    parameter int SW                = AXI_DATA_WIDTH / 8,
    parameter int APBAW             = APB_ADDR_WIDTH,
    parameter int APBDW             = APB_DATA_WIDTH,
    parameter int APBSW             = APB_DATA_WIDTH / 8,
    parameter int AXI2APBRATIO      = DW / APBDW,
    parameter int AWSize            = IW + AW + 8 + 3 + 2 + 1 + 4 + 3
+ 4 + 4 + UW,
    parameter int WSize             = DW + SW + 1 + UW,
    parameter int BSize             = IW + 2 + UW,
    parameter int ARSize            = IW + AW + 8 + 3 + 2 + 1 + 4 + 3
+ 4 + 4 + UW,
    parameter int RSize             = IW + DW + 2 + 1 + UW,
    parameter int APBCmdWidth       = APBAW + APBDW + APBSW + 3 + 1 +
1 + 1,
    parameter int APBRspWidth       = APBDW + 1 + 1 + 1,
    parameter int SideSize          = 1 + IW + 1 + UW
) (
    // Clock and Reset
    input  logic                    aclk,
    input  logic                    aresetn,

    // Inputs from axi_slave_stub
    input  logic [AWSize-1:0]       r_s_axi_aw_pkt,
    input  logic [3:0]              r_s_axi_aw_count,
    input  logic                    r_s_axi_awvalid,
    output logic                    w_s_axi_awready,

    input  logic [WSize-1:0]        r_s_axi_w_pkt,
    input  logic                    r_s_axi_wvalid,
    output logic                    w_s_axi_wready,

    output logic [BSize-1:0]        r_s_axi_b_pkt,
    output logic                    w_s_axi_bvalid,
    input  logic                    r_s_axi_bready,
```

```
    input  logic [ARSize-1:0]       r_s_axi_ar_pkt,
    input  logic [3:0]              r_s_axi_ar_count,
    input  logic                    r_s_axi_arvalid,
    output logic                    w_s_axi_arready,

    output logic [RSize-1:0]        r_s_axi_r_pkt,
    output logic                    w_s_axi_rvalid,
    input  logic                    r_s_axi_rready,

    // APB Master Interface
    output logic                    w_cmd_valid,
    input  logic                    r_cmd_ready,
    output logic [APBCmdWidth-1:0]  r_cmd_data,

    input  logic                    r_rsp_valid,
    output logic                    w_rsp_ready,
    input  logic [APBRspWidth-1:0]  r_rsp_data
);
```

## 13.4 3.4.4 Clock Domains

The shim is a **two-clock** block. `aclk` runs the AXI side and `pclk` the APB side, with independent resets (`aresetn`, `presetn`). Commands cross aclk→pclk and responses pclk→aclk through gray-pointer asynchronous FIFOs (`gaxi_fifo_async`).

The reset behaviour is the part worth knowing. Each domain resets its own pointer and its crossed copy of the remote pointer from its LOCAL reset, so resetting one side alone leaves that side self-consistent — both pointers at 0, meaning empty. Because the pointers are absolute positions rather than toggle parity, an independent reset of one side cannot fabricate or swallow a transfer. The earlier 2-phase handshake could, and the failure was permanent rather than transient: the response stream ends up offset by one and every read returns the previous read's data.

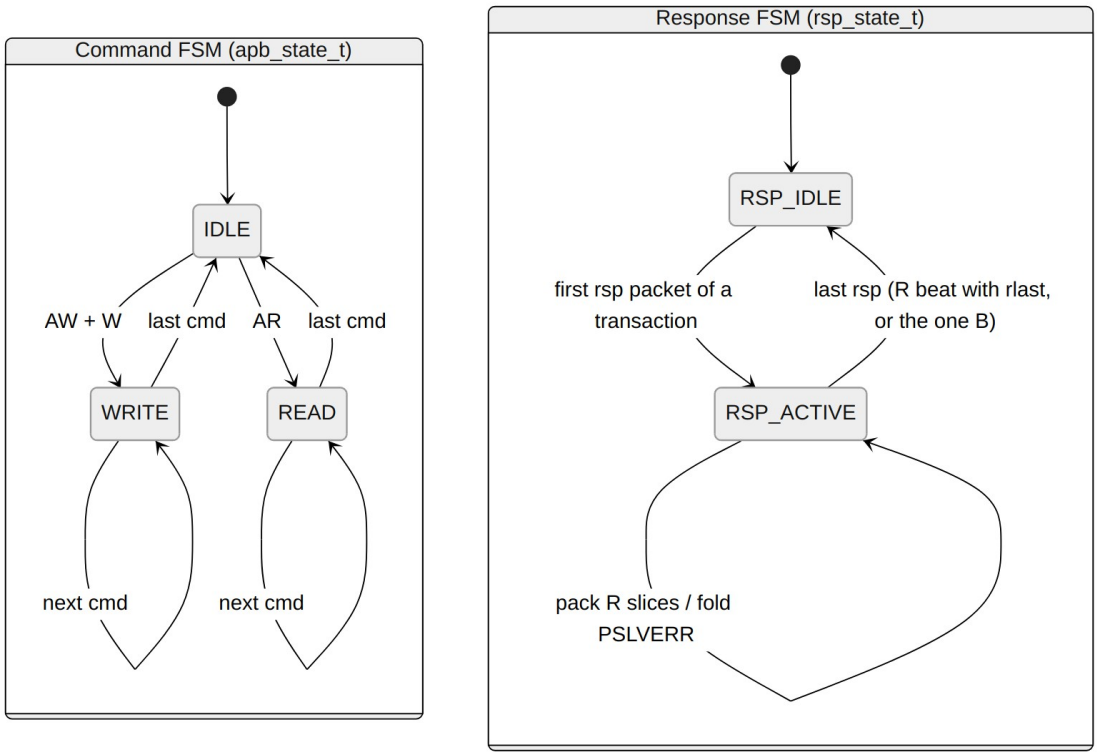
CDC FIFO depths follow the APB cmd/rsp queue depths, floored at 4:

```
localparam int CDC_CMD_DEPTH = (APB_CMD_DEPTH < 4) ? 4 :
APB_CMD_DEPTH;
localparam int CDC_RSP_DEPTH = (APB_RSP_DEPTH < 4) ? 4 :
APB_RSP_DEPTH;
```

A power-of-2 depth is preferred: the default `USE_JOHNSON=0` Gray encoding requires it (see the parameter table above).

## 13.5 3.4.5 State Machine

### 13.5.1 Figure 3.7: AXI4 to APB FSM




*AXI4 to APB FSM*

### 13.5.2 States

The convert core runs TWO state machines, and neither of them touches PSEL/PENABLE — the APB setup/access phases belong to `apb4_master`, on the far side of the shim's CDC. The core's job is packetizing.

**Command FSM** — walks the accepted AXI burst and emits one APB command packet per APB-width beat into the cmd stream:

```
typedef enum logic [2:0] {
    IDLE  = 3'b001,   // wait for an accepted AW+W pair or AR
    READ  = 3'b010,   // emit one read command per beat
    WRITE = 3'b100    // emit one write command per beat
} apb_state_t;
```

Writes are preferred over reads when both are pending. Each command carries `first`/`last` flags; `last` is set on the final beat, which is also when the AXI address channel is released (`awready`/`arready`). A data pointer sub-divides each AXI beat into `AXI2APBRATIO` APB beats

when the widths differ, and the next address comes from the shared `axi_gen_addr` (INCR/FIXED/WRAP per the AXI burst type).

**Response FSM** — assembles returning rsp packets into AXI responses:

```
typedef enum logic [1:0] {
    RSP_IDLE   = 2'b01,   // no response in flight
    RSP_ACTIVE = 2'b10    // consuming rsp packets for one transaction
} rsp_state_t;
```

Read data re-packs APB-width beats into AXI-width R beats; write responses collapse into the single B, worst response winning. The `first` flag in the command stream is what arms this FSM — stamping it from transaction progress rather than the previous command state is the fix for a hang recorded in the RTL history (a `first=0` first command froze the response FSM in RSP_IDLE).

## 13.6 3.4.6 Burst Handling

### 13.6.1 Burst Decomposition

AXI4 bursts are decomposed into sequential APB transfers:

```
AXI4: AWADDR=0x1000, AWLEN=3 (4 beats)

APB sequence:
  Transfer 0: PADDR=0x1000
  Transfer 1: PADDR=0x1004
  Transfer 2: PADDR=0x1008
  Transfer 3: PADDR=0x100C
```

### 13.6.2 Address Calculation

The per-beat address comes from the shared `axi_gen_addr` block, keyed on the AXI burst type (INCR/FIXED/WRAP) and size — the convert core does not roll its own increment, and there is no APB-phase state here (PREADY pacing happens in `apb4_master` beyond the CDC; see 3.4.5).

## 13.7 3.4.7 Address Width Adaptation

### 13.7.1 64-bit to 32-bit Conversion

```
// Truncate upper address bits
assign paddr = s_awaddr[APB_ADDR_WIDTH-1:0];

// Optional: Check for out-of-range access
wire w_addr_oor = |s_awaddr[AXI_ADDR_WIDTH-1:APB_ADDR_WIDTH];
```

## 13.8 3.4.8 Error Response Mapping

*Table 3.18: Error Mapping*

| APB Signal | AXI4 Response |
|---|---|
| PSLVERR = 0 | OKAY (2'b00) |
| PSLVERR = 1 | SLVERR (2'b10) |

### 13.8.1 Error Aggregation

Each rsp packet carries the APB `pslverr` for its beat. The response FSM accumulates across the transaction's packets and maps any error to SLVERR on the AXI side — B for writes, per-beat RRESP for reads (the error lands on the beats it belongs to, not smeared across the burst). There is no APB-phase sampling here; `pslverr` is captured by `apb4_master` and travels back in the packet.

## 13.9 3.4.9 Implementation

The implementation is the two FSMs of 3.4.5 plus the packet plumbing: the shim skid-buffers each AXI channel, packs AW/W/AR into per-channel packets for the core, and carries the core's cmd/rsp streams through gray-pointer async FIFOs to `apb4_master`, which owns the actual APB setup/access phases. The full source is `projects/components/utility-ip/converters/rtl/axi4_to_apb4_convert.sv` (core) and `axi4_to_apb4_shim.sv` (integration); their headers document the packet formats. No separate simplified listing is maintained here — an earlier one drifted into describing states the RTL never had.

## 13.10 3.4.10 Resource Utilization

```
State machine:        ~50 LUTs, ~20 regs
Address logic:        ~30 LUTs, ~40 regs
Data buffering:       ~10 LUTs, ~70 regs
Control:              ~60 LUTs, ~20 regs

Total: ~150 LUTs, ~150 regs
```

## 13.11 3.4.11 Timing

### 13.11.1 Timing Analysis

*Table 3.19: APB Converter Timing*

| Operation | Cycles |
|---|---|
| Single write | 3-4 APB-side + ~2 pclk in + ~2 aclk out for the CDC crossings (see 3.4.4) |
| Single read | 3-4 (setup + access + R) |
| N-beat write burst | ~2N+1 APB-side + the same two CDC crossings, paid once per command run |
| N-beat read burst | ~2N+1 APB-side |

The 2N+1 shape falls out of the FSM: the first transfer pays IDLE→SETUP→ACCESS (3 pclk), but with commands queued in the skid the ACCESS state hands straight back to SETUP — every subsequent transfer is SETUP→ACCESS, 2 pclk, PREADY permitting.

### 13.11.2 Throughput

**Best case (sustained):** 1 transfer per 2 pclk cycles once the first transfer's extra IDLE cycle is paid
**With slow PREADY:** Additional cycles per transfer

## 13.12 3.4.12 Usage Example

Integrators instantiate the SHIM (the core's cmd/rsp packet ports are internal plumbing):

```
axi4_to_apb4_shim #(
    .AXI_ID_WIDTH     (8),
    .AXI_ADDR_WIDTH   (32),
    .AXI_DATA_WIDTH   (32),
    .APB_ADDR_WIDTH   (32),
    .APB_DATA_WIDTH   (32),
    .APB_CMD_DEPTH    (4),
    .APB_RSP_DEPTH    (4)
) u_axi2apb (
    // AXI side clock/reset
    .aclk             (aclk),
    .aresetn          (aresetn),
    // APB side clock/reset -- this is a TWO-CLOCK block (see 3.4.4)
    .pclk             (pclk),
    .presetn          (presetn),

    // AXI4 slave interface: s_axi_aw*/w*/b*/ar*/r* (full signal list
in
    // the module header)
    .s_axi_awvalid    (cpu_awvalid),
    .s_axi_awready    (cpu_awready),
    // ...

    // APB master interface
    .m_apb_PSEL       (periph_psel),
    .m_apb_PENABLE    (periph_penable),
    .m_apb_PADDR      (periph_paddr),
    .m_apb_PWRITE     (periph_pwrite),
    .m_apb_PWDATA     (periph_pwdata),
    .m_apb_PSTRB      (periph_pstrb),
    .m_apb_PPROT      (periph_pprot),
    .m_apb_PRDATA     (periph_prdata),
    .m_apb_PSLVERR    (periph_pslverr),
    .m_apb_PREADY     (periph_pready)
);
```

---

**Next:** AXI4 to APB5 Shim

## 13.13 3.4.12 Design Decision: Inline Width Conversion

The converter narrows AXI write data to the APB width and widens APB read data back with its own pointer logic rather than instantiating `axi_data_dnsize` / `axi_data_upsize` (Chapter 2). This was examined on 2025-10-25 and kept deliberately:

- The conversion is incremental and interlocked with the protocol FSM: the pointer advances on each APB transfer's completion, and FIRST/LAST for the side queue depend on both the burst state and the conversion state. The generic blocks consume whole beats and would need glue FIFOs and a second state machine around them for no functional gain.
- A refactor would add latency and registers while making the design harder to reason about and to cover in verification.
- The two implementations were compared line for line and use the same algorithm (lane pointer = address bits above the APB width, same strobe slicing, same accumulate-then-present read path), so the generic blocks were independently validated by this converter rather than duplicated by it.

Standing rule: `axi4_dwidth_converter_{wr,rd}` (Chapter 2) are the reusable width converters; this block keeps its inline conversion. (The analysis note that recorded this, `ANALYSIS_APB_CONVERTER.md`, lived at the component root until 2026-09-29 and was folded into this section; `git log --follow` has it.)

# 14 AXI4 to APB5 Shim

**Module:** `axi4_to_apb5_shim.sv` **Filelist:** `rtl/filelists/axi4_to_apb5_shim.f` (-f's the APB4 shim's closure)

## 14.1 Overview

APB5 changes less than the version bump suggests. The transfer protocol is pure APB4 — PSEL/PENABLE phases, PREADY, PSLVERR, all unchanged — and everything new lives in the sideband: requester-driven user and wake-up signals on one side, completer-driven user signals on the other. So this block is a thin **pin-superset wrapper** over `axi4_to_apb4_shim`. The entire protocol engine — CDC, cmd/rsp FIFOs, burst decomposition, response assembly — is the APB4 shim instantiated unchanged, with all 58 ports and 18 parameters forwarded 1:1.

The APB5 additions on the requester surface:

| Signal | Dir | Handling |
|---|---|---|
| `m_apb_PAUSER[APB_AUSER_WIDTH-1:0]` | out | tied `'0` — nothing upstream sources it (AXI USER bits do not map onto APB user semantics) |
| `m_apb_PWUSER[APB_WUSER_WIDTH-1:0]` | out | tied `'0` |
| `m_apb_PWAKEUP` | out | registered copy of `m_apb_PSEL` – see below |
| `m_apb_PRUSER[APB_RUSER_WIDTH-1:0]` | in | accepted and terminated |
| `m_apb_PBUSER[APB_BUSER_WIDTH-1:0]` | in | accepted and terminated |

One thing to get right before you wire this up. User-signal widths default to 1 here, but `rtl/amba/apb5/apb5_slave.sv` defaults its own to 4, so the two do NOT line up out of the box — set `APB_{A,W,R,B}USER_WIDTH` to match whatever completer you attach. The signal set is otherwise pin-for-pin with that slave.

PWAKEUP is a requester output (IHI 0024E 3.7.2, Table 3-7). This shim used to take it as an input, following an old repo convention that had the direction backwards (GitHub #102 item 7). It is now a flop of `m_apb_PSEL`: glitch-free because it comes straight from a register, high from the cycle after the Setup phase, and held through the Access phase up to and including the PREADY cycle, so the 3.7.2 rule that PWAKEUP stays asserted until PREADY holds, and a Completer that waits for PWAKEUP before driving PREADY always gets it. It falls when no transfer is in progress. 3.7.2 permits PWAKEUP before, during or after PSEL. The recommended one-cycle lead over PSEL is not available, because the command queue that would predict the next transfer sits inside the wrapped APB4 shim, whose port list every APB4 consumer shares. No check signals are carried (Check_Type = False), so there is no PWAKEUPCHK.

## 14.2 Parameters

The width and depth parameters are the same as the APB4 shim's – see Table 3.17 – with the APB5 user-signal widths (`APB_AUSER_WIDTH`, `APB_WUSER_WIDTH`, `APB_RUSER_WIDTH`, `APB_BUSER_WIDTH`) added for the sideband above.

One parameter is inert, and I'd rather call it out here than watch someone wire a knob to it:

*AXI4 to APB5 Shim – Inert Parameter*

| Parameter | Type | Default | Description |
|---|---|---|---|
| `USE_2_PHASE_CDC` | bit | 1 | **Deprecated and ignored.** This shim wraps `axi4_to_apb4_shim` and forwards the parameter to it; that module declares it `// deprecated, ignored` and never references it, so it has no effect at any level |

## 14.3 Design Notes

Deriving the wrapper from the APB4 shim's actual port surface — rather than reimplementing the conversion — means the APB4 engine's fixes and its dependency closure are inherited automatically. The closure filelist simply `-f`'s the APB4 shim's. When a future consumer needs real PAUSER/PWUSER sourcing or a PWAKEUP that leads PSEL, those grow here without touching the conversion core.

## 14.4 Related Modules

The bridge generator instantiates this shim for `protocol = "apb5"` slaves (bridge TASK-002 (was BRIDGE-002) A5-3c) through the same component path as the APB4 shim — the `Axi4ToApbShim` component takes `protocol='apb5'` and wires the five extra pairs. Verified by `projects/components/fabric-gen-ip/bridge/dv/tests/test_bridge_1x2_rw_apb5.py` (APB4 BFM legally drives the port: same transfer protocol).

## 14.5 Navigation

**Next:** AXI4 to AXI5-Lite

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

# 15 AXI4 to AXI5-Lite

**Modules:** `axi4_to_axil5.sv`, `axi4_to_axil5_wr.sv`, `axi4_to_axil5_rd.sv` **Filelists:** `rtl/filelists/axi4_to_axil5{,_wr,_rd}.f` (each `-f`'s the AXI4-Lite converter's closure)

## 15.1 Overview

AXI5-Lite changes less than the version bump suggests. The transfer protocol is AXI4-Lite – single-beat, no burst, same handshakes and response codes – plus a transaction ID (AWID/BID, ARID/RID) and a few optional signals (IHI 0022L section B2.1.5, Table B2.2). So these blocks are thin wrappers over `axi4_to_axil4_{wr,rd}`. Burst decomposition, address incrementing, response folding and the one-outstanding guards are that module, instantiated unchanged. What lives here is the ID, the optional signals, and one timing decision they force.

`axi4_to_axil5` pairs the two halves exactly as `axi4_to_axil4` pairs its own, and contains nothing but the two instantiations.

The AXI4 slave surface is the **full** AXI4 interface, not a subset. A master upstream of this converter may be feeding a fabric whose other branches reach AXI4 slaves, so nothing is dropped from the boundary on the grounds that this particular downstream path cannot use it.

## 15.2 Where each signal gets its value

Every AXI5-Lite signal is in one of four groups. This is the whole content of the block, so it is worth reading as a table rather than as prose.

*AXI4 to AXI5-Lite – Signal Disposition*

| Group | Signals | Handling |
|---|---|---|
| **ID** | `awid`/`arid` out; `bid` -> `s_axi_bid`, `rid` -> `s_axi_rid` | AXI4 ID of the burst the beat belongs to; the response carries the ID the AXI5-Lite completer returned. `AXI_ID_WIDTH` bits |
| **FORWARDED** | `awuser`/`aruser`, `wuser` | carries the AXI4 value, gated by `ENABLE_USER` |
| **FORWARDED (response)** | `buser` -> `s_axi_buser`, `ruser` -> `s_axi_ruser` | returned onto the AXI4 response channel, gated by `ENABLE_USER` |
| **TIED** | `awtrace`, `wpoison`, `artrace` | driven `'0`; AXI4 has no source for them |
| **TERMINATED** | `btrace`, `rtrace`, `rpoison` | completer-driven, nothing on the AXI4 side to return them to |

**What is not there.** AxLOCK, AxMPAM, AxMECID, AxNSAID and xLOOP are not present on AXI5-Lite, and exclusive access is not supported (IHI 0022L B2.1.5, Table B2.2). Until issue #102 these converters carried them (LOCK forwarded, the rest tied to zero); they now have no such ports and no `ENABLE_LOCK`, `LOOP_WIDTH`, `MPAM_WIDTH`, `MECID_WIDTH` or `NSAID_WIDTH` parameters. The AXI4 `AxLOCK` input is dropped. An AXI4 exclusive access through this converter therefore completes as a normal access with an OKAY response, which is exactly what AXI4 specifies for a subordinate without exclusive support: OKAY, not EXOKAY, tells the master the exclusive failed, and an exclusive write still updates memory (IHI 0022H A7.2.3, A7.2.5).

**The tied group has no `ENABLE_` parameter, and that is deliberate.** A knob that cannot change the design's behaviour is worse than no knob: a reader sets it and believes something happened. `ENABLE_USER` exists because it gates a real choice; `ENABLE_TRACE` and `ENABLE_POISON` do not exist at all. The `USER_WIDTH` parameter remains, because it sets the port shape.

**The tied group's ports still exist.** An AXI5-Lite boundary whose shape changes with a config knob cannot be wired to a fixed external completer, so the ports are always present and always driven – zero rather than floating.

## 15.3 IDs and response ordering

The AXI4 ID crosses unchanged: `m_axil_awid`/`m_axil_arid` carry it on every decomposed beat, held across the burst the same way the USER sideband is (see the next section), and `s_axi_bid`/`s_axi_rid` are driven from `m_axil_bid`/`m_axil_rid` – the ID the AXI5-Lite completer returned, which a compliant completer reflects from AxID (IHI 0022L A5.1.1).

This is correct for any mix of AXI4 IDs because the core holds **one** AXI4 burst in flight per direction: `s_axi_awready`/`s_axi_arready` stay low until the burst's B, or its RLAST beat, has been handed back. Every AXI5-Lite response therefore belongs to the single burst in flight. The completer is allowed to reorder responses across IDs, but there is never a second ID outstanding for it to reorder against, and responses with the same ID return in order. An AXI4 master issuing several IDs gets each transaction's response under its own ID, in issue order, which AXI4 permits (there is no ordering requirement across IDs). Using the returned ID rather than a locally stored copy means a completer that fails to reflect AxID is visible at the AXI4 boundary instead of being masked.

`rpoison` deserves its own note. AXI4 has no poison bit, so a poisoned read beat arriving from an AXI5-Lite completer cannot be signalled to the AXI4 master through any protocol field. It is terminated rather than folded into RRESP, because turning poison into SLVERR would invent an error the completer never reported. A design that must observe poison needs an AXI5 master port, not an AXI4 one.

## 15.4 The AW/AR sideband is held, not passed through

This is the one place the wrapper is not trivial, and it is worth understanding before modifying it.

The core **decomposes**: one AXI4 AW handshake becomes N AXI5-Lite AW handshakes, one per beat. And `axi4_to_axil4_wr` drops `s_axi_awready` as soon as it accepts the AW (`s_axi_awready = !r_aw_active && !r_wr_outstanding && ...`). So from the cycle after acceptance, an AXI4 master is free to present the **next** transaction's AW signals – new address, new ID, new USER – while beats 2..N of the current burst are still going out.

A combinational passthrough of the sideband therefore stamps those later beats with the wrong values. The core has the identical problem with the address and solves it with a capture-and-hold mux:

```
assign m_axil_awaddr = r_aw_active ? r_aw_addr : s_axi_awaddr;
```

`r_aw_active` is registered, so it is still 0 during the accept cycle itself and 1 for every beat after. The wrapper cannot see that signal, so it reconstructs the same selector from outside:

```
wire w_aw_accept = s_axi_awvalid && s_axi_awready;
// ... captured into r_held_aw{id,user} on w_aw_accept ...
wire [AXI_USER_WIDTH-1:0] w_awuser_sel = w_aw_accept ? s_axi_awuser
                                                     : r_held_awuser;
```

Accept cycle takes the live value; every beat after takes the captured one – identical to the core's mux, so the ID and sideband travel with the address they were issued against. `m_axil_awid` uses the same selector.

The first version of this RTL passed it through combinationally. The failure is not subtle once you can see it: with two overlapping writes, burst A's beats carry burst B's USER **from beat 0**, because B's AW is already on the bus. It is invisible to sequential traffic, which is why the test that catches it (`test_axi4_to_axil5.py`, `full` level) issues two writes concurrently rather than one after another.

W beats need no hold. The core assigns `m_axil_wdata = s_axi_wdata` directly – W is 1:1 with the AXI4 side – so `wuser` is a plain passthrough.

## 15.5 Reset style

These modules use the `ALWAYS_FF_RST` macro from `reset_defs.svh`, as do the AXI4-Lite converters they wrap, so one compiled design has one reset style. Mixing the macro with hand-written `always_ff @(posedge aclk or negedge aresetn)` blocks would make it half synchronous-reset and half asynchronous – Verilator says so with `SYNCASYNCNET`. The area conversion is recorded as projects/components/utility-ip/converters BUG-007 (was CONV-009).

## 15.6 Verification

`projects/components/utility-ip/converters/dv/tests/test_axi4_to_axil5.py` covers only what the wrapper adds; the transfer protocol is already covered by the AXI4-Lite suites, and duplicating it would test code this module does not contain. Eight configurations across three levels check that:

- the signals AXI5-Lite excludes (LOCK, LOOP, MPAM, MECID, NSAID) are absent from the DUT;
- every AXI5-Lite AW/AR beat carries its burst's AXI4 ID and USER, and every AXI4 B/R carries the ID the completer returned;
- several AXI4 IDs in flight at once (`func` and up) each complete under their own ID with their own data – the AXI4 master BFM picks responses up by ID, and the AXI5-Lite slave BFM reflects AxID and shares one memory between its read and write sides;
- tied signals read 0 (never X) at every handshake, the response USER returns on the AXI4 channels, and an `ENABLE_USER=0` build produces zeros from non-zero AXI4 traffic.

The overlapping-burst case was confirmed RED against the unfixed USER hold and GREEN after. For the ID, a mutant that passes `s_axi_awid` through combinationally fails 6 of 8 configurations, a mutant that zeroes `s_axi_rid` fails all 8, and the pre-#102 RTL (no ID ports) fails all 8.

## 15.7 Related Modules

The bridge generator instantiates these converters for `protocol = "axil5"` slaves through the same component path as the AXI4-Lite ones – the `Axi4ToAxilShim` component takes `protocol='axil5'`. Which sideband groups are live comes from the port's `axi5_features`. Since issue #102 the only gate the converter has is `ENABLE_USER`; the bridge generator's `axil5` handling (`bridge_pkg/axil5_sideband.py`) has to follow the new port list – no LOCK/LOOP/MPAM/MECID/NSAID, plus `m_axil_{aw,ar}id` and `m_axil_{b,r}id`. Verified by `projects/components/fabric-gen-ip/bridge/dv/tests/test_bridge_1x2_rw_axil5.py`.

## 15.8 Navigation

**Next:** PeakRDL Adapter

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 16 3.5 PeakRDL Adapter

The **peakrdl_to_cmdrsp** module drives a PeakRDL-generated register block from a command/response stream. Data flows cmd/rsp -> PeakRDL: `cmd_*` are inputs and `regblk_*` request signals are outputs, with the register block's acks and read data coming back in. Fair warning — the name reads in the opposite order to the dataflow. The ports below are authoritative.

## 16.1 Overview

PeakRDL generates register blocks with a selectable cpuif, and this adapter mates with the **passthrough** cpuif (`regblk_req` / `req_is_wr` / `wr_biten` / stall / ack / err — NOT the APB cpuif's PSEL/PENABLE pins, so if you came here looking for PSEL, back up). It:

1. Decouples the register interface from the implementation
2. Provides a clean handshake protocol
3. Enables pipelined register access
4. Supports custom control logic integration

## 16.2 Parameters

*Table 3.20: PeakRDL Adapter Parameters*

| Parameter | Type | Default | Description |
|---|---|---|---|
| ADDR_WIDTH | int | 12 | Address width (register-block sized) |
| DATA_WIDTH | int | 32 | Data width |

## 16.3 Ports

The full port list, straight from the RTL:

```
module peakrdl_to_cmdrsp #(
    parameter int ADDR_WIDTH = 12,  // Address width for cmd/rsp
interface
    parameter int DATA_WIDTH = 32   // Must match PeakRDL generation
(typically 32)
) (
    // Clock and Reset
    input  logic                    aclk,
    input  logic                    aresetn,

    //
================================================================================
    // CMD/RSP Interface (rtldesignsherpa standard)
    //
================================================================================
    // Command Channel
    input  logic                    cmd_valid,
    output logic                    cmd_ready,
    input  logic                    cmd_pwrite,         // 1=write,
0=read
    input  logic [ADDR_WIDTH-1:0]   cmd_paddr,          // Byte
address
    input  logic [DATA_WIDTH-1:0]   cmd_pwdata,         // Write data
    input  logic [DATA_WIDTH/8-1:0] cmd_pstrb,          // Byte
strobes

    // Response Channel
    output logic                    rsp_valid,
    input  logic                    rsp_ready,
    output logic [DATA_WIDTH-1:0]   rsp_prdata,         // Read data
    output logic                    rsp_pslverr,        // Error flag

    //
================================================================================
    // PeakRDL Passthrough Interface
    //
================================================================================
    output logic                    regblk_req,         // Request --
```

```
HELD until ack
    output logic                    regblk_req_is_wr,   // Write flag
    output logic [ADDR_WIDTH-1:0]   regblk_addr,        // Address
    output logic [DATA_WIDTH-1:0]   regblk_wr_data,     // Write data
    output logic [DATA_WIDTH-1:0]   regblk_wr_biten,    // Write bit
enables
    input  logic                    regblk_req_stall_wr, // Write
stall
    input  logic                    regblk_req_stall_rd, // Read stall
    input  logic                    regblk_rd_ack,      // Read
acknowledge
    input  logic                    regblk_rd_err,      // Read error
    input  logic [DATA_WIDTH-1:0]   regblk_rd_data,     // Read data
    input  logic                    regblk_wr_ack,      // Write
acknowledge
    input  logic                    regblk_wr_err       // Write error
);
```

## 16.4 Functional Description

Direction: cmd/rsp -> PeakRDL. `cmd_*` arrive from upstream (e.g. the APB shim's packet stream); the adapter drives the register block's `regblk_*` request signals and returns its acks as rsp packets.

### 16.4.1 Write

```
Cycle 0: cmd_valid, cmd_pwrite=1 accepted (cmd_ready high in
CMD_IDLE);
         regblk_req driven the same cycle, strobes converted to bit
         enables
Cycle N: regblk_wr_ack -> rsp queued (RSP_VALID), pslverr from
         regblk_wr_err
         (if regblk_req_stall_wr: state parks in CMD_STALLED and
         retries when the stall clears)
Cycle M: rsp_valid && rsp_ready -> back to idle

`regblk_req` is HELD asserted from acceptance through the ack cycle
(`(cmd_state == CMD_WAIT_ACK) || (CMD_IDLE && cmd_valid)`) -- it is
not a one-cycle strobe. The generated passthrough regblock samples the
request level and needs the hold; a consumer that starts a new access
every cycle req is high would double-fire non-idempotent registers
(TASK-064 records the earlier strobe mis-documentation).
```

### 16.4.2 Read

Same shape via `regblk_rd_ack`/`regblk_rd_data`/`regblk_rd_err`, with `rsp_prdata` carried in the response.

### 16.4.3 3.5.5 Implementation

Two small FSMs do all the work, from the RTL:

```
typedef enum logic [1:0] {
    CMD_IDLE     = 2'b00,   // ready to accept a command
    CMD_WAIT_ACK = 2'b01,   // register block has the request
    CMD_STALLED  = 2'b10    // req_stall_* asserted; retry
} cmd_state_t;

typedef enum logic {
    RSP_IDLE  = 1'b0,
    RSP_VALID = 1'b1        // response held until rsp_ready
} rsp_state_t;
```

The command is registered on acceptance so the request can be replayed out of CMD_STALLED; in CMD_IDLE the request muxes straight from the live cmd inputs, so an unstalled single-cycle ack

costs no extra latency. `cmd_ready = (cmd_state == CMD_IDLE)` gives one outstanding command, which is what a register block wants.

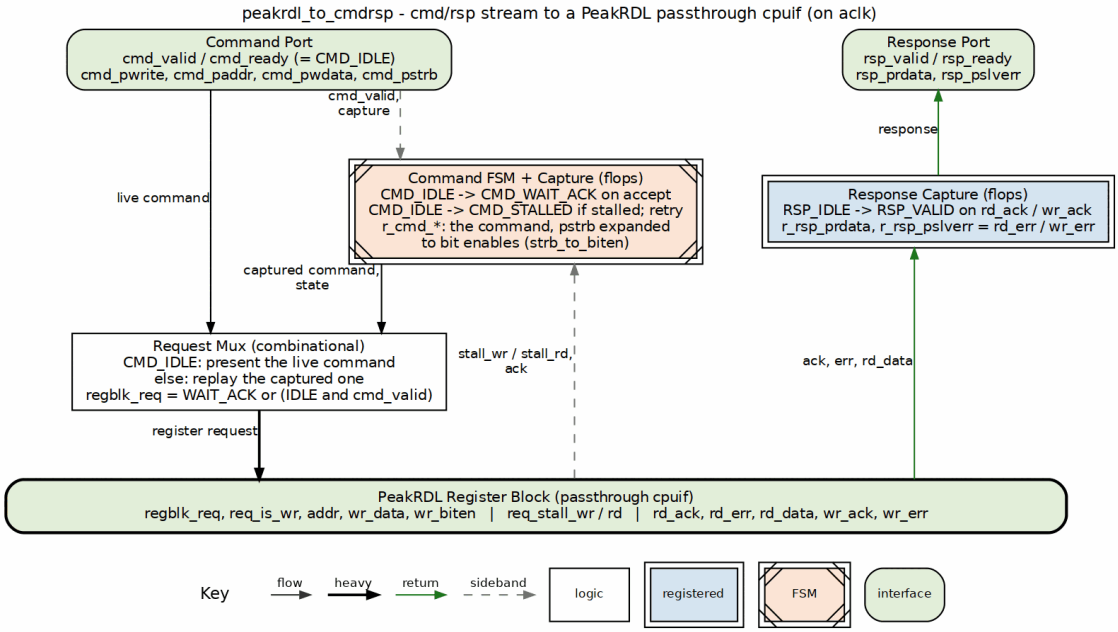
### 16.4.4 3.5.6 Resource Utilization

```
FSM state:            3 regs (cmd_state 2b + rsp_state 1b)
Command capture:      77 regs (pwrite 1 + paddr 12 + pwdata 32 +
wr_biten 32)
Response capture:     33 regs (prdata 32 + pslverr 1)
Request replay muxes: ~77 LUTs (four 2:1 selects, 1+12+32+32 bits --
                      IDLE-vs-registered request presentation)
FSM + control decode: ~25 LUTs

Total: ~113 regs, ~100 LUTs  (counted from the declarations at the
documented defaults ADDR_WIDTH=12, DATA_WIDTH=32; all of it is live
state -- the command capture replays the request out of
CMD_STALLED/CMD_WAIT_ACK, and the response capture holds the rsp
packet)
```

## 16.5 Block Diagram

### 16.5.1 Figure 3.8: PeakRDL Adapter




*PeakRDL Adapter*

## 16.6 Usage Example

### 16.6.1 3.5.7 Use Cases

The adapter sits between a command/response packet stream and a PeakRDL-generated register block:

```
peakrdl_to_cmdrsp #(
    .ADDR_WIDTH (12),
    .DATA_WIDTH (32)
) u_adapter (
    .aclk                (aclk),
    .aresetn             (aresetn),

    // command/response stream (e.g. from apb4_slave_cdc's unpacker)
    .cmd_valid           (cmd_valid),
    .cmd_ready           (cmd_ready),
    .cmd_pwrite          (cmd_pwrite),
    .cmd_paddr           (cmd_paddr),
    .cmd_pwdata          (cmd_pwdata),
    .cmd_pstrb           (cmd_pstrb),
    .rsp_valid           (rsp_valid),
    .rsp_ready           (rsp_ready),
    .rsp_prdata          (rsp_prdata),
    .rsp_pslverr         (rsp_pslverr),

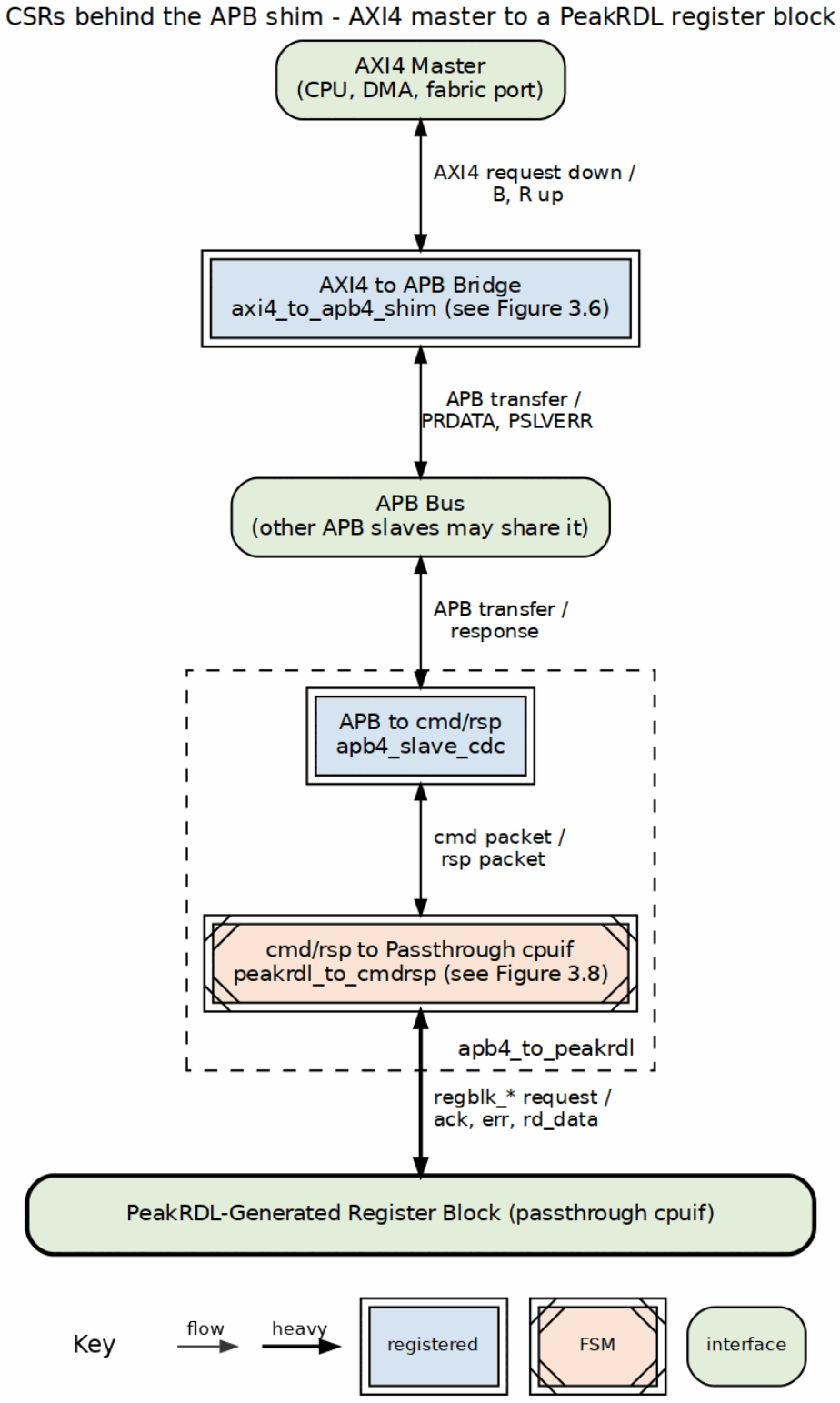
    // PeakRDL register block hookup
    .regblk_req          (hwif_req),
    .regblk_req_is_wr    (hwif_req_is_wr),
    .regblk_addr         (hwif_addr),
    .regblk_wr_data      (hwif_wr_data),
    .regblk_wr_biten     (hwif_wr_biten),
    .regblk_req_stall_wr (hwif_req_stall_wr),
    .regblk_req_stall_rd (hwif_req_stall_rd),
    .regblk_rd_ack       (hwif_rd_ack),
    .regblk_rd_err       (hwif_rd_err),
    .regblk_rd_data      (hwif_rd_data),
    .regblk_wr_ack       (hwif_wr_ack),
    .regblk_wr_err       (hwif_wr_err)
);
```

Typical deployments: CSR blocks behind the APB shim (see 3.4), or any fabric whose endpoint speaks the cmd/rsp packet convention.

### 16.6.2 3.5.8 Integration Example

The flow a real deployment uses (CSRs behind the APB shim):

### 16.6.3 Figure 3.9: CSRs Behind the APB Shim



### *CSRs Behind the APB Shim*

The shim's own cmd/rsp stream is internal — its external pins are APB. The block that turns a bus back into the cmd/rsp surface this adapter consumes is `apb4_to_peakrdl` (which wraps `apb4_slave_cdc` + `peakrdl_to_cmdrsp`); wire the adapter directly only when your fabric already speaks cmd/rsp packets.

The wiring is exactly the instantiation in 3.5.7; the earlier example here showed the adapter hanging off a register block's APB port through `reg_*` signals that do not exist — the reversed-direction reading this page used to make.

## 16.7 Related Modules

### 16.7.1 3.5.9 APB4 Front End (apb4_to_peakrdl)

`peakrdl_to_cmdrsp` above takes a cmd/rsp stream. `apb4_to_peakrdl` is the module that produces one from an APB4 slave port, and it crosses clock domains while doing it – APB on `pclk`, the register block on `aclk`. It is a two-stage composition and adds no register logic of its own:

| Stage | Module | Domain |
|---|---|---|
| 1 | `apb4_slave_cdc` | `pclk` in, `aclk` out |
| 2 | `peakrdl_to_cmdrsp` | `aclk` |

*apb4_to_peakrdl Parameters*

| Parameter | Type | Default | Description |
|---|---|---|---|
| `ADDR_WIDTH` | int | 12 | APB and cpuif byte address width |
| `DATA_WIDTH` | int | 32 | Must match the PeakRDL generation |
| `PROT_WIDTH` | int | 3 | `PPROT` width |
| `STRB_WIDTH` | int | DATA_WIDTH/8 | Derived; do not override |
| `CDC_DEPTH` | int | 2 | Command/ response CDC FIFO depth (>= 2) |
| `USE_JOHNSON` | int | 0 | CDC pointer encoding: 0 = Gray (power-of-2 depth only), 1 = Johnson (any depth) |
| `USE_2_PHASE_CDC` | bit | 1 | **Deprecated and ignored.** Accepted for compatibility; `apb4_slave_cdc` |

| Parameter | Type | Default | Description |
|---|---|---|---|
| | | | does not reference it, so setting it changes nothing |

*apb4_to_peakrdl Ports*

| Port | Width | Direction | Description |
|---|---|---|---|
| `aclk` | 1 | Input | Register-block clock; drives the cpuif master |
| `aresetn` | 1 | Input | Active-low reset, `aclk` domain |
| `pclk` | 1 | Input | APB clock |
| `presetn` | 1 | Input | Active-low reset, `pclk` domain |
| `s_apb_PSEL` | 1 | Input | APB select |
| `s_apb_PENABLE` | 1 | Input | APB enable |
| `s_apb_PREADY` | 1 | Output | APB ready |
| `s_apb_PADDR` | ADDR_WIDTH | Input | APB address |
| `s_apb_PWRITE` | 1 | Input | Write/read direction |
| `s_apb_PWDATA` | DATA_WIDTH | Input | Write data |
| `s_apb_PSTRB` | STRB_WIDTH | Input | Write strobes |
| `s_apb_PPROT` | PROT_WIDTH | Input | Protection attributes |
| `s_apb_PRDATA` | DATA_WIDTH | Output | Read data |
| `s_apb_PSLVERR` | 1 | Output | Slave error |
| `cpuif_req` | 1 | Output | Passthrough request |
| `cpuif_req_is_wr` | 1 | Output | Request is a write |
| `cpuif_addr` | ADDR_WIDTH | Output | Request address |
| `cpuif_wr_data` | DATA_WIDTH | Output | Write data |

| Port | Width | Direction | Description |
|---|---|---|---|
| cpuif_wr_biten | DATA_WIDTH | Output | Per-bit write enables, expanded from PSTRB |
| cpuif_req_stall_wr | 1 | Input | Register block stalls a write |
| cpuif_req_stall_rd | 1 | Input | Register block stalls a read |
| cpuif_rd_ack | 1 | Input | Read acknowledge |
| cpuif_rd_err | 1 | Input | Read error, returned as PSLVERR |
| cpuif_rd_data | DATA_WIDTH | Input | Read data |
| cpuif_wr_ack | 1 | Input | Write acknowledge |
| cpuif_wr_err | 1 | Input | Write error, returned as PSLVERR |

# 17 3.7 UART to AXI4-Lite Bridge

The `uart_to_axil4` block turns a two-wire serial link into an AXI4-Lite master, so a host can read and write a design's register space over a USB serial adapter with no JTAG, no debug core and no vendor tooling. It is the access path the characterization harnesses use for board bring-up.

Three modules ship in `rtl/uart_to_axil4/`: the byte-level `uart_rx` and `uart_tx`, and `uart_axil_bridge`, which parses commands and drives the bus.

## 17.1 3.7.1 Module Organization

`uart_axil_bridge` instantiates four blocks:

| Instance | Module | Role |
|---|---|---|
| `u_uart_rx` | `uart_rx` | Serial to byte, with a ready/valid handshake |
| `u_uart_tx` | `uart_tx` | Byte to serial |
| `u_axil_wr_timing` | `axil4_master_wr` | Write-channel skid isolation |
| `u_axil_rd_timing` | `axil4_master_rd` | Read-channel skid isolation |

The two `axil4_master_*` instances are the same AMBA library blocks documented in the RTL AMBA AXI4-Lite book; the bridge does not hand-roll a bus master. The `SKID_DEPTH_*` parameters below are passed straight through to them.

`uart_rx` and `uart_tx` use the `i_clk`/`i_rst_n` common-block naming while the bridge presents `aclk`/`aresetn`. The bridge maps them directly – there is one clock domain and no CDC anywhere in this block.

## 17.2 3.7.2 Command Protocol

Commands are ASCII, terminated by newline:

| Command | Form | Response |
|---|---|---|
| Write | `W <addr_hex> <data_hex>\n` | `OK\n` |
| Read | `R <addr_hex>\n` | `0x<data_hex>\n` |

The command letter is **case-insensitive**: `w`/`W` and `r`/`R` are both accepted.

Hex digits are consumed as nibbles and shifted in, with `' '`, `'\n'` and `'\r'` skipped as separators. The field widths follow the bus parameters – `ceil(AXIL_ADDR_WIDTH/4)` address digits and `ceil(AXIL_DATA_WIDTH/4)` data digits, so 8 of each at the 32-bit defaults.

Because digits shift rather than fill a fixed field, **the parser does not validate length**. Supplying too few digits left-aligns nothing and yields a small address; supplying too many silently discards the oldest nibbles. The host is responsible for sending exactly the digit count the bus width implies.

## 17.3 3.7.3 Error Responses

A transfer that returns a non-OKAY response answers ERR\n:

| Outcome | Response |
|---|---|
| Write accepted (bresp == OKAY) | OK\n |
| Write rejected (SLVERR/DECERR) | ERR\n |
| Read accepted (rresp == OKAY) | 0x<data>\n |
| Read rejected (SLVERR/DECERR) | ERR\n |

A failed read returns no data at all rather than 0x followed by whatever sat on rdata, because a value that looks well-formed is worse than no value.

Until 2026-09-03 the bridge did not do this. It brought bresp and rresp back from the two axil4_master_* instances and never examined either, so a write that returned SLVERR still answered OK and a failed read returned stale bus data formatted as a normal result. A host had no way to tell a failure from a success.

The host side needed no change: uart_axi_bridge.py already returns response.strip() == b'OK' from write(), and read() already requires a 0x prefix. Both were written expecting error signalling the RTL was not producing.

## 17.4 3.7.4 Parameters

uart_axil_bridge:

*uart_axil_bridge Parameters*

| Parameter | Default | Description |
|---|---|---|
| AXIL_ADDR_WIDTH | 32 | AXI4-Lite address width; sets the address digit count |
| AXIL_DATA_WIDTH | 32 | AXI4-Lite data width; sets the data digit count and m_axil_wstrb width |
| CLKS_PER_BIT | 868 | Baud divisor, aclk cycles per bit. The default is 100 MHz / 115200 baud |
| SKID_DEPTH_AR | 2 | Passed to axil4_master_rd |
| SKID_DEPTH_R | 4 | Passed to axil4_master_rd |
| SKID_DEPTH_AW | 2 | Passed to axil4_master_wr |
| SKID_DEPTH_W | 4 | Passed to axil4_master_wr |
| SKID_DEPTH_B | 2 | Passed to axil4_master_wr |

uart_rx and uart_tx each take one parameter, CLKS_PER_BIT (default 868), with the same meaning.

## 17.5 3.7.5 Ports

### 17.5.1 uart_axil_bridge

*uart_axil_bridge Ports*

| Port | Width | Direction | Description |
|---|---|---|---|
| `aclk` | 1 | Input | Bridge clock; also clocks both UART blocks |
| `aresetn` | 1 | Input | Active-low reset |
| `i_uart_rx` | 1 | Input | Serial receive line |
| `o_uart_tx` | 1 | Output | Serial transmit line |
| `m_axil_awaddr` | AXIL_ADDR_WIDTH | Output | Write address |
| `m_axil_awprot` | 3 | Output | Write protection attributes |
| `m_axil_awvalid` | 1 | Output | Write address valid |
| `m_axil_awready` | 1 | Input | Write address ready |
| `m_axil_wdata` | AXIL_DATA_WIDTH | Output | Write data |
| `m_axil_wstrb` | AXIL_DATA_WIDTH/8 | Output | Write byte strobes |
| `m_axil_wvalid` | 1 | Output | Write data valid |
| `m_axil_wready` | 1 | Input | Write data ready |
| `m_axil_bresp` | 2 | Input | Write response; a non-OKAY value yields `ERR\n` (3.7.3) |
| `m_axil_bvalid` | 1 | Input | Write response valid |
| `m_axil_bready` | 1 | Output | Write response ready |
| `m_axil_araddr` | AXIL_ADDR_WIDTH | Output | Read address |

| Port | Width | Direction | Description |
|---|---|---|---|
| m_axil_arprot | 3 | Output | Read protection attributes |
| m_axil_arvalid | 1 | Output | Read address valid |
| m_axil_arready | 1 | Input | Read address ready |
| m_axil_rdata | AXIL_DATA_WIDTH | Input | Read data, returned in the `0x...` response |
| m_axil_rresp | 2 | Input | Read response; a non-OKAY value yields `ERR\n` (3.7.3) |
| m_axil_rvalid | 1 | Input | Read data valid |
| m_axil_rready | 1 | Output | Read data ready |

### 17.5.2 uart_rx

*uart_rx Ports*

| Port | Width | Direction | Description |
|---|---|---|---|
| i_clk | 1 | Input | Clock |
| i_rst_n | 1 | Input | Active-low reset |
| i_rx | 1 | Input | Serial receive line |
| o_rx_data | 8 | Output | Received byte |
| o_rx_valid | 1 | Output | Byte valid |
| i_rx_ready | 1 | Input | Consumer ready |

### 17.5.3 uart_tx

*uart_tx Ports*

| Port | Width | Direction | Description |
|---|---|---|---|
| i_clk | 1 | Input | Clock |
| i_rst_n | 1 | Input | Active-low reset |

| Port | Width | Direction | Description |
|---|---|---|---|
| o_tx | 1 | Output | Serial transmit line |
| i_tx_data | 8 | Input | Byte to send |
| i_tx_valid | 1 | Input | Byte valid |
| o_tx_ready | 1 | Output | Transmitter ready for the next byte |

# 18 3.8 Width-plus-Protocol Chains

A chain is a FUB-level module that wires one width converter directly to one protocol converter. Neither chain adds logic of its own: they exist because that pairing is what a real bridge instantiates, and because the bugs worth catching live at the seam between the two, not inside either one.

Both are downsizing chains – a wide AXI4 master talking to a narrow peripheral – and both were built to reproduce the back-to-back page-boundary class of defect at FUB level, where it can be driven directly, instead of only at the bridge level where it was first seen.

## 18.1 3.8.1 axi4_dwidth_to_axil4_wr_chain

Write path. The slave port is wide AXI4 (S_AXI_DATA_WIDTH); the master port is AXI4-Lite at M_AXIL_DATA_WIDTH.

| Stage | Module | What it does |
|---|---|---|
| 1 | axi4_dwidth_converter_wr | Downsizes the W beats and rewrites awlen/awsize, so the next stage sees a narrow AXI4 burst of M_AXIL_DATA_WIDTH-wide beats |
| 2 | axi4_to_axil4_wr | Decomposes that narrow burst into single-beat AXI4-Lite writes |

*axi4_dwidth_to_axil4_wr_chain Parameters*

| Parameter | Default | Description |
|---|---|---|
| S_AXI_DATA_WIDTH | 64 | Slave-side (wide) AXI4 data width |
| M_AXIL_DATA_WIDTH | 32 | Master-side AXI4-Lite data width; the width beats are downsized to |
| AXI_ID_WIDTH | 8 | Slave-side ID width |
| AXI_ADDR_WIDTH | 32 | Address width, both sides |
| AXI_USER_WIDTH | 1 | User-signal width |

Ports are the slave-side AXI4 write channels and the master-side AXI4-Lite write channels, unchanged from the two modules it wraps – see AXI4 to AXI4-Lite for the AXI4-Lite side and the width block chapters for the AXI4 side.

## 18.2 3.8.2 axi4_dwidth_to_apb4_chain

Read path only. **The write channels on the slave port are tied off inside the chain** – this is a read chain, not a general bridge.

| Stage | Module | What it does |
|---|---|---|
| 1 | `axi4_dwidth_converter_rd` | Rewrites `arlen`/`arsize` down to `APB_DATA_WIDTH`-wide narrow beats, and upsizes the returning R beats back to `S_AXI_DATA_WIDTH` |
| 2 | `axi4_to_apb4_shim` | Decomposes the narrow burst into single-beat APB reads |

*axi4_dwidth_to_apb4_chain Parameters*

| Parameter | Default | Description |
|---|---|---|
| `S_AXI_DATA_WIDTH` | 64 | Slave-side (wide) AXI4 data width |
| `APB_DATA_WIDTH` | 32 | APB data width on the master side |
| `AXI_ID_WIDTH` | 8 | Slave-side ID width |
| `AXI_ADDR_WIDTH` | 32 | AXI address width |
| `APB_ADDR_WIDTH` | 32 | APB address width |
| `AXI_USER_WIDTH` | 1 | User-signal width |
| `USE_JOHNSON` | 0 | CDC-FIFO pointer encoding forwarded to the shim: 0 = Gray (power-of-2 depth only), 1 = Johnson |

The direction is worth stating plainly, because the two readings of “downsize” point opposite ways. The wide master issues a wide-beat AR burst; the read converter rewrites it into narrow `APB_DATA_WIDTH` beats for the shim; the R data coming back is reassembled into wide beats for the master.

# 19 3.10 AXI4-Lite to Wishbone B4 Converter

The **axil4_to_wb4** converter presents an AXI4-Lite slave and drives a Wishbone B4 master. It is the bridge to drop in when an AXI4-Lite interconnect has to reach a Wishbone peripheral, and it is built from parts the library already has: the AXI4-Lite slave skids, a small conversion core, and `wb4_master` from `rtl/amba/wb4/`.

## 19.1 3.10.1 Module Organization

```
axil4_to_wb4.sv                # Wrapper: skids + core + wb4_master
├── axil4_slave_wr.sv          # AW / W / B skid buffers
(rtl/amba/axil4)
├── axil4_slave_rd.sv          # AR / R skid buffers (rtl/amba/axil4)
├── axil4_to_wb4_core.sv       # Five channels -> one command and one
response queue
└── wb4_master.sv              # Queues -> Wishbone B4 bus
(rtl/amba/wb4)
```

## 19.2 3.10.2 Design Philosophy

**One queue, in order.** AXI4-Lite has separate write and read paths; Wishbone has one bus. The core merges the two paths into the single command queue of `wb4_master` and relies on a B4 rule to route the answers back: terminations return in issue order. A one-bit side queue, written at issue with the direction of each command, tells the core whether the oldest response belongs on B or on R. There is no ID, no reorder buffer, and no state machine.

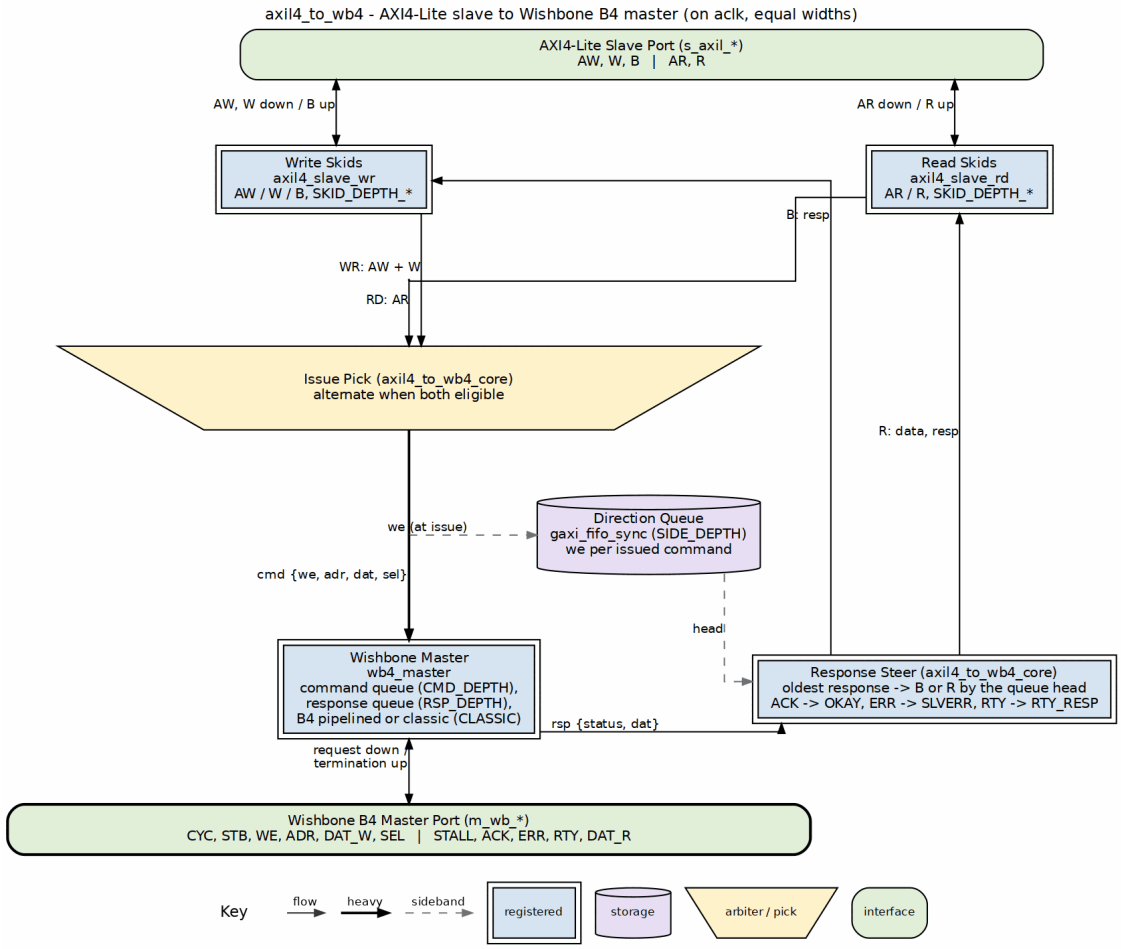
**No starvation.** A write is eligible when both AW and W are present, a read when AR is. When both are eligible in the same clock the side not served last is picked, so a stream of writes cannot hold reads off the bus and vice versa.

**Wishbone status becomes an AXI response, nothing more.** `ACK` maps to `OKAY`, `ERR` to `SLVERR`, and `RTY` to the `RTY_RESP` parameter (`SLVERR` by default). AXI has no retry, so a retrying Wishbone slave is reported to the requester as an error; a retry wrapper on the FUB side of `wb4_master` is the place to absorb `RTY` transparently.

**No width conversion.** The AXI4-Lite and Wishbone address and data widths are the same. Put `axi4_dwidth_*` or the wide-alignment blocks in front when the interconnect width differs.

## 19.3 3.10.3 Block Diagram

### 19.3.1 Figure 3.12: AXI4-Lite to Wishbone B4 Converter




*AXI4-Lite to Wishbone B4 Converter*

## 19.4 3.10.4 Interface Specification

### 19.4.1 Parameters

*Table 3.26: AXI4-Lite to Wishbone Parameters*

| Parameter | Default | Description |
|---|---|---|
| ADDR_WIDTH | 32 | Address width, both sides |
| DATA_WIDTH | 32 | Data width, both sides |
| SKID_DEPTH_AW/W/B/AR/R | 2 | AXI4-Lite channel skid depths (2..8) |
| CMD_DEPTH | 4 | `wb4_master` command queue depth |
| RSP_DEPTH | 4 | `wb4_master` response queue depth; bounds the transfers on the bus |
| SIDE_DEPTH | 8 | Core direction queue; at least CMD_DEPTH + RSP_DEPTH so the master sets the pipeline depth |
| CLASSIC | 0 | 0 = B4 pipelined, 1 = B4 standard (classic) mode; match the slave |
| RTY_RESP | 2'b10 | AXI response returned for a Wishbone RTY |

The core alone (`axil4_to_wb4_core`) takes `ADDR_WIDTH`, `DATA_WIDTH`, `SIDE_DEPTH` and `RTY_RESP`.

### 19.4.2 Ports

*Table 3.27: AXI4-Lite to Wishbone Ports*

| Port group | Direction | Description |
|---|---|---|
| `s_axil_aw*`, `s_axil_w*`, `s_axil_b*` | slave | AXI4-Lite write channels (AWPROT accepted, not carried) |
| `s_axil_ar*`, `s_axil_r*` | slave | AXI4-Lite read channels |

| Port group | Direction | Description |
|---|---|---|
| `m_wb_CYC/STB/WE/ADR/DAT_W/SEL` | out | Wishbone request |
| `m_wb_STALL/ACK/ERR/RTY/DAT_R` | in | Wishbone flow control and termination |
| `busy` | out | Anything buffered in the skids or a cycle open on the bus |

The core's ports are the five `fub_*` channels of the two slave blocks on one side and the `cmd_*` / `rsp_*` queues of `wb4_master` on the other.

## 19.5 3.10.5 Command Formation

*Table 3.28: Command Formation*

| AXI4-Lite | Wishbone command |
|---|---|
| AWADDR, WDATA, WSTRB | `we=1`, `adr`, `dat`, `sel` = WSTRB |
| ARADDR | `we=0`, `adr`, `sel` = all ones |
| AWPROT / ARPROT | not carried (Wishbone has no protection bits) |

A write consumes AW and W in the same clock (`fub_awready` and `fub_wready` are the same signal), so a W beat that arrives before its AW simply waits in its skid. A command is only issued when the side queue has room; when the side queue is full the core stalls, which with `SIDE_DEPTH >= CMD_DEPTH + RSP_DEPTH` never happens before the master's own queues are full.

## 19.6 3.10.6 Response Mapping

*Table 3.29: Response Mapping*

| Wishbone status | B / R response |
|---|---|
| ACK | OKAY (2'b00) |
| ERR | SLVERR (2'b10) |
| RTY | `RTY_RESP` (default SLVERR) |

`RDATA` is the Wishbone `DAT_R` of the termination. A response that arrives with nothing recorded in the side queue is a master protocol violation (`wb4_master` reports it in simulation); the core drops it so the queue cannot wedge on it.

## 19.7 3.10.7 Timing

*Table 3.30: AXI4-Lite to Wishbone Timing*

| Path | Clocks |
|---|---|
| AW+W or AR accepted at `s_axil_*` to `STB` on the bus | 2 (skid, master command queue) |
| Termination on the bus to `BVALID` / `RVALID` | 2 (master response queue, skid) |
| Sustained rate, pipelined slave | one transfer per clock per direction pair, up to `RSP_DEPTH` in flight |
| Sustained rate, `CLASSIC=1` | one transfer at a time (B4 standard mode) |

## 19.8 3.10.8 Formal

`formal/converters/axil4_to_wb4_core/` (sv2v flatten, then SymbiYosys) proves on the core: a command is issued only with a request present and carries the payload of the channel it took; B and R never fire together and each consumes exactly one response; the response steered to B or R is the oldest open transfer's; the open count never exceeds `SIDE_DEPTH`; OKAY / SLVERR / `RTY_RESP` follow the status; and a write is never issued straight after a write while a read is pending. Covers reach both directions, the alternation, an SLVERR on B, a retry on R, and a full side queue.

## 19.9 3.10.9 Testing

`dv/tests/test_axil4_to_wb4.py` drives `s_axil_*` with the AXI4-Lite master BFMs and answers `m_wb_*` with the framework's Wishbone slave BFM over a memory model (ERR and RTY windows by address) while the Wishbone monitor checks the B4 rules on the wire. The scoreboard mirrors every OKAY write byte-by-byte (strobed writes included) and checks every read against the mirror, and every response code against the window its address is in. Phases: sequential; several transfers in flight against a stalling, a slow-terminating and a mixed slave; drain with one monitor transfer per transaction and no violations. The classic build runs the same phases against a classic-mode slave and returns DECERR for a retry to prove the `RTY_RESP` path. Four RTL mutations (direction bit inverted, RTY mapped to OKAY, write strobes dropped, read address taken from AW) all fail the test.

## 19.10 3.10.10 Usage Example

```
axil4_to_wb4 #(
    .ADDR_WIDTH (32),
    .DATA_WIDTH (32),
    .CMD_DEPTH  (4),
    .RSP_DEPTH  (4),
    .SIDE_DEPTH (8),
    .CLASSIC    (0)
) u_axil_to_wb (
    .aclk           (clk),
    .aresetn        (rst_n),
    .s_axil_awaddr  (awaddr),  .s_axil_awprot
(awprot), .s_axil_awvalid (awvalid), .s_axil_awready (awready),
    .s_axil_wdata   (wdata),   .s_axil_wstrb  (wstrb),  .s_axil_wvalid
(wvalid),  .s_axil_wready  (wready),
    .s_axil_bresp   (bresp),   .s_axil_bvalid (bvalid), .s_axil_bready
(bready),
    .s_axil_araddr  (araddr),  .s_axil_arprot
(arprot), .s_axil_arvalid (arvalid), .s_axil_arready (arready),
    .s_axil_rdata   (rdata),   .s_axil_rresp  (rresp),  .s_axil_rvalid
(rvalid),  .s_axil_rready  (rready),
    .m_wb_CYC (wb_cyc), .m_wb_STB (wb_stb), .m_wb_WE
(wb_we), .m_wb_ADR (wb_adr),
    .m_wb_DAT_W (wb_dat_w), .m_wb_SEL (wb_sel),
    .m_wb_STALL (wb_stall), .m_wb_ACK (wb_ack), .m_wb_ERR
(wb_err), .m_wb_RTY (wb_rty),
    .m_wb_DAT_R (wb_dat_r),
    .busy ()
);
```

# 20 3.11 Wishbone B4 to AXI4-Lite Converter

The **wb4_to_axil4** converter presents a Wishbone B4 slave and drives an AXI4-Lite master. It is the mirror of 3.10 and the harder direction, for one reason given below.

## 20.1 3.11.1 Module Organization

```
wb4_to_axil4.sv              # Wrapper: slave + core + the two masters
├── wb4_slave.sv             # Wishbone bus -> cmd/rsp queues
(rtl/amba/wb4)
├── wb4_to_axil4_core.sv     # Queues -> five AXI4-Lite channels,
responses merged
├── axil4_master_wr.sv       # AW / W / B channels (rtl/amba/axil4)
└── axil4_master_rd.sv       # AR / R channels (rtl/amba/axil4)
```

## 20.2 3.11.2 Why This Direction Is Harder

Going AXI4-Lite to Wishbone, the two request paths merge into one bus and the answers come back in issue order for free, because that is what Wishbone does.

Coming back the other way the ordering runs against you. **Wishbone B4 terminates in issue order. AXI4-Lite's B and R channels are independent and a slave may answer them in any order.** A read issued behind a slow write will routinely finish first. If the converter passed each response straight through, the Wishbone master would see its transfers terminate out of order, which the protocol forbids and its own bookkeeping would mispair.

So the core records the direction of every command it issues in an in-order side queue and releases only the response at the head. A response that completed early waits in its channel's skid until its predecessors have retired.

**Memory order.** Terminations in order are not enough: B4 also completes the accesses themselves in issue order, so a read behind a write to the same word must return the new data, and a write behind a read must not reach memory first. AXI4-Lite gives no order between its read and write channels, so a command waits until every command of the other direction has retired. Reads pipeline behind reads and writes behind writes, up to `OUTSTANDING`; a direction change costs one round trip. Until 2026-10-18 the core held only the responses in order, and a read issued right behind a slow write to the same word returned the old data (found by the converters regression at SEED=21655, 64-bit, OUTSTANDING 2 and 4).

**Ordering cost.** A slow write at the head holds back a read that finished behind it. `OUTSTANDING = 1` avoids the question entirely and is the default; raise it only when the AXI slave answers roughly in order, or head-of-line waiting eats the benefit.

## 20.3 3.11.3 Interface Specification

### 20.3.1 Parameters

*Table 3.31: Wishbone to AXI4-Lite Parameters*

| Parameter | Default | Description |
|---|---|---|
| ADDR_WIDTH | 32 | Address width, both sides |
| DATA_WIDTH | 32 | Data width, both sides |
| CMD_DEPTH / RSP_DEPTH | 2 | The wb4_slave's queue depths |
| MAX_OUTSTANDING | 16 | The wb4_slave's bus-side bound |
| CLASSIC | 0 | 0 = B4 pipelined, 1 = B4 standard; match the Wishbone master |
| OUTSTANDING | 1 | Commands in flight through the core. 1 = strictly serialised |
| SKID_DEPTH_AW/W/B/AR/R | 2 | The AXI4-Lite masters' channel skids |
| AXIL_PROT | 3'b000 | `AWPROT`/`ARPROT` driven on every command |

### 20.3.2 Ports

*Table 3.32: Wishbone to AXI4-Lite Ports*

| Port group | Direction | Description |
|---|---|---|
| `s_wb_CYC/STB/WE/ADR/DAT_W/SEL/CTI/BTE` | in | Wishbone request |
| `s_wb_STALL/ACK/ERR/RTY/DAT_R` | out | Flow control and termination |
| `m_axil_aw*`, `m_axil_w*`, `m_axil_b*` | master | AXI4-Lite write channels |
| `m_axil_ar*`, `m_axil_r*` | master | AXI4-Lite read channels |
| `busy` | out | Anything buffered, or a cycle open on the |

| Port group | Direction | Description |
| --- | --- | --- |
| | | Wishbone side |

## 20.4 3.11.4 Command Formation

*Table 3.33: Command Formation*

| Wishbone | AXI4-Lite |
|---|---|
| WE=1, ADR, DAT_W, SEL | AWADDR + WDATA, WSTRB = SEL |
| WE=0, ADR | ARADDR |
| CTI / BTE | **dropped** (see below) |
| - | AWPROT/ARPROT = AXIL_PROT |

A write only launches when **both** AWREADY and WREADY are high in the same clock. Letting AW go while W stalled would leave a half-issued transfer that the outstanding count could not describe honestly.

**The burst hints stop here.** CTI and BTE describe a registered-feedback burst, and AXI4-Lite is single-beat with no field to carry them. A burst-aware target behind an AXI4-Lite bus is a contradiction, so the converter drops them rather than inventing a mapping.

## 20.5 3.11.5 Response Mapping

*Table 3.34: Response Mapping*

| AXI4-Lite BRESP / RRESP | Wishbone termination |
|---|---|
| OKAY | ACK |
| SLVERR | ERR |
| DECERR | ERR |
| - | RTY is never produced |

B4 has one error termination and no way to say which kind, so SLVERR and DECERR both become ERR. RTY never appears because an AXI slave has no way to ask for a retry; a Wishbone master that wants retries needs `wb4_retry` on the other side of the bus, not here.

## 20.6 3.11.6 Formal

`formal/converters/wb4_to_axil4_core/` proves: a write takes `AW` and `W` in the same clock and never one alone; a command becomes a write or a read, never both; the response consumed is the head's channel **and is backed by a real response on that channel**; the open count never exceeds `OUTSTANDING`; OKAY becomes `ACK`, any error becomes `ERR`, and `RTY` is never produced.

The backing property earned its place. An earlier property set proved every ordering rule above and still passed a mutation that asserted `rsp_valid` whenever *either* channel had a response, returning a status read off a channel that had nothing. Simulation caught it as a hang; formal did not, until the property was added.

## 20.7 3.11.7 Testing

`dv/tests/test_wb4_to_axil4.py` drives `s_wb_*` with the framework's Wishbone master BFM and answers `m_axil_*` with the AXI4-Lite slave BFMs over one shared memory model, so a write really lands where a later read finds it. Addresses beyond the memory model answer SLVERR and must arrive as `ERR`.

The phases are sequential traffic, pipelined traffic under gappy and sparse request pacing, a directed **hazard probe** (write, read, read, write, read of one word, back to back: it fails every pipelined `OUTSTANDING > 1` configuration on the core without the memory-order rule), and a directed **ordering probe**: write-then-read pairs where the testbench answers the write channel slowly and the read channel quickly, so the AXI side completes the read first every time. The scoreboard pairs terminations with issue order, so a response that overtook its predecessor shows up as a status or data mismatch rather than as silence.

Three RTL mutations fail the test: completing from whichever channel is ready, corrupting read data, and launching `AW` without `W`.

## 20.8 3.11.8 Usage Example

```
wb4_to_axil4 #(
    .ADDR_WIDTH  (32),
    .DATA_WIDTH  (32),
    .OUTSTANDING (1),      // raise only if the AXI slave answers in
order
    .CLASSIC     (0)
) u_wb_to_axil (
    .aclk (clk), .aresetn (rst_n),
    .s_wb_CYC (wb_cyc), .s_wb_STB (wb_stb), .s_wb_WE
(wb_we), .s_wb_ADR (wb_adr),
    .s_wb_DAT_W (wb_dat_w), .s_wb_SEL (wb_sel), .s_wb_CTI
(wb_cti), .s_wb_BTE (wb_bte),
    .s_wb_STALL (wb_stall), .s_wb_ACK (wb_ack), .s_wb_ERR
(wb_err), .s_wb_RTY (wb_rty),
    .s_wb_DAT_R (wb_dat_r),
    .m_axil_awaddr (awaddr), .m_axil_awprot (awprot), .m_axil_awvalid
(awvalid), .m_axil_awready (awready),
    .m_axil_wdata (wdata), .m_axil_wstrb (wstrb), .m_axil_wvalid
(wvalid), .m_axil_wready (wready),
    .m_axil_bresp (bresp), .m_axil_bvalid (bvalid), .m_axil_bready
(bready),
    .m_axil_araddr (araddr), .m_axil_arprot (arprot), .m_axil_arvalid
(arvalid), .m_axil_arready (arready),
    .m_axil_rdata (rdata), .m_axil_rresp (rresp), .m_axil_rvalid
(rvalid), .m_axil_rready (rready),
    .busy ()
);
```

## 20.9 Navigation

**Next:** APB to AXI4 (Requester)

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 21 APB to AXI4 (Requester)

**Modules:** `apb4_to_axi4.sv`, `apb5_to_axi4.sv`, `apb_cmdrsp_to_axi4.sv` **Filelists:** `rtl/filelists/apb4_to_axi4.f`, `rtl/filelists/apb5_to_axi4.f` (each `-f`'s its `rtl/amba` APB completer and the shared requester core)

## 21.1 Overview

The mirror of AXI4 to APB. There an AXI4 requester is turned into an APB requester; here an APB requester – a small CPU, a debug port, a register sequencer – becomes an AXI4 requester. The bridge generator puts one of these in front of a master port declared `protocol = "apb"` or `"apb5"` (bridge TASK-004, was BRIDGE-014).

Two proven pieces and one small new one:

*Table 3.35: APB-to-AXI4 requester pieces*

| Piece | Job |
|---|---|
| `apb4_slave` / `apb5_slave` (`rtl/amba`) | The APB completer surface: PSEL/PENABLE/PREADY handshake, skid-buffered cmd/rsp pair, the orphan-response guard |
| `apb_cmdrsp_to_axi4` | One command -> one single-beat AXI4 transaction -> one response |
| `apb4_to_axi4` / `apb5_to_axi4` | The wiring, and for APB5 the USER mapping |

APB is strictly one-outstanding, so there is never more than one AXI4 transaction in flight and the response needs no ID matching: every request carries the constant `DEFAULT_ID`. `AW` and `W` are presented together and each is held until its own handshake, so the requester is legal against a completer that takes `W` before `AW` or the other way round.

```
pwrite=1 : AW + W (awlen=0, wlast=1)  ->  B  ->  rsp  (PSLVERR =
bresp[1])
pwrite=0 : AR      (arlen=0)          ->  R  ->  rsp  (PRDATA, PSLVERR
= rresp[1])
```

Both `SLVERR` and `DECERR` fold to `PSLVERR`: APB has one error bit, so a requester cannot tell an unmapped address from a failing completer. `R` is drained to `RLAST` even though every request asks for one beat, so a completer that answers with more cannot wedge the port; only the first beat's data is returned.

## 21.2 Fields the requester invents

*Table 3.36: AXI4 request fields from an APB transfer*

| Field | Value | Why |
|---|---|---|
| AxID | DEFAULT_ID | APB has no ID; one value is enough with one transaction outstanding |
| AxLEN | 0 | one beat per transfer |
| AxSIZE | $clog2(DATA_WIDTH/8) | the full data width; PSTRB becomes WSTRB unchanged |
| AxBURST | INCR | |
| AxCACHE | DEFAULT_CACHE (device non-bufferable) | the natural attribute for a peripheral-bus requester |
| AxPROT | PPROT | forwarded |
| AxLOCK, AxQOS, AxREGION | 0 | |
| WLAST | 1 | |

## 21.3 APB5 sideband

apb5_to_axi4 maps the APB5 USER signals onto the AXI USER fields with a plain size cast – the low bits travel, anything wider is zero-filled or dropped:

*Table 3.37: APB5 USER mapping*

| APB5 | AXI4 |
|---|---|
| PAUSER | awuser / aruser |
| PWUSER | wuser |
| buser | PBUSER |
| ruser | PRUSER |

PWAKEUP is requester-driven (IHI 0024E section 3.7.2, Table 3-7) and is an **input** here. It is passed to apb5_slave's s_apb_PWAKEUP input, which is also an input since issue #102; there is nothing on the AXI side for it to become, so it goes no further.

Interface parity is not part of this surface. The *CHK check signals are conditional on the Check_Type property (IHI 0024E Table B-2), and this converter is a Check_Type = False completer, so none of them appear on its ports. Inside, apb5_slave is built with ENABLE_PARITY=0, which leaves its check inputs unchecked and drives its check outputs LOW. Every check input – PADDRCHK, PCTRLCHK, PSELCHK, PENABLECHK, PWDATACHK, PSTRBCHK, PAUSERCHK, PWUSERCHK and PWAKEUPCHK – is tied LOW, the same constant apb5_master drives on its check outputs with parity off, and the check outputs (PRDATACHK, PREADYCHK, PSLVERRCHK, PRUSERCHK, PBUSERCHK) are left open.

Inside the bridge the AXI USER width is 1, so one bit of PAUSER reaches the fabric – the same width the AXI5-Lite slave side exposes, which is how an APB5 requester's user bit can be observed at an AXI5-Lite completer. The return direction is not symmetrical: the bridge's master adapters tie response-side USER to zero (see the bridge PRD), so PBUSER/PRUSER read 0 on a bridge port even though the converter itself carries them.

## 21.4 Parameters

*Table 3.38: APB-to-AXI4 parameters*

| Parameter | Default | Description |
|---|---|---|
| APB_ADDR_WIDTH | 32 | PADDR and AxADDR width (one address space; no windowing) |
| APB_DATA_WIDTH | 32 | PWDATA/PRDATA and the AXI data width – equal by construction |
| AXI_ID_WIDTH | 1 | width of the constant ID |
| AXI_USER_WIDTH | 1 | AXI USER width (APB4 wrapper drives 0) |
| DEFAULT_ID | '0 | the ID on every request |
| DEPTH | 2 | cmd/rsp skid depth inside the APB completer |
| APB_{A,W,R,B}USER_WIDTH | 1 | APB5 only; independent of AXI_USER_WIDTH |

DEFAULT_CACHE on the core is not exposed by the wrappers; a consumer that needs a different cache attribute instantiates apb_cmdrsp_to_axi4 directly.

## 21.5 Verification

`dv/tests/test_apb4_to_axi4.py` and `dv/tests/test_apb5_to_axi4.py` share `dv/tbclasses/apb_to_axi4_tb.py`: an APB master BFM drives the completer surface, memory-backed AXI4 slave BFMs answer the requester surface, and a monitor samples every AW/W/AR handshake. Checked: the data round trip (the completer's memory is read directly as well as through a second transfer, so a converter echoing its own write data is caught), the single-beat shape of every request, `PPROT->AxPROT`, `PSTRB->WSTRB` against a byte-accurate shadow, out-of-range `SLVERR` folding to `PSLVERR` with nothing written and the port working afterwards, the APB5 USER mapping in both directions, and a slow completer holding `AW`/`W`/`AR` to their handshakes. Twelve cells at FULL (data widths 16/32/64, AXI USER 1/2/4, APB USER 2/4).

## 21.6 Related Modules

The bridge generator's master adapter instantiates `apb4_to_axi4` / `apb5_to_axi4` for `protocol = "apb"` / `"apb5"` master ports and feeds the result to the same `axi4_slave_{wr,rd}` timing wrapper an AXI4 master port gets, so decode, width adaptation (including the wide-slave aligner) and the response mux are untouched. Sign-off: `projects/components/fabric-gen-ip/bridge/dv/tests/test_bridge_2x3_apb_req_paths.py`.

## 21.7 Navigation

**Previous:** Wishbone B4 to AXI4-Lite **Next:** AXI4 to Wishbone B4

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 22 AXI4 to Wishbone B4

**Module:** `axi4_to_wb4.sv` **Filelist:** `rtl/filelists/axi4_to_wb4.f` (`-f`'s the two AXI4-Lite decomposers and `axil4_to_wb4`)

## 22.1 Overview

Full AXI4 completer in, Wishbone B4 requester out – the Wishbone counterpart of `axi4_to_apb4_shim`, built entirely from blocks this component already has:

*Table 3.39: axi4_to_wb4 stages*

| Stage | Block | Job |
|---|---|---|
| 1 | `axi4_to_axil4_wr`, `axi4_to_axil4_rd` | burst decomposition: one AXI4-Lite beat per AXI4 beat, response folding, ID/LAST bookkeeping |
| 2 | `axil4_to_wb4` | five AXI4-Lite channels to one Wishbone command stream and back, in-order termination |

Every AXI4 beat becomes one Wishbone transfer: `WE` from the channel it came on, `SEL` from `WSTRB`, `ADR` the beat's address. Termination maps `ACK` to `OKAY`, `ERR` to `SLVERR`, `RTY` to `RTY_RESP` (`SLVERR` by default; AXI has no retry, so a completer that wants retries handled needs a `wb4_retry` between it and this block). Same address and data width on both sides; the bridge's width converters sit in front when they differ.

The Wishbone side is B4 pipelined unless `CLASSIC = 1` (B4 standard mode); match the completer – the two do not mix (see the `rtl/amba/wb4` README). `CTI`/`BTE` are driven CLASSIC/LINEAR: the decomposer issues one transfer at a time and burst hints are advisory in B4.

## 22.2 Parameters

*Table 3.40: axi4_to_wb4 parameters*

| Parameter | Default | Description |
|---|---|---|
| AXI_ID_WIDTH, AXI_ADDR_WIDTH, AXI_DATA_WIDTH, AXI_USER_WIDTH | 8, 32, 32, 1 | the AXI4 face; data width is also the Wishbone width |
| SKID_DEPTH_AW/W/B/AR/R | 2 | axil4_to_wb4's channel skids |
| CMD_DEPTH, RSP_DEPTH | 4, 4 | wb4_master queues; RSP_DEPTH bounds transfers in flight on the bus |
| SIDE_DEPTH | 8 | direction queue in axil4_to_wb4_core (>= CMD + RSP) |
| CLASSIC | 0 | 0 = B4 pipelined, 1 = B4 standard |
| RTY_RESP | 2'b10 | AXI response for a Wishbone RTY |

## 22.3 Verification

`dv/tests/test_axi4_to_wb4.py` with `dv/tbclasses/axi4_to_wb4_tb.py`: AXI4 master BFMs drive bursts of 1..16 beats into a `WB4Slave` whose `status_hook` turns two address windows into ERR and RTY. Checked: an N-beat burst is exactly N Wishbone transfers in address order, each landing in the completer's memory, and the read burst returns it beat by beat; `WSTRB` becomes `SEL` against a byte shadow; ERR folds to `SLVERR` and RTY to `RTY_RESP` on both B and R with the port working after; fixed, stalling and slow completer profiles; pipelined and classic builds.

## 22.4 Related Modules

The bridge generator's slave adapter instantiates this for a slave port declared `protocol = "wb4"` (bridge TASK-008, was BRIDGE-019), through `Axi4ToWb4Shim`, with the same monitor-wrapper sandwich and response intercepts the APB shim gets. Sign-off: `projects/components/fabric-gen-ip/bridge/dv/tests/test_bridge_2x2_wb4_paths.py`.

## 22.5 Navigation

**Previous:** APB to AXI4 (Requester) **Next:** Wishbone B4 to AXI4

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 23 Wishbone B4 to AXI4

**Module:** `wb4_to_axi4.sv` **Filelist:** `rtl/filelists/wb4_to_axi4.f` (`-f`'s `wb4_to_axil4`'s closure)

## 23.1 Overview

Wishbone B4 completer in, full AXI4 requester out – the Wishbone counterpart of `apb4_to_axi4`. `wb4_to_axil4` does the work (`wb4_slave`, the in-order response merge, the AXI4-Lite masters); this wrapper promotes its AXI4-Lite face to AXI4 the way any AXI4-Lite requester is promoted in the fabric:

*Table 3.41: AXI4 fields the wrapper invents*

| Field | Value |
|---|---|
| `AxID` | `DEFAULT_ID` (constant; Wishbone has no ID) |
| `AxLEN`, `AxBURST`, `WLAST` | 0, INCR, 1 – every transfer is one beat |
| `AxSIZE` | the full data width; `SEL` rides `WSTRB` unchanged |
| `AxPROT` | `AXIL_PROT` (Wishbone carries no protection bits) |
| `AxLOCK`, `AxCACHE`, `AxQOS`, `AxREGION`, USER | 0 (`AxCACHE` = `DEFAULT_CACHE`) |
| `BID`/`RID`, `RLAST`, response USER | dropped: one ID in use, one beat per read |

`ERR` to the requester when the AXI response is `SLVERR` or `DECERR` (Wishbone has one way to say either); `RTY` is never generated. The core's `OUTSTANDING` (default 1) is how many commands it lets into the AXI side at once – B4 terminates in issue order, and above 1 a slow write at the head holds back a read that finished behind it.

## 23.2 Parameters

ADDR_WIDTH, DATA_WIDTH, CMD_DEPTH, RSP_DEPTH, MAX_OUTSTANDING, CLASSIC, OUTSTANDING, SKID_DEPTH_*, AXIL_PROT are wb4_to_axil4's (Table 3.31); added here: AXI_ID_WIDTH (1), AXI_USER_WIDTH (1), DEFAULT_ID ('0), DEFAULT_CACHE (4'b0000).

## 23.3 Verification

`dv/tests/test_wb4_to_axi4.py` with `dv/tbclasses/wb4_to_axi4_tb.py`: a `WB4Master` drives the completer surface, memory-backed AXI4 slave BFMs answer, a monitor samples every AW/W/AR handshake. Checked: the single-beat shape of every request, `SEL` to `WSTRB` against a byte shadow, the data round trip (memory read directly and through a second transfer), the completer's out-of-range `SLVERR` terminating `ERR` with nothing written and the port working after, a slow completer, pipelined and classic builds.

## 23.4 Related Modules

The bridge generator's master adapter instantiates this for a master port declared `protocol = "wb4"` (bridge TASK-008, was BRIDGE-019) and feeds the ordinary `axi4_slave_{wr,rd}` timing wrapper with its `m_axi` face, exactly as it does `apb4_to_axi4` for an APB requester.

## 23.5 Navigation

**Previous:** AXI4 to Wishbone B4

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 24 3.15 AXI4 to AXI3 Converter

The **axi4_to_axi3** converter family bridges an AXI4 master to an AXI3 slave. The first customer is the DE10 leg-bridge `f2h` seam, where the programmable-logic fabric speaks AXI4 and the hard processor subsystem presents an AXI3 interface.

## 24.1 3.15.1 Module Organization

```
axi4_to_axi3.sv          # Full bidirectional wrapper
├── axi4_to_axi3_rd.sv   # Read path: AR/R with burst chopping and
response merge
└── axi4_to_axi3_wr.sv   # Write path: AW/W/B with WID stamping and B
merge
```

Read and write are split into independent modules so a user can instantiate only the path that is needed and so verification can target each path separately.

## 24.2 3.15.2 Protocol Deltas

*Table 3.45: AXI4 to AXI3 protocol differences*

| Aspect | AXI4 (slave side) | AXI3 (master side) | Converter action |
|---|---|---|---|
| `AxLEN` width | 8 bits (1–256 beats) | 4 bits (1–16 beats) | Chop long bursts into ≤16-beat chunks; truncate `AxLEN` per chunk. |
| `AxLOCK` | 1 bit (normal / exclusive) | 2 bits (`2'b00` normal, `2'b01` exclusive, `2'b10` locked) | Map `1'b0` → `2'b00`, `1'b1` → `2'b01`. AXI4 cannot express locked; none is generated. |
| Write data ID | None | `WID` per beat | Stamp every AXI3 W beat with the owning AWID. |
| `QoS` / `REGION` | Present on AW/AR | Absent | Drop silently. |
| Data width | Must match | Must match | No width conversion. |

### 24.2.1 Exclusive-access caveat

The DE10 leg-bridge `f2h` seam is not expected to exercise AXI exclusive accesses. The converter still maps an AXI4 exclusive request (`AxLOCK = 1'b1`) to the AXI3 exclusive encoding (`2'b01`, per ARM IHI 0022), but the upstream system does not implement exclusive-monitor semantics at this boundary. If a legacy AXI3 locked (`2'b10`) transaction arrives on a downstream return path, it is treated as a normal access. (The encoding was swapped — exclusive emitted as `2'b10` — until bridge GitHub #98; the RTL, this table, and the directed lock-map DV now agree with the ARM encoding.)

## 24.3 3.15.3 Read Path (axi4_to_axi3_rd)

### 24.3.1 Operation

The read path accepts an AXI4 AR burst and issues one or more AXI3 AR bursts:

- **Single-beat and short bursts** (`ARLEN ≤ 15`) pass through with only field-width changes.
- **Long bursts** (`ARLEN ≥ 16`) are chopped into sequential AXI3 bursts of up to 16 beats each.
- Returning R beats are streamed back to the AXI4 master with `RLAST` suppressed on every chunk except the last, so the master sees one logical burst.
- `RRESP` from all chunks is merged using worst-rank order (`OKAY < EXOKAY < SLVERR < DECERR`); the merged value is driven on the final beat of the reconstructed burst.

### 24.3.2 Per-ID Merge Isolation

Each transaction ID has its own outstanding-chunk counter and worst-response accumulator. This keeps concurrent chopped bursts from different IDs from crossing: a late R beat on ID *M* cannot corrupt the merge state for ID *N*.

### 24.3.3 Address Advance

For `INCR` bursts the start address of chunk *n+1* is the previous chunk's start address, **aligned to the transfer size**, plus `chunk_beats << size`: every beat after the first is size-aligned, so an unaligned first address must not carry into chunk 2. (Until 2026-10-19 the alignment was missing and every later chunk of an unaligned burst started unaligned, converters BUG-011.) `FIXED` bursts reuse the same address for every chunk; `WRAP` bursts longer than 16 beats are unsupported (see 3.15.7 Non-goals).

## 24.4 3.15.4 Write Path (axi4_to_axi3_wr)

### 24.4.1 Operation

The write path mirrors the read path on AW/W/B:

- AW bursts longer than 16 beats are chopped into multiple AXI3 AW bursts.
- Each AXI3 W beat is stamped with the owning AWID on `m_axi3_wid`.
- `WLAST` is forced at the end of every chopped chunk, producing a legal AXI3 WLAST for each sub-burst.
- Per-chunk B responses are merged worst-rank and returned as a single `s_axi_bresp` after the last chunk's B.

### 24.4.2 Early-W Buffering

AXI4 allows W to arrive before AW. The converter buffers up to 16 full W beats ahead of the matching AW handshake. When the buffer is full, `s_axi_wready` is deasserted, which is the legal AXI4 mechanism for throttling early write data.

The write segmenter consumes the buffered beats using a descriptor FIFO pushed at AW acceptance, one entry per AXI4 AW. The descriptor carries the AWID and the burst's total beat count. The segmenter cuts that count into AXI3 bursts of up to 16 and places WID and WLAST on each, even for beats that arrived before the AW. Consecutive AXI3 bursts of one AXI4 AW follow each other without a dead cycle.

### 24.4.3 Storage

The descriptor FIFO and the early-W buffer are `gaxi_fifo_sync` instances from `rtl/amba/gaxi` (depths 4 and 16). The read path has no queue. They used to be SystemVerilog queues, which Vivado does not synthesize ("[Synth 8-27] dollar-sign operator not supported"). An AW arriving while the W segmenter is idle, or finishing, with the descriptor FIFO empty bypasses the FIFO, so the FIFOs add no latency.

### 24.4.4 Request Flow and Per-ID Ordering

Both paths take one request per cycle and hold no granted request behind the bridge's arbiter:

- An unchopped request (`AxLEN ≤ 15`) passes straight to the AXI3 port and is accepted with it.
- A chopped one (`AxLEN > 15`) is best effort. It is taken into a staging register with no per-ID check, and moves to the active register, which issues its AXI3 chunks, a cycle later once its ID has drained. New requests wait while it is staged or issuing. The write path pushes the chopped write's W descriptor when it moves, so its W starts with its AW as before.

- Up to `MAX_OUTSTANDING` (default 32) transactions may be in flight per direction, across all IDs. An unchopped transaction needs no per-ID check. The AXI3 slave returns same-ID responses in order, so an unchopped transaction behind a chopped one of its ID comes back after the chopped one's last chunk, and the chunk counter finds that chunk.
- A chopped transaction waits for its ID to drain: an unchopped one ahead of it would otherwise be counted as one of its chunks. The check reads the staged ID from registers, so neither the per-ID lookup nor the chunk-address load is in series with the incoming AR / AW. (Until 2026-10-18 the check ran on the incoming request and the read converter's chunk-address adder hung off the crossbar's AR grant: 17 LUT levels, -0.387 ns at 10 ns on the Artix-7 in `bridge_mix_e`, bridge W13.)
- B from the AXI3 port lands raw in a one-entry output register, and the per-ID merge runs from the registered BID on its way out, so no ID-indexed logic sits between the BID pin and a flop (bridge W13: the BID pin fed the merge count combinationally). BREADY means the register is empty or leaving, so it never depends on BID, and a downstream that waits for BVALID before BREADY still drains it. A chopped write's non-final chunk B leaves the register absorbed, merged into its write's response, without being forwarded. The register was already there, so this costs no cycle. (A skid in front of it cost single-beat AXI3 writes a cycle of round trip: 0.801 against 0.862 beat/cycle at 8 in flight.)
- W passes straight through once a descriptor is loaded and nothing is buffered ahead of it.

Until 2026-10-18 both paths took one transaction per ID at a time. They issued at most one AR or AW every other cycle, lost a W cycle per AXI3 burst, and queued four requests behind the arbiter (bridge W8: single-beat reads 0.50, same-ID single-beat reads 0.334, round-robin share 0.26).

## 24.5 3.15.5 Bidirectional Wrapper (axi4_to_axi3)

The top wrapper instantiates the read and write converters, routes the clock and reset, and exposes the union of both submodule port lists: a full AXI4 slave interface and a full AXI3 master interface. The wrapper adds no logic beyond the submodule instantiations.

## 24.6 3.15.6 Parameters

*Table 3.46: AXI4 to AXI3 parameters*

| Parameter | Type | Default | Description |
|---|---|---|---|
| AXI_ID_WIDTH | int | 8 | Transaction ID width (1–16). |
| AXI_ADDR_WIDTH | int | 32 | Address width (12–64). |
| AXI_DATA_WIDTH | int | 32 | Data width; must match on both sides (32/64/128/256). |
| AXI_USER_WIDTH | int | 1 | User-signal width (0–1024). |
| MAX_OUTSTANDING | int | 32 | Transactions in flight per direction, all IDs together. |

## 24.7 3.15.7 Non-goals

The following AXI3/AXI4 features are intentionally outside the scope of this converter:

- **WRAP bursts longer than 16 beats.** AXI3 itself limits all bursts to 16 beats; an AXI4 WRAP burst longer than that has no legal AXI3 representation and is rejected by an assertion.
- **Write-data interleaving.** The converter stamps every W beat with the current AWID and does not support AXI3 interleaved W IDs.
- **Data-width conversion.** `AXI_DATA_WIDTH` must be identical on both sides.
- **QoS / REGION propagation.** These AXI4-only qualifiers are dropped.

## 24.8 3.15.8 Resource Utilization

*Table 3.47: AXI4 to AXI3 resources*

| Module | Registers | LUTs | BRAM |
|---|---|---|---|
| axi4_to_axi3_rd | ~200 | ~250 | 0 |
| axi4_to_axi3_wr | ~300 | ~350 | 0 |
| axi4_to_axi3 (combined) | ~500 | ~600 | 0 |

The tables are approximate hand estimates; no post-route characterization has been run.

## 24.9 3.15.9 Testing

**Test Suite:** `test_axi4_to_axi3_rd.py`, `test_axi4_to_axi3_wr.py`, `test_axi4_to_axi3.py`.

*Table 3.48: AXI4 to AXI3 test coverage*

| Test Category | Coverage | Status |
|---|---|---|
| Single-beat read | Pass-through, ID preservation, ARLOCK mapping | Pass |
| Burst read ≤ 16 beats | Pass-through, RLAST/RRESP integrity | Pass |
| Burst read > 16 beats | Chopping, address advance, worst-response merge | Pass |
| Multi-ID concurrent reads | Per-ID merge isolation | Pass |
| Single-beat write | Pass-through, WID stamping | Pass |
| Burst write ≤ 16 beats | WLAST, B pass-through | Pass |
| Burst write > 16 beats | AW chopping, per-chunk WLAST, B merge | Pass |
| Early-W before AW | 16-beat buffer, backpressure on 17th beat | Pass |
| Multi-outstanding writes across IDs | Inter-ID concurrency; same-ID unchopped writes overlap, a chopped one holds its ID | Pass |
| Chopped read behind an unchopped read of its ID | The staged read waits for its ID to drain (fails without the check) | Pass |
| Chopped write behind an unchopped write of its ID | Same for writes: a SLVERR on the chopped write's second chunk must reach the chopped write (fails without the | Pass |

| Test Category | Coverage | Status |
|---|---|---|
| | check) | |

**Next:** AXI3 to AXI4

## 24.10 References

- ARM, *AMBA 3 AXI Protocol Specification*, ARM IHI 0022 — verified against this spec for AXI3-specific behavior: 4-bit `AxLEN`, 2-bit `AxLOCK`, write-data `WID`, and absence of `QoS`/`REGION`.
- ARM, *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022E — verified against this spec for AXI4-specific behavior: 8-bit `AxLEN`, 1-bit `AxLOCK`, no `WID`, and `QoS`/`REGION` qualifiers.

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 25 3.16 AXI3 to AXI4 Converter

The **axi3_to_axi4** converter family bridges an AXI3 master to an AXI4 slave. It is the complement of the AXI4-to-AXI3 converter and is used on the DE10 leg-bridge `h2f_axi_master` seam when traffic flows from the AXI3 hard processor subsystem toward AXI4 fabric logic.

## 25.1 3.16.1 Module Organization

```
axi3_to_axi4.sv          # Full bidirectional wrapper
├── axi3_to_axi4_rd.sv   # Read path: AR/R qualifier mapping
└── axi3_to_axi4_wr.sv   # Write path: AW/W/B with WID stripping
```

As with the opposite direction, read and write are independent modules for selective instantiation and targeted verification.

## 25.2 3.16.2 Protocol Deltas

*Table 3.49: AXI3 to AXI4 protocol differences*

| Aspect | AXI3 (slave side) | AXI4 (master side) | Converter action |
|---|---|---|---|
| `AxLEN` width | 4 bits (1–16 beats) | 8 bits (1–256 beats) | Zero-extend `AxLEN` (`{4'b0, s_axi3_axlen}`). No chopping needed. |
| `AxLOCK` | 2 bits (`2'b00` normal, `2'b01` exclusive, `2'b10` locked) | 1 bit (normal / exclusive) | Map `2'b01` → `1'b1`; `2'b00`, `2'b10` (locked, which AXI4 dropped), and the reserved `2'b11` → `1'b0`. |
| Write data ID | `WID` per beat | None | Strip `WID`; AXI4 W has no ID concept. |
| `QoS` / `REGION` | Absent | Present on AW/AR | Drive to zero. |
| Data width | Must match | Must match | No width conversion. |

### 25.2.1 Exclusive-access caveat

The DE10 leg-bridge `h2f_axi_master` seam does not support AXI exclusive accesses. The converter maps an AXI3 exclusive request (`AxLOCK = 2'b01`, per ARM IHI 0022) to the AXI4 encoding (`AxLOCK = 1'b1`), but an AXI3 locked transaction (`AxLOCK = 2'b10`) is collapsed to normal (`AxLOCK = 1'b0`) because AXI4 dropped locked accesses and the HPS interface does not implement locked-transaction semantics. (The encoding was swapped — `2'b10` treated as exclusive — until bridge GitHub #98; the RTL, this table, and the directed lock-map DV now agree with the ARM encoding.)

## 25.3 3.16.3 Read Path (axi3_to_axi4_rd)

The read path is a straight qualifier mapping block:

- `ARID`, `ARADDR`, `ARSIZE`, `ARBURST`, `ARCACHE`, `ARPROT`, and `ARUSER` pass through unchanged.
- `ARLEN` is zero-extended from 4 bits to 8 bits.
- `ARLOCK` is mapped from AXI3 2-bit encoding to AXI4 1-bit encoding.
- `ARQOS` and `ARREGION` are driven to `4'b0`.
- The R channel passes through unchanged (`RID`, `RDATA`, `RRESP`, `RLAST`, `RUSER`).

No burst chopping is required because AXI3 bursts are already bounded to 16 beats, which fits comfortably in a single AXI4 transaction.

## 25.4 3.16.4 Write Path (axi3_to_axi4_wr)

### 25.4.1 Operation

The write path maps the AXI3 AW/W/B channels to AXI4:

- `AWID`, `AWADDR`, `AWSIZE`, `AWBURST`, `AWCACHE`, `AWPROT`, and `AWUSER` pass through.
- `AWLEN` is zero-extended from 4 bits to 8 bits.
- `AWLOCK` is mapped from 2-bit AXI3 to 1-bit AXI4.
- `AWQOS` and `AWREGION` are driven to `4'b0`.
- `WID` is stripped; the AXI4 W channel has no ID.
- `WDATA`, `WSTRB`, `WLAST`, and `WUSER` pass through.
- B channel (`BID`, `BRESP`, `BUSER`) passes through.

### 25.4.2 Write Ordering Contract

`axi3_to_axi4_wr` passes AW, W and B straight through, so any number of writes may be in flight. The contract is AXI3 write-interleave depth 1: W bursts arrive in AW order, and W may lead its AW. AXI3 allows write-data interleaving across IDs, but supporting it would mean demuxing W beats by `WID` into per-ID buffers before AXI4, which has no WID. The upstream AXI3 master must not interleave.

Until 2026-10-18 the converter took one write at a time: AWREADY was held low from an AW's acceptance until its B. That capped an AXI3 master's writes at beats / (beats + round trip), 0.111 beat/cycle single-beat and 0.674 at 16 beats behind the bridge (bridge W7).

### 25.4.3 WID Order Check

A simulation-only check (`ifndef SYNTHESIS`), `ASSERT_A3_WID_STABLE`, fires if `s_axi3_wid` changes inside a burst, or if a burst's WID is not the AWID of the AW it pairs with. Pairing goes in order (n-th AW with n-th W burst), whichever arrives first. A dedicated negative test (`test_axi3_to_axi4_wr_wid_mismatch`) verifies that it fires, and `overlapped_writes` verifies that a second AW is accepted before the first B returns.

## 25.5 3.16.5 Bidirectional Wrapper (axi3_to_axi4)

The top wrapper instantiates the read and write converters, ties off the AXI4-only `QOS`/`REGION` outputs to zero, and exposes the union of both submodule port lists. No additional protocol logic is present in the wrapper.

## 25.6 3.16.6 Parameters

*Table 3.50: AXI3 to AXI4 parameters*

| Parameter | Type | Default | Description |
|---|---|---|---|
| AXI_ID_WIDTH | int | 8 | Transaction ID width (1–16). |
| AXI_ADDR_WIDTH | int | 32 | Address width (12–64). |
| AXI_DATA_WIDTH | int | 32 | Data width; must match on both sides (32/64/128/256). |
| AXI_USER_WIDTH | int | 1 | User-signal width (0–1024). |

## 25.7 3.16.7 Non-goals

The following features are intentionally outside the scope of this converter:

- **Interleaved W IDs.** W bursts must follow AW order; interleaved or reordered W data is flagged by `ASSERT_A3_WID_STABLE` in simulation.
- **Data-width conversion.** `AXI_DATA_WIDTH` must be identical on both sides.
- **QoS / REGION generation.** These AXI4-only qualifiers are driven to zero.

## 25.8 3.16.8 Resource Utilization

*Table 3.51: AXI3 to AXI4 resources*

| Module | Registers | LUTs | BRAM |
|---|---|---|---|
| axi3_to_axi4_rd | ~20 | ~30 | 0 |
| axi3_to_axi4_wr | ~40 | ~50 | 0 |
| axi3_to_axi4 (combined) | ~60 | ~80 | 0 |

The tables are approximate hand estimates; no post-route characterization has been run.

## 25.9 3.16.9 Testing

**Test Suite:** `test_axi3_to_axi4_rd.py`, `test_axi3_to_axi4_wr.py`, `test_axi3_to_axi4.py`, `test_axi3_loopback.py`.

*Table 3.52: AXI3 to AXI4 test coverage*

| Test Category | Coverage | Status |
|---|---|---|
| Single-beat read | Pass-through, ARLEN zero-extension, ARLOCK mapping | Pass |
| Burst read ≤ 16 beats | Pass-through, RID/RRESP/RLAST integrity | Pass |
| Single-beat write | Pass-through, WID strip | Pass |
| Burst write ≤ 16 beats | WLAST/B pass-through | Pass |
| Overlapped writes | Second AW accepted before the first B returns; both complete in order | Pass |
| WID mismatch negative test | `ASSERT_A3_WID_STABLE` fires on wrong WID | Pass |
| Loopback equivalence | `axi4_to_axi3` → `axi3_to_axi4` identity for legal AXI4 traffic | Pass |

**Next:** Verification

## 25.10 References

- ARM, *AMBA 3 AXI Protocol Specification*, ARM IHI 0022 — verified against this spec for AXI3-specific behavior: 4-bit `AxLEN`, 2-bit `AxLOCK`, write-data `WID`, and absence of `QoS`/`REGION`.
- ARM, *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022E — verified against this spec for AXI4-specific behavior: 8-bit `AxLEN`, 1-bit `AxLOCK`, no `WID`, and `QoS`/`REGION` qualifiers.

# 26 3.17 AXI-Stream to CXS Adapters

The **axis_to_cxs** and **cxs_to_axis** adapters let an AXI-Stream design use a credited CXS link (Arm IHI 0079, Issue D) without changing its stream logic. `axis_to_cxs` sits on the FUB side of a `cxs_master` and turns AXIS beats into CXS flits; `cxs_to_axis` sits on the FUB side of a `cxs_slave` and turns them back. The CXS endpoints and the rest of the CXS family are in `rtl/amba/cxs/` (`docs/markdown/rtl-amba/cxs/README.md`).

## 26.1 3.17.1 Module Organization

```
projects/components/utility-ip/converters/rtl/
  axis_to_cxs.sv    AXIS beats -> CXS flits, one packet per TLAST run
  cxs_to_axis.sv    CXS flits -> AXIS beats, CXS END -> TLAST
rtl/filelists/axis_to_cxs.f, cxs_to_axis.f
```

Both are combinational on the data path and pass the valid/ready handshake straight through, so neither adds a pipeline stage. `axis_to_cxs` keeps one flop per stream key (whether that stream has a packet open), and `cxs_to_axis` keeps one sticky flag. One AXIS beat is one CXS flit, and one TLAST-delimited AXIS packet is one CXS packet.

## 26.2 3.17.2 Field Mapping

*Table 3.53: AXI-Stream to CXS field mapping*

| CXS field | From the AXIS beat |
|---|---|
| CXSDATA | TDATA |
| CXSCNTL START[0] | first beat of a stream's packet, START0PTR = 0 |
| CXSCNTL END[0] | the TLAST beat; END0PTR = the highest 4-byte word with any strobe set |
| CXSSRCID | TID |
| CXSTGTID | TDEST |
| CXSUSER | {TUSER, TSTRB} (CARRY_TUSER = 1) or TSTRB alone (CARRY_TUSER = 0) |

CXS packets end on a 4-byte boundary, but AXIS packets can end on any byte. Carrying TSTRB in CXSUSER keeps byte-exact framing across the link: cxs_to_axis restores TSTRB from the user bits, so a packet of any length or alignment comes out exactly as it went in. ENDERROR is never set. On the way back, TLAST is END[0] of the flit.

## 26.3 3.17.3 Interleaved Packets

AXIS may interleave packets of different TID/TDEST, and CXS lets flits of different streams (CXSSRCID/CXSTGTID) interleave too (IHI 0079D 2.5.2). So `axis_to_cxs` frames START per stream: one state bit per key built from the low `STREAM_ID_BITS` of TID and the low `STREAM_DEST_BITS` of TDEST (`cxs_stream_key`). The default 0/0 means the input never interleaves. The receiving `cxs_slave` must track the same streams (`CXS_STREAM_SRCID_BITS` / `CXS_STREAM_TGTID_BITS`). `cxs_to_axis` is stateless, so it needs no key.

## 26.4 3.17.4 Parameters

*Table 3.54: AXI-Stream to CXS adapter parameters*

| Parameter | Default | Description |
|---|---|---|
| DATA_WIDTH | 512 | TDATA and CXSDATAFLITWIDTH (256 / 512 / 1024) |
| ID_WIDTH / DEST_WIDTH / USER_WIDTH | 8 / 4 / 1 | TID / TDEST / TUSER; 0 is legal |
| CXS_MAX_PKT_PER_FLIT | 2 | Sizes the CXSCNTL layout; must be at least 2 |
| CXS_START_ALIGN | 16 | CXS_START_ALIGNMENT in bytes |
| CARRY_TUSER | 1 | 0 leaves TUSER off the link, so CXSUSER = TSTRB only |
| STREAM_ID_BITS / STREAM_DEST_BITS | 0 / 0 | Per-stream framing key (axis_to_cxs only) |

Derived: `CXS_UW = DATA_WIDTH/8 + (CARRY_TUSER ? USER_WIDTH : 0)`. CXSUSER is at most 128 bits, so carrying strobes limits DATA_WIDTH to 1024, or to `8 * (128 - USER_WIDTH)` with TUSER carried. The link must be Issue D (`CXS_ISSUE = 4`), because the SRCID, TGTID and USER sidebands exist only there.

## 26.5 3.17.5 Limits

- `axis_to_cxs` never packs two packets into one flit. That costs some link efficiency on short packets, but it is what lets a flit map back onto a single AXIS beat.
- AXIS allows only one TLAST per beat, so `cxs_to_axis` handles only flits shaped the way `axis_to_cxs` makes them: at most one START and one END, and START at pointer 0. If a transmitter sends a flit with two packet boundaries, or a packet starting mid-flit, the sticky `unsupported` output is set. The beat is still forwarded, with TLAST taken from END[0].
- Parity (`Odd_Byte_Parity`) is not part of these adapters. Configure it on the endpoints.

## 26.6 3.17.6 Used By and Verification

*Table 3.55: AXI-Stream to CXS adapter users*

| User | Path |
|---|---|
| RAPIDS CXS network planes | `projects/components/dma-ip/rapids/rtl/plane/rapids_plane_cxs_{tx,rx}.sv` |
| CXS pattern injector / checker | `rtl/amba/shared/cxs_master_injector.sv`, `cxs_slave_pattern_check.sv` |

They are verified through those users: `val/amba/test_cxs_pattern_pair.py` (every AXIS pattern-pair scenario, interleave included, across a CXS link with a wire monitor) and the RAPIDS CXS top suites (`rapids_cxs_top`, `rapids_beats_cxs_top`, including concurrent channels interleaved mid-packet).

# 27 4.1 Width Converter FSMs

## 27.1 Overview

Here's the twist with the width converters: the upsize and downsize primitives don't have a state machine at all, and that's deliberate — what they have instead is tighter. The full AXI4 dwidth converters do run explicit FSMs, and those are small.

## 27.2 Functional Description

### 27.2.1 4.1.1 Upsize Structure (no FSM)

The **axi_data_upsize** module has NO state machine. Earlier revisions of this chapter documented an ACCUMULATE/OUTPUT FSM; the RTL is a register-and-gating structure built deliberately so that output and the next accumulation OVERLAP — an explicit OUTPUT state would insert the very bubble the design avoids.

The real structure:

- `r_beat_ptr` walks slots as narrow beats land in the accumulator. A fresh BURST's first beat lands at `start_lane` (mid-word INCR starts, projects/components/utility-ip/converters TASK-001 (was CONV-006)), so the effective lane is `w_lane = (ptr=='0) ? (burst_fresh ? start_lane : 0) : ptr;`
- a group completes when the LANE reaches `WIDTH_RATIO-1` or on an early `narrow_last` — a mid-word burst's first wide group holds `WIDTH_RATIO - start_lane` beats, not `WIDTH_RATIO`; later groups start at lane 0 and hold the full count;
- `r_wide_valid` presents the completed group on the wide side while the NEXT group starts accumulating — when a wide handshake and a completing narrow beat coincide, a single priority selection keeps the freshly completed group. (Last-write-wins ordering across two NBA blocks was the BUG here, not the fix — the wide-accept block's clear clobbered the fresh group's valid; the dnsize is the one that legitimately relies on last-write-wins, for its atomic replace.)
- `narrow_ready = !r_wide_valid || wide_ready`, so the narrow side stalls only when the wide side is holding a beat hostage.

On an early `narrow_last`, unwritten accumulator slots are zeroed in the same single write that deposits the first beat of a group — residual data from the previous group must not leak into a partial wide beat.

### 27.2.2 4.1.2 Downsize Structure (single buffer, no FSM)

Same story: no IDLE/LOAD/OUTPUT machine. The single buffer performs an **atomic replace** — `wide_ready` asserts during the LAST narrow beat of the current wide beat, so the replacement lands as the drain finishes and a steady stream pays no per-beat bubble (measured 0.992 beats/cycle, see 2.3):

```
assign wide_ready = !r_wide_buffered || (narrow_ready &&
w_last_narrow_beat);
```

With TRACK_BURSTS=1 the condition narrows to mid_burst_replace, which excludes each burst's final beat — one bubble per burst boundary, not per beat. A beat pointer (r_beat_ptr) selects the slice driven onto the narrow side; on the burst's FIRST wide word it starts at start_lane (mid-word INCR starts, projects/components/utility-ip/converters TASK-001 (was CONV-006) — the narrow master receives the bytes it addressed, not the aligned-down word's bytes), and at lane 0 for every later wide word. narrow_last in tracked mode comes only from the beat counter reaching burst_len + 1.

## 27.2.3 4.1.3 Full Converter FSMs

### *27.2.3.1 Write Converter (axi4_dwidth_converter_wr)*

```
IDLE:
  - AW valid → accept AW, store info
  - W valid → buffer W data

AW_ACCEPT:
  - downstream AW ready → forward adjusted AW

W_CONVERT:
  - upsize accumulating narrow W beats
  - on output → forward wide W beat

B_FORWARD:
  - B from downstream → forward to master
```

### *27.2.3.2 Read Converter (axi4_dwidth_converter_rd)*

```
IDLE:
  - AR valid → accept AR, store info, forward adjusted AR

AR_FORWARD:
  - downstream AR ready → wait for R

R_CONVERT:
  - downsize splitting wide R into narrow beats
  - track burst count for RLAST

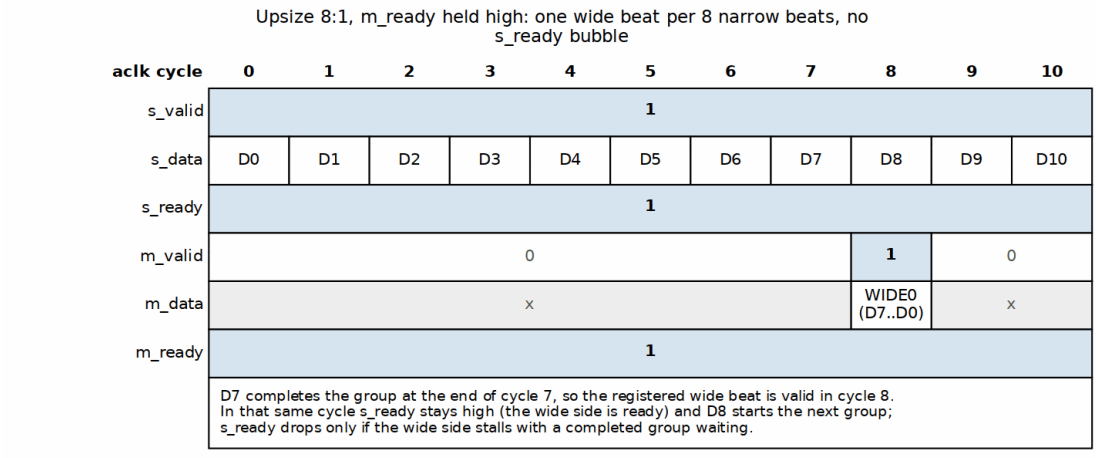
R_FORWARD:
  - narrow R beat ready → forward to master
  - on RLAST → IDLE
```

## 27.3 Timing

### 27.3.1 Upsize (8:1 ratio, m_ready held high)

No `s_ready` bubble: the wide handshake overlaps the next group's first narrow beat.

### 27.3.2 Figure 4.1: Upsize 8:1 Timing



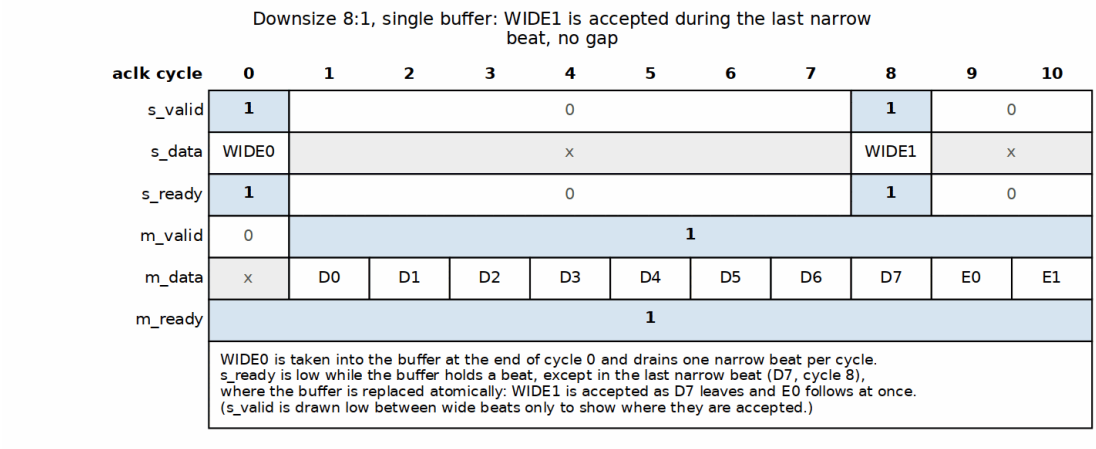

*Upsize 8:1 Timing*

`s_ready` only drops if the wide side stalls with a completed group waiting (`!r_wide_valid || wide_ready` fails).

### 27.3.3 Downsize (8:1 ratio, single buffer, atomic replace)

`s_ready` re-asserts DURING the last narrow beat, so WIDE1 is accepted as D7 drains — back-to-back wide beats with no gap:

### 27.3.4 Figure 4.2: Downsize 8:1 Timing




*Downsize 8:1 Timing*

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 28 4.2 Protocol Converter FSMs

## 28.1 Overview

The state machines inside the protocol converters: every one of them is smaller than the earlier documentation made it look.

## 28.2 Functional Description

### 28.2.1 4.2.1 AXI4 to AXI4-Lite Read FSM

The real machine has THREE states, and none of them waits for R:

```
typedef enum logic [1:0] {
    RD_IDLE       = 2'b00,   // no burst in flight
    RD_BURST      = 2'b01,   // issuing the AXIL4 reads of a burst
    RD_LAST_BEAT  = 2'b10    // issuing the burst's final read
} rd_state_t;
```

Earlier revisions documented IDLE/SINGLE/DECOMPOSE/WAIT_R. Two of those never existed: a single-beat read (`ARLEN==0`) takes a passthrough leg and never leaves `RD_IDLE`, and there is no wait-for-R state at all — the address and data paths are decoupled, with ARs issuing at the downstream accept rate regardless of R (see 3.2). What serializes bursts is not an FSM state but the one-outstanding-burst guard: the burst-tracking registers (`r_r_id`/`r_r_len`/`r_r_beat_count`) hold exactly one burst, so `s_axi_aready` is held off until the in-flight burst delivers its last beat. Response aggregation is the live-beat worst-case fold of 4.3.3.

### 28.2.2 4.2.2 AXI4 to AXI4-Lite Write FSM

Also three states (`WR_IDLE`/`WR_BURST`/`WR_LAST_BEAT`), same passthrough leg for `AWLEN==0`, same one-outstanding guard on the B side. There are no `r_aw_pending`/`r_w_pending` flags: W is gated directly against the state of the AW that owns it —

- during the burst-capture cycle W is held off (`w_burst_capture`), or the first W beat of the next burst would slip out while the previous burst is finishing and pair with the wrong address;
- during a burst, W passes only once this burst's AW has been sent (`r_aw_sent`) or is being sent in the same cycle;
- single beats pass straight through.

See 3.2's "W gated on the AW that owns it" for the exact equations.

### 28.2.3 4.2.3 AXI4 to APB FSMs

The convert core runs a command FSM (`IDLE`/`READ`/`WRITE`, writes preferred) that packetizes each burst into per-APB-beat commands, and a response FSM (`RSP_IDLE`/`RSP_ACTIVE`) that reassembles rsp packets into AXI responses. PSEL/PENABLE setup/access phases are NOT here — they belong to `apb4_master` on the far side of the shim's CDC. Full description in 3.4.5.

## 28.3 Timing

### 28.3.1 4.2.4 Timing Analysis

**AXI4 to AXIL4.** A single-beat transfer is a combinational passthrough — no converter-inserted wait state. In a burst the decomposed requests issue at the downstream ACCEPT rate, independent of responses: `m_axil_arvalid` in RD_BURST holds every cycle and the beat tracker advances on `m_axil_arvalid && m_axil_arready` — no signal in the request path samples `m_axil_rvalid`/`m_axil_bvalid`. Against a pipelining slave, N requests issue in N consecutive cycles and the burst finishes one response latency later; the old "2 cycles overhead / 2N cycles per burst" figures described a serialization the RTL does not have. Bursts do not overlap each other (the one-outstanding guard above); that is the only converter-imposed serialization.

**AXI4 to APB.** Per APB beat: command packet -> CDC (2-flop gray crossing each way) -> APB setup + access (2 pclk minimum, plus PREADY stalls) -> response packet -> CDC back. Latency is dominated by the two clock crossings and the APB protocol itself; no measured characterization is published here.

# 29 4.3 Burst Decomposition

## 29.1 Overview

Burst handling is where the two converter families diverge: width converters rescale the burst length, protocol converters split the burst into single beats. Same AXI burst in — two very different kinds of bookkeeping out.

## 29.2 Functional Description

### 29.2.1 4.3.1 Width Converter Burst Handling

#### *29.2.1.1 Burst Length Adjustment*

When converting widths, burst length changes inversely with data width:

```
M_AWLEN = ceil((S_AWLEN + 1) / RATIO) - 1  =  ((S_AWLEN + 1) + RATIO -
1) / RATIO - 1

Example (64-bit to 512-bit, RATIO=8):
  S_AWLEN = 7 (8 beats × 64 bits = 512 bits)
  M_AWLEN = (7 + 1) / 8 - 1 = 0 (1 beat × 512 bits)

  S_AWLEN = 15 (16 beats × 64 bits = 1024 bits)
  M_AWLEN = (15 + 1) / 8 - 1 = 1 (2 beats × 512 bits)
```

#### *29.2.1.2 Non-Aligned Bursts*

When the burst length is not a multiple of the ratio:

```
S_AWLEN = 5 (6 beats), RATIO = 8
M_AWLEN = ((5 + 1) + 7) / 8 - 1 = 0  (1 wide beat -- the ceiling term
is what prevents the -1 a floor divide gives)

The 6 narrow beats pack into 1 wide beat.
Last 2 positions have WSTRB = 0 (no write).
```

### 29.2.2 4.3.2 Protocol Converter Burst Handling

#### *29.2.2.1 AXI4 to AXI4-Lite Decomposition*

AXI4-Lite only supports single-beat transactions, so all bursts must be decomposed:

```
AXI4 Burst:
  ARADDR = 0x1000
  ARLEN = 3 (4 beats)
  ARSIZE = 2 (4 bytes)

AXIL4 Sequence:
  Transaction 0: ARADDR = 0x1000
  Transaction 1: ARADDR = 0x1004
  Transaction 2: ARADDR = 0x1008
  Transaction 3: ARADDR = 0x100C
```

#### *29.2.2.2 Address Increment Calculation*

```
// Calculate address increment based on burst type and size
function automatic [ADDR_WIDTH-1:0] next_address(
    input [ADDR_WIDTH-1:0] current_addr,
    input [2:0] size,
    input [1:0] burst,
    input [7:0] len,
    input [7:0] beat
);
    logic [ADDR_WIDTH-1:0] increment;
    logic [ADDR_WIDTH-1:0] wrap_mask;

    increment = 1 << size;

    case (burst)
        2'b00: // FIXED
            return current_addr;  // No increment

        2'b01: // INCR
            return current_addr + increment;

        2'b10: // WRAP
            wrap_mask = ((len + 1) << size) - 1;
            return (current_addr & ~wrap_mask) |
                   ((current_addr + increment) & wrap_mask);

        default:
            return current_addr + increment;
    endcase
endfunction
```

### 29.2.3 4.3.3 Response Aggregation

Several downstream responses collapse into one upstream response, worst case winning (OKAY < EXOKAY < SLVERR < DECERR by numeric compare). The subtlety is WHERE the final beat's own response enters the result, and getting it wrong is not hypothetical: the converters shipped with the registered-only version below, and every single-beat error was reported upstream as OKAY.

**Wrong (the shipped bug).** Accumulate in a register and emit the register:

```
always_ff @(posedge aclk or negedge aresetn) begin
    ...
    else if (m_axil_bvalid && m_axil_bready)
        if (m_axil_bresp > r_b_resp_accum)
            r_b_resp_accum <= m_axil_bresp;
end
```

```
assign s_axi_bresp = r_b_resp_accum;   // WRONG
```

The accumulator is a non-blocking assign: on the handshake that emits the response it does not yet contain THAT beat's `bresp`. For a single-beat transfer the only beat is the final one, so its error is exactly the one dropped.

**Right (the current RTL).** Fold the live beat combinationally at the point of emission:

```
assign w_b_resp_worst = (m_axil_bresp > r_b_resp_accum) ? m_axil_bresp
                                                        :
r_b_resp_accum;
assign s_axi_bresp    = w_b_resp_worst;
```

The read path is the same shape, gated to the final beat:

```
assign s_axi_rresp = (r_r_beat_count == r_r_len) ? w_r_resp_worst
                                                 : m_axil_rresp;
```

Both were fixed with tests that inject SLVERR from the downstream slave and assert it reaches the upstream master — including the last-beat-only case, where earlier beats cannot mask the loss. See `test_error_response` in the axi4_to_axil4 testbenches.

### 29.2.4 4.3.4 Burst Tracking Registers

#### *29.2.4.1 Required State*

The listing below is the generic PATTERN, not the RTL's register names: the real modules split this state between separate read and write converters and count UP (`r_ar_beat_count`/`r_aw_beat_count` against a stored length) rather than down — grep for those names, not these.

```
// Burst tracking registers (generic pattern)
logic [ADDR_WIDTH-1:0] r_base_addr;
logic [ADDR_WIDTH-1:0] r_current_addr;
logic [7:0]            r_original_len;
logic [7:0]            r_remaining_beats;
logic [2:0]            r_size;
logic [1:0]            r_burst;
logic [ID_WIDTH-1:0]   r_id;
logic                  r_is_write;
```

#### *29.2.4.2 Initialization*

```
always_ff @(posedge clk) begin
    if (accept_new_transaction) begin
        r_base_addr <= s_axaddr;
        r_current_addr <= s_axaddr;
        r_original_len <= s_axlen;
        r_remaining_beats <= s_axlen;
```

```
        r_size <= s_axsize;
        r_burst <= s_axburst;
        r_id <= s_axid;
        r_is_write <= is_write_transaction;
    end else if (beat_complete) begin
        r_current_addr <= next_address(...);
        r_remaining_beats <= r_remaining_beats - 1;
    end
end
```

## 29.3 Timing

### 29.3.1 4.3.5 Timing Impact

#### *29.3.1.1 Decomposition Overhead*

*Table 4.6: Decomposition Overhead*

| Transaction Type | Overhead |
|---|---|
| Single-beat | 0 cycles (passthrough) |
| N-beat burst | none per beat – requests stream at the downstream accept rate, independent of responses (see 4.2.4) |
| Back-to-back bursts | serialized by the one-outstanding-burst guard |

The old “2 cycles per beat” figure described a request-response lockstep the RTL does not have; nothing in the address path waits on responses.

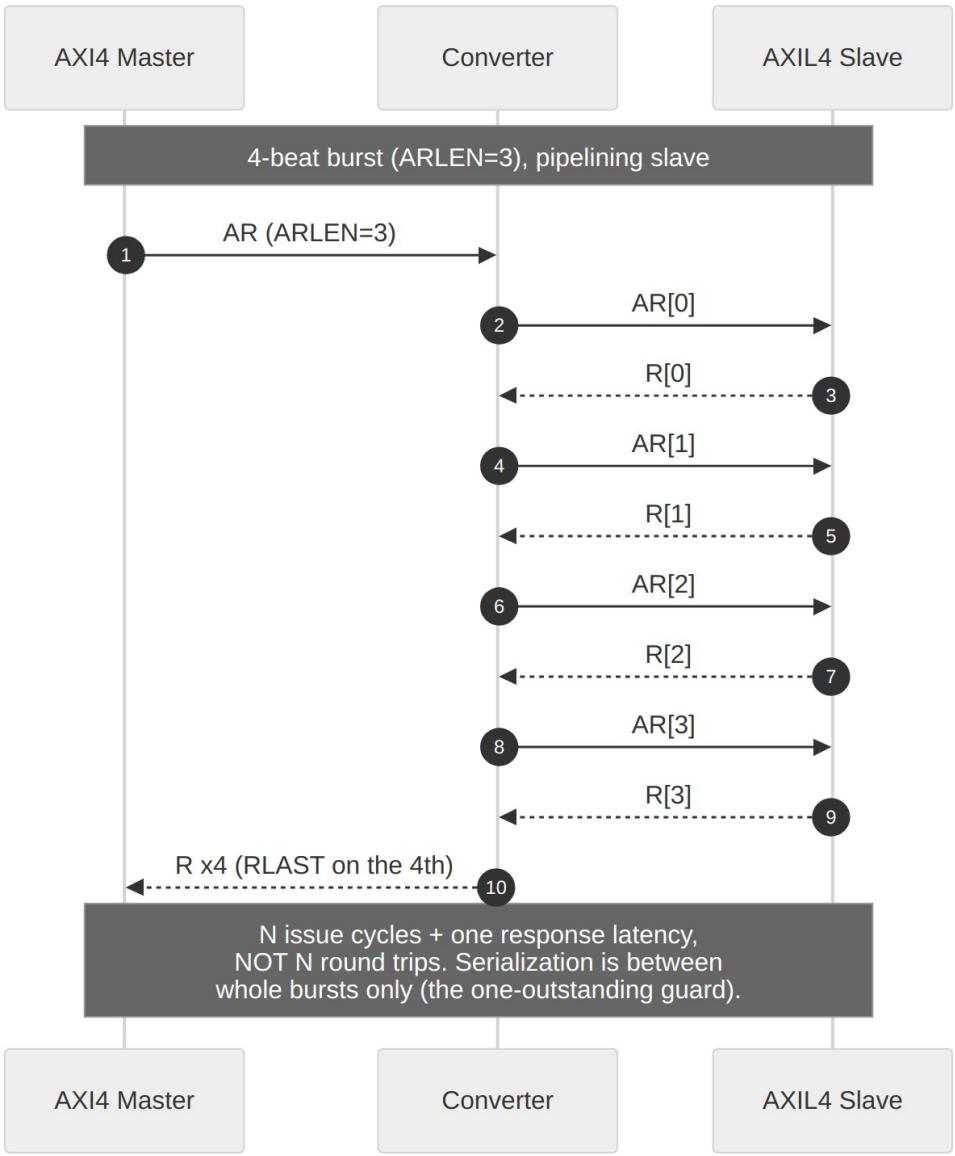
#### *29.3.1.2 Pipeline Considerations*

Decomposed requests are independent of responses (Table 4.6 above): - ARs issue at the downstream accept rate — nothing in the AR path samples `m_axil_rvalid` - AW/W beats advance on their own handshakes; intermediate Bs are absorbed (`m_axil_bready = s_axi_bready || !w_b_all_beats_done`) without stalling issuance

Against a pipelining AXIL4 slave, an N-beat burst costs N issue cycles plus one response latency — not N round trips. The only serialization the converter imposes is between whole bursts (the one-outstanding guard).

## 29.4 Waveforms

### 29.4.1 Figure 4.3: Width Burst Conversion




*Width Burst Conversion*

## 29.5 Navigation

**Next:** Chapter 5: Verification

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

# 30 5.1 Test Strategy

## 30.1 Overview

How the converter modules get verified, from unit tests to system stress: three test levels, a coverage model, and the exact commands to run it all.

## 30.2 Testing

### 30.2.1 5.1.1 Test Organization

```
projects/components/utility-ip/converters/dv/tests/
├── width_converters/
│   ├── test_axi_data_upsize.py
│   ├── test_axi_data_dnsize.py
│   ├── test_axi4_dwidth_converter_wr.py
│   └── test_axi4_dwidth_converter_rd.py
├── protocol_converters/
│   ├── test_axi4_to_axil4_rd.py
│   ├── test_axi4_to_axil4_wr.py
│   ├── test_axil4_to_axi4_rd.py
│   ├── test_axil4_to_axi4_wr.py
│   ├── test_axi4_to_apb4.py
│   └── test_peakrdl_adapter.py
└── integration/
    ├── test_width_protocol_chain.py
    └── test_full_system.py
```

### 30.2.2 5.1.2 Test Levels

#### *30.2.2.1 Level 1: Unit Tests*

**Purpose:** Verify individual module functionality in isolation.

**Coverage:** - All parameters combinations - Edge cases (min/max values) - Error injection

**Example:**

```
@pytest.mark.parametrize("narrow_width,wide_width", [
    (32, 64),
    (32, 128),
    (64, 256),
    (64, 512),
    (128, 1024),
])
async def test_upsize_ratios(dut, narrow_width, wide_width):
    """Test various width ratios."""
    tb = UpsizeTB(dut, narrow_width, wide_width)
    await tb.reset()
    await tb.run_basic_transfer(count=100)
    assert tb.scoreboard.check_passed()
```

#### *30.2.2.2 Level 2: Integration Tests*

**Purpose:** Verify module combinations work together.

**Coverage:** - Width converter + protocol converter chains - Back-to-back converters - Mixed traffic patterns

#### *30.2.2.3 Level 3: System Tests*

**Purpose:** Verify in realistic system context.

**Coverage:** - CPU-to-DDR paths - Peripheral access paths - Full bandwidth stress

### 30.2.3 5.1.3 Test Categories

#### *30.2.3.1 Functional Tests*

*Table 5.1: Functional Test Categories*

| Category | Description | Example |
|---|---|---|
| Basic | Single transactions | Single read, single write |
| Burst | Multi-beat transactions | INCR burst, WRAP burst |
| Mixed | Interleaved R/W | Read-modify-write sequences |
| Edge | Boundary conditions | Max burst length, min width |

#### *30.2.3.2 Stress Tests*

*Table 5.2: Stress Test Categories*

| Category | Description | Duration |
|---|---|---|
| Throughput | Maximum bandwidth | 10,000 transactions |
| Backpressure | Ready signal variations | Random delays |
| Reset | Reset during operation | Mid-transaction reset |

#### *30.2.3.3 Error Tests*

*Table 5.3: Error Test Categories*

| Category | Description | Expected Behavior |
|---|---|---|
| SLVERR | Slave error injection | Error propagation |
| DECERR | Decode error | Error response |

| Category | Description | Expected Behavior |
|---|---|---|
| Timeout | Response timeout | Error handling |

### 30.2.4 5.1.4 Testbench Architecture

#### *30.2.4.1 Components*

```
Testbench
├── Drivers
│   ├── AXI4 Master Driver
│   ├── AXI4 Slave Driver
│   ├── AXIL4 Master/Slave
│   └── APB Master/Slave
├── Monitors
│   ├── AXI4 Monitor
│   ├── AXIL4 Monitor
│   └── APB Monitor
├── Scoreboard
│   ├── Transaction Queue
│   ├── Response Checker
│   └── Coverage Collector
└── Generators
    ├── Random Transaction Generator
    └── Directed Sequence Generator
```

#### *30.2.4.2 Driver Implementation*

```python
class AXI4MasterDriver:
    def __init__(self, dut, clock, prefix="s_axi"):
        self.dut = dut
        self.clock = clock
        self.prefix = prefix

    async def write(self, addr, data, burst_len=0):
        """Issue AXI4 write transaction."""
        # AW phase
        # the DUT ports are prefixed (s_axi_awvalid, ...) -- resolve
        # through the stored prefix or every access AttributeErrors
        sig = lambda name: getattr(self.dut, f"{self.prefix}_{name}")
        sig("awvalid").value = 1
        sig("awaddr").value = addr
        sig("awlen").value = burst_len
        await self._wait_ready("awready")

        # W phase
        for i, d in enumerate(data):
            sig("wvalid").value = 1
            sig("wdata").value = d
            sig("wlast").value = (i == len(data) - 1)
```

```
            await self._wait_ready("wready")

        # B phase
        await self._wait_valid("bvalid")
        return sig("bresp").value
```

### 30.2.5 5.1.5 Coverage Model

#### *30.2.5.1 Functional Coverage*

```
@coverage
class ConverterCoverage:
    # Width ratio coverage
    ratio_cp = coverpoint(
        lambda: self.width_ratio,
        bins=[2, 4, 8, 16]
    )

    # Burst length coverage
    burst_len_cp = coverpoint(
        lambda: self.burst_len,
        bins=list(range(0, 256, 16)) + [255]
    )

    # Burst type coverage
    burst_type_cp = coverpoint(
        lambda: self.burst_type,
        bins=["FIXED", "INCR", "WRAP"]
    )

    # Cross coverage
    ratio_x_burst = cross(ratio_cp, burst_len_cp)
```

#### *30.2.5.2 Code Coverage*

**Target:** >95% line coverage, >90% branch coverage

*Table 5.4: Coverage Targets*

| Module | Line | Branch | FSM |
|---|---|---|---|
| axi_data_upsize | 98% | 95% | n/a (no FSM – see 4.1.1) |
| axi_data_dnsize | 97% | 93% | n/a (no FSM – see 4.1.2) |
| axi4_to_axil4_rd | 96% | 91% | 100% |

| Module | Line | Branch | FSM |
|---|---|---|---|
| axi4_to_axil4_wr | 95% | 90% | 100% |

### 30.2.6 5.1.6 Test Execution

#### *30.2.6.1 Running Tests*

```
# Run all converter tests
cd projects/components/utility-ip/converters/dv/tests
pytest -v

# Run specific module tests
pytest test_axi_data_upsize.py -v

# Run with coverage
pytest --cov=projects/components/utility-ip/converters/rtl -v

# Run with waveform dump
pytest test_axi_data_upsize.py -v --waves
```

#### *30.2.6.2 CI/CD Integration*

```
converter_tests:
  stage: test
  script:
    - cd projects/components/utility-ip/converters/dv/tests
    - pytest -v --junitxml=results.xml
    - pytest --cov=rtl --cov-report=html
  artifacts:
    reports:
      junit: results.xml
    paths:
      - htmlcov/
```

## 30.3 Navigation

**Next:** Debug Guide

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 31 5.2 Debug Guide

## 31.1 Overview

Start here when a converter misbehaves. Each entry runs symptoms, probe points, then fix — the order you want them in at the bench.

## 31.2 Testing

### 31.2.15.2.1 Common Issues

#### *31.2.1.1 Width Converter Issues*

##### 31.2.1.1.1 Issue: Data Corruption

**Symptoms:** - Output data doesn't match expected - Random bits flipped or missing

**Debug Steps:** 1. Check width ratio calculation: `RATIO = WIDE_WIDTH / NARROW_WIDTH` 2. Verify beat counter is correct: `$clog2(RATIO)` bits 3. Check data packing/unpacking slice indices 4. Verify sideband mode matches use case

**Waveform Checkpoints:**

```
r_beat_ptr             - slot/slice pointer, cycles 0 to RATIO-1
r_data_accumulator     - (upsize) each slot fills as narrow beats land
r_data_buffer          - (dnsize) captured wide beat being sliced
narrow_/wide_ data     - compare input/output patterns
```

##### 31.2.1.1.2 Issue: LAST Signal Incorrect

**Symptoms:** - Transaction ends early or late - Master receives wrong beat count

**Debug Steps:** 1. Check TRACK_BURSTS parameter (axi_data_dnsize); there is no USE_LAST 2. Verify burst tracker logic if enabled 3. Check s_last input timing

**Solution:**

```
// LAST is the buffered last flag AND the final beat -- from the RTL:
//   upsize:  wide_last   = r_last_buffered && r_wide_valid;
//   dnsize:  narrow_last = r_wide_buffered && r_last_buffered
//                          && w_last_narrow_beat;
// An OR of (final beat, saw-last) asserts LAST at the end of EVERY
// group and truncates any burst spanning more than one wide beat.
```

##### 31.2.1.1.3 Issue: Throughput Lower Than Expected

**Symptoms:** - Gaps between output beats - Single-buffer stalling between wide beats (it should replace during the last narrow beat)

**Debug Steps:** 1. Check downstream ready signal behavior 2. Look for backpressure stalls

#### *31.2.1.2 Protocol Converter Issues*

##### 31.2.1.2.1 Issue: Burst Decomposition Incorrect

**Symptoms:** - Wrong number of single transactions - Address increment wrong

**Debug Steps:** 1. Check burst type (FIXED, INCR, WRAP) 2. Verify size calculation 3. Check address increment logic

**Waveform Checkpoints:**

```
r_ar_addr / r_aw_addr - walk by (1 << size) per issued beat
r_ar_beat_count / r_aw_beat_count - count to arlen/awlen
m_axil_arvalid  - asserts for each decomposed beat
```

#### 31.2.1.2.2 Issue: Response Aggregation Wrong

**Symptoms:** - Wrong BRESP/RRESP value - Error not propagated correctly

**Debug Steps:** 1. Check worst-case response tracking 2. Verify response counter 3. Check RLAST generation

**Solution:**

```
// Accumulate per beat...
always_ff @(posedge aclk or negedge aresetn) begin
    ...
    else if (m_axil_rvalid && m_axil_rready)
        if (m_axil_rresp > r_r_resp_accum)
            r_r_resp_accum <= m_axil_rresp;
end

// ...and ALWAYS fold the live beat in at emission. The accumulator
// alone is the shipped bug (see 4.3.3): it does not yet contain the
// final beat's own response, so a single-beat error reports OKAY.
assign w_r_resp_worst = (m_axil_rresp > r_r_resp_accum) ? m_axil_rresp
                                                        :
r_r_resp_accum;
assign s_axi_rresp = (r_r_beat_count == r_r_len) ? w_r_resp_worst
                                                 : m_axil_rresp;
```

### 31.2.25.2.2 Debug Signals

#### *31.2.2.1 Recommended Internal Signals*

For width converters:

```
// Add debug outputs
output logic [$clog2(RATIO)-1:0] dbg_beat_count,
output logic                     dbg_buffer_valid,
output logic [1:0]               dbg_state
```

For protocol converters:

```
// Add debug outputs
output logic [7:0]  dbg_remaining_beats,
```

```
output logic [2:0]  dbg_state,
output logic [1:0]  dbg_worst_resp,
output logic        dbg_in_burst
```

#### *31.2.2.2 ILA Configuration*

```
# Create ILA for converter debug
create_debug_core u_ila ila

# Add probes
set_property probe_count 10 [get_debug_cores u_ila]
connect_debug_port u_ila/clk [get_nets aclk]

# Key signals
connect_debug_port u_ila/probe0 [get_nets r_beat_ptr]
connect_debug_port u_ila/probe1 [get_nets r_ar_beat_count]
connect_debug_port u_ila/probe2 [get_nets s_axi_arvalid]
connect_debug_port u_ila/probe3 [get_nets m_axil_arvalid]
```

### 31.2.3 5.2.3 Simulation Debug

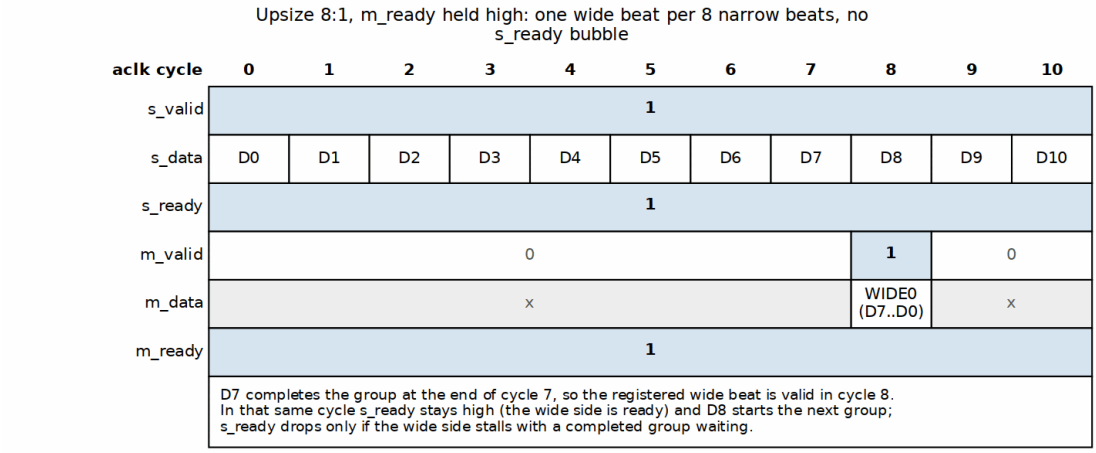
#### *31.2.3.1 Waveform Analysis*

**Key Signal Groups** (real port names — the width primitives are named by WIDTH, not direction: `narrow_*` is the INPUT of the upsize but the OUTPUT of the dnsize):

1. **Narrow side:**
   - narrow_valid, narrow_ready, narrow_data, narrow_sideband, narrow_last (upsize input / dnsize output)
2. **Wide side:**
   - wide_valid, wide_ready, wide_data, wide_sideband, wide_last (upsize output / dnsize input)
3. **Control:**
   - r_beat_ptr, r_slave_beat_count, r_burst_active (tracked mode), start_lane, burst_len/burst_start (dnsize tracked mode)
4. **Sideband contents:**
   - WSTRB rides narrow_/wide_sideband on the write path
   - RRESP rides narrow_/wide_sideband on the read path

#### *31.2.3.2 Reference Timing*

### 31.2.4 Figure 5.1: Upsize Reference Timing




*Upsize Reference Timing*

Compare a failing upsize waveform against this. Check:

1. `s_ready` stays high during accumulation (it drops only while a completed group waits on a stalled wide side)
2. `m_valid` is a one-cycle pulse per group, in the cycle after the RATIO-th narrow beat is accepted (with `m_ready` high)
3. Data packing is correct: D0 in the low lane, D(RATIO-1) in the high lane

### 31.2.5 5.2.4 Common Mistakes

#### *31.2.5.1 Mistake 1: Wrong Width Ratio*

```
// WRONG: Manual ratio
localparam RATIO = 8;  // May not match actual widths

// CORRECT: Calculated ratio
localparam RATIO = WIDE_WIDTH / NARROW_WIDTH;
```

#### *31.2.5.2 Mistake 2: Missing Sideband Handling*

```
// WRONG: Forgetting sideband
assign m_data = r_data;
// Missing: assign m_wstrb = ...

// CORRECT: Handle both
assign m_data = r_data;
assign m_wstrb = r_sideband;
```

#### 31.2.5.3 Mistake 3: Incorrect LAST Timing

```
// WRONG: LAST on wrong beat
assign narrow_last = (r_beat_ptr == 0);  // First slice!

// CORRECT: LAST on final beat
// AND of final-beat with the buffered last flag (see 5.2.1); the OR
// form truncates multi-wide-beat bursts
assign narrow_last = r_wide_buffered && r_last_buffered &&
w_last_narrow_beat;
```

#### 31.2.5.4 Mistake 4: Burst Length Calculation Error

```
// WRONG: Off-by-one
assign m_awlen = s_awlen / RATIO;  // Wrong!

// CORRECT: Account for LEN encoding
assign m_axi_awlen = ((int_awlen + 8'(WIDTH_RATIO)) / 8'(WIDTH_RATIO))
- 8'd1;  // round UP
```

### 31.2.6 5.2.5 Verification Checklist

Before signoff, verify:

- ☐ All parameter combinations tested
- ☐ Single-beat transactions work
- ☐ Multi-beat bursts work (INCR, WRAP, FIXED)
- ☐ Backpressure handling correct
- ☐ Error propagation correct
- ☐ LAST signal timing correct
- ☐ Sideband signals handled correctly
- ☐ Reset behavior verified
- ☐ Coverage targets met

### 31.2.7 5.2.6 Performance Validation

#### 31.2.7.1 Throughput Measurement

```
async def measure_throughput(tb, transaction_count=1000):
    start_time = get_sim_time()

    for _ in range(transaction_count):
        await tb.send_transaction()

    end_time = get_sim_time()
    elapsed_cycles = (end_time - start_time) / clock_period
```

```
    throughput = transaction_count / elapsed_cycles
    print(f"Throughput: {throughput:.2f} transactions/cycle")

    return throughput
```

#### *31.2.7.2 Expected Throughput*

*Table 5.5: Expected Throughput*

| Module | Mode | Expected |
|---|---|---|
| axi_data_upsize | Single | 1.0 trans/cycle |
| axi_data_dnsize | Single | 0.992 narrow beats/cycle (measured) |
| axi4_to_axil4 | Single-beat | ~1/(2+slave latency) trans/cycle (one-outstanding guard) |
| axi4_to_axil4 | Burst | ~N/(N+2) beats/cycle vs a pipelining slave (N issue cycles + one response latency, see 4.2.4); whole bursts serialize on the one-outstanding guard |

## 31.3 Navigation

**End of Micro-Architecture Specification**